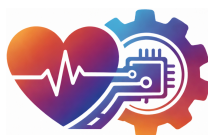




**UNIVERSIDAD
DE BURGOS**

Escuela Politécnica Superior
Grado en Ingeniería de la Salud



**TFG del Grado en Ingeniería de la
Salud**

**Aplicación web de cálculo de
indicadores en la URCCPQ**

Presentado por Nicolás José García Gómez
en Universidad de Burgos

4 de junio de 2026

Tutores: David García García – Cristina Arlanzón Pérez

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	vi
Apéndice A Plan de Proyecto	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Planificación económica	5
A.4. Viabilidad legal	8
Apéndice B Manual de especificación de diseño	15
B.1. Planos	15
B.2. Diseño arquitectónico	15
B.3. Arquitectura de Despliegue en Entorno Hospitalario (<i>HUBU</i>)	22
B.4. Arquitectura de Despliegue en Entorno Local	24
B.5. Arquitectura de Despliegue en Terminal Portátil (<i>Raspberry Pi</i>)	26
Apéndice C Especificación de Requisitos	31
C.1. Diagrama de casos de uso	32
C.2. Explicación casos de uso.	32
C.3. Prototipos de interfaz o interacción con el proyecto	37
Apéndice D Documentación de usuario	51
D.1. Requisitos software y hardware para ejecutar el proyecto. . .	51
D.2. Puesta en marcha	52

D.3. Manuales y/o Demostraciones prácticas	56
Apéndice E Manual del programador.	59
E.1. Estructura de directorios	59
E.2. Pruebas del sistema	65
E.3. Instrucciones para la modificación o mejora del proyecto . .	75
E.4. Vías de Implantación en la Infraestructura Hospitalaria . . .	88
Apéndice F Descripción de adquisición y tratamiento de datos	91
F.1. Descripción Formal de los Datos	91
F.2. Descripción Clínica de los Datos	94
Apéndice G Estudio experimental	99
G.1. Cuaderno de Trabajo y Evolución Experimental	99
G.2. Configuración y Parametrización del Sistema	102
G.3. Evaluación de Resultados y Usabilidad Clínica	104
Apéndice H Anexo de sostenibilización curricular	111
H.1. Introducción	111
Apéndice I Registro de interacciones con sistemas de IA	115
I.1. Registro de interacciones con sistemas de IA	115
Diccionario de Variables Clínicas	119
Bibliografía	121

Índice de figuras

A.1. Diagrama de <i>Gantt</i> inicial.	4
A.2. Diagrama de <i>Gantt</i> final.	5
B.1. Montaje inicial para pruebas del prototipo. El esquema de la placa se basa en [YoungWonks,].	16
B.2. Esquema del montaje final.	17
B.3. Prototipo final con todos sus elementos implementados.	18
B.4. Diagrama de flujo - Inicio de la <i>web</i>	19
B.5. Diagrama de flujo - Selección de indicadores.	20
B.6. Diagrama de flujo - Resumen.	21
B.9. Arquitectura de Despliegue.	23
B.10.Arquitectura de Despliegue en Entorno Local.	25
B.11.Arquitectura de Despliegue en Entorno Portátil.	27
B.7. Diagrama de flujo - Mezcla.	29
B.8. Diagrama de flujo - Interfaz de la <i>BBDD</i>	30
C.1. Diagrama <i>UML</i> del proyecto.	33
C.2. Pantalla principal de la <i>web</i>	43
C.3. Panel de selección lateral.	44
C.4. Indicador individual.	45
C.5. Mezcla de indicadores.	46
C.6. Resumen final de los indicadores.	47
C.7. Pantalla de inicio de sesión y validación de roles.	48
C.8. Interfaz unificada para la gestión y consulta de la <i>BBDD</i>	49
D.1. Acceso a la plataforma clínica mediante la introducción de la <i>IP</i> local en el navegador.	53

D.2. Panel de <i>Docker</i> monitorizando el estado activo (<i>Up</i>) de los contenedores que dan soporte a la plataforma en la <i>intranet</i> simulada.	54
D.3. Terminal portátil encendido mostrando la interfaz inicial tras el arranque automático.	55
E.1. Trazas de consola confirmando la creación de las imágenes . . .	66
E.2. Consola ejecutando la migración exitosa de la base de datos. . .	67
E.3. Consola mostrando todos los pasos necesarios para la importación de los registros iniciales.	67
E.4. Consola mostrando el arranque y detención correcto.	68
E.5. Botones en la interfaz de <i>Docker Desktop</i>	69
E.6. Borrado y reinicio del los contenedores.	69
E.7. Rechazo de formatos no válidos.	71
E.8. Gestión de la cola <i>FIFO</i>	72
E.9. Interrupción controlada del sistema ante la ausencia de columnas requeridas para el cálculo.	73
E.10. Número de historia incorrecto.	73
E.11. Fechas introducidas erróneas.	74
E.12. Cadena de texto corregida.	74
E.13. Registro de auditoria.	75
E.14. Definición del Método.	76
E.15. Validación de Variables Clínicas.	76
E.16. Cálculo del Indicador.	77
E.17. Exportación de los Datos.	78
E.18. Renderizado Final.	78
E.19. Inclusión en el Resumen.	79
E.20. Método mezcla en Programa.py.	80
E.21. Método mezcla en mezcla.py.	80
E.22. Método seleccion en seleccion.py.	81
E.23. Claves para su Renderizado.	82
E.24. Tabla Registro en el archivo Modelo.py.	83
E.25. Lista cabeceras.	84
E.26. Lista cabeceras_display.	84
E.27. Lista cabeceras_exportacion.	84
E.28. Lista orden_formulario.	85
E.29. Lista campos_display_fecha y campos_display_fecha_multiple.	85
E.30. Lista campos_display_numerico.	85
E.31. Lista campos_display_booleano.	86
E.32. Verificación N° Historia.	86

G.1. Respuesta SUS.	105
G.2. Respuesta SUS.	106
G.3. Respuesta SUS.	106
G.4. Gráfico individual.	107
G.5. Correlación de tres indicadores.	108
G.6. Resumen tabular de los indicadores calculados.	109
G.7. Gráfico poseedor de la regla de las 2 <i>Sigmas</i> incrustado en el informe final.	110

Índice de tablas

A.1. Desglose de costes de contratación y cotizaciones sociales actualizado.	6
A.2. Desglose de costes de los componentes físicos del terminal. . . .	8
A.3. Desglose de herramientas <i>software</i> empleadas y sus respectivas licencias.	12
C.1. Especificación del Caso de Uso CU-1.	36
C.2. Especificación del Caso de Uso CU-2.	37
C.3. Especificación del Caso de Uso CU-3.	38
C.4. Especificación del Caso de Uso CU-4.	39
C.5. Especificación del Caso de Uso CU-5.	40
C.6. Especificación del Caso de Uso CU-6.	41
C.7. Especificación del Caso de Uso CU-7.	42
I.1. Diccionario Detallado de Variables Clínicas del Sistema.	119

Apéndice A

Plan de Proyecto

A.1. Introducción

Antes de abordar la fase de ejecución, es fundamental dimensionar el proyecto en términos temporales, viabilidad económica y cumplimiento normativo. Una planificación estratégica adecuada es vital para mitigar riesgos tempranos, administrar los recursos de forma eficiente, y certificar la adecuación del sistema al marco jurídico actual.

A.2. Planificación temporal

Para la gestión integral y el seguimiento temporal del desarrollo se empleó la plataforma *GitHub* [Nicolás, 2026]. La metodología de trabajo dentro del repositorio se estructuró mediante la definición de tareas o incidencias (*issues*), las cuales fueron agrupadas lógicamente en hitos de progreso (*milestones*) con el objetivo de optimizar la trazabilidad de los avances.

A continuación, se presenta el desglose detallado de los hitos y tareas definidos para la consecución del proyecto:

1. Botón de subida.
 - Creación del botón físico de subida.
 - Conexión del botón con los datos.
 - Implementación lógica FIFO.

2. Lista desplegable de indicadores de calidad.

- Desplegable físico en la aplicación.
- Conexión de cada opción de los indicadores con su método.
- Mostrar los resultados correctamente.
- Creación de gráficos.

3. *Backend* de la aplicación.

- Normalización de los nombres de las columnas.
- Método de cálculo de cada indicador.
- Resumen de todos los indicadores.
- Exportación del proyecto.
- Tendencia de cada indicador.
- Informe final.
- Método de combinación de indicadores.

4. *Tablet* Médica.

- Montaje de los componentes.
- Introducción del SO.
- Funcionamiento de la herramienta en el dispositivo/arranque automático.

5. Diseño *web*.

- Adecuado diseño del botón de subida.
- Correcto diseño de la barra desplegable.
- Colocación del texto explicativo.
- Posicionamiento de las imágenes.
- Ordenación de los elementos gráficos.

6. Creación de la memoria y los anexos.

- Elaboración escrito comité de ética.
- 1. Introducción.
- 2. Objetivos.
- 3. Teóricos.

- 4. Metodología.
- 5. Resultados.
- 6. Discusión.
- 7. Conclusiones.
- 8. Lineas Futuras.
- A. Planificación.
- B. Diseño.
- C. Requisitos.
- D. Manual Usuario.
- E. Manual Programador.
- F. Datos.
- G. Experimental.
- H. *ODS*.
- I. *Prompts*.

7. BBDD

- Creación de botones sensibles al rol.
- Interfaz de carga de datos.
- Interfaz de edición de los registros.
- Exportación anónima de los registros en formato *CSV*.

8. Control de Acceso

- Inicio de sesión.
- Cambio de contraseña.
- Interfaz que permita la gestión de usuarios.
- Conexión de las funcionalidades a las web y *PostgreSQL* con la creación de un servicio de logs.

Diagrama de *Gantt*

Para la planificación temporal del proyecto, se elaboró un diagrama de *Gantt*. Esta herramienta permite visualizar los hitos del proyecto y su duración en el tiempo.

El cronograma preliminar del proyecto se detalla en la Figura A.1. Sin embargo, a medida que avanzaba el ciclo de vida del desarrollo, se identificaron desviaciones en las estimaciones temporales que evidenciaron la necesidad de reajustar los plazos. Estas desviaciones se atribuyen principalmente a la curva de aprendizaje inherente a la adopción de nuevas tecnologías y a la resolución de incidencias técnicas imprevistas. Como medida correctora para garantizar la viabilidad del sistema, se procedió a elaborar una replanificación ajustada a la realidad del proyecto, la cual se ilustra en la Figura A.2.

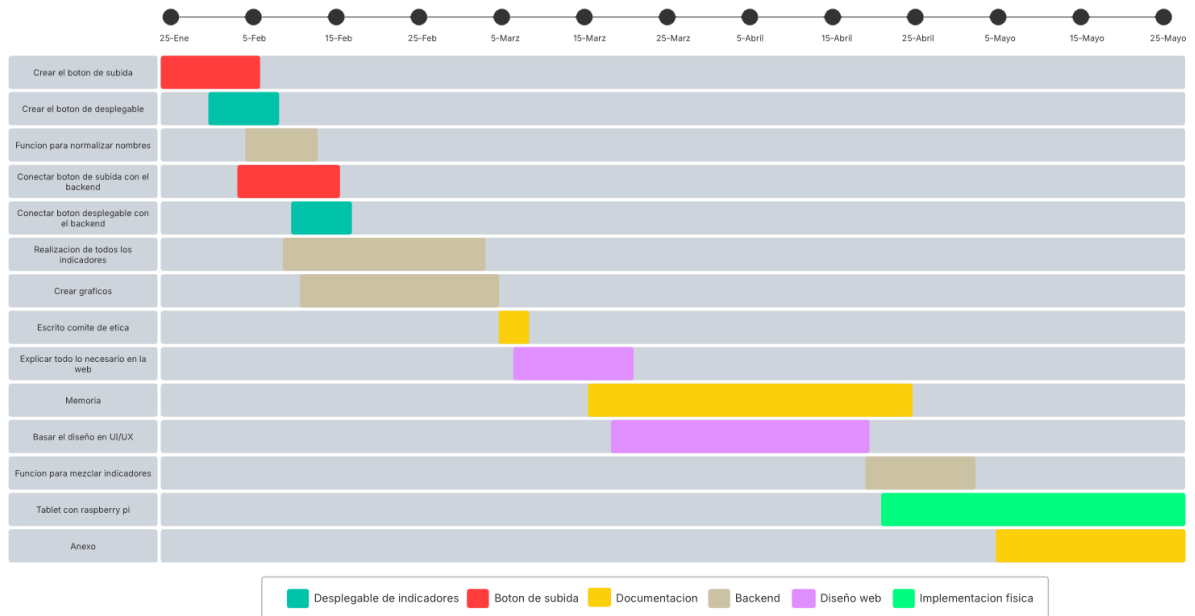


Figura A.1: Diagrama de *Gantt* inicial.

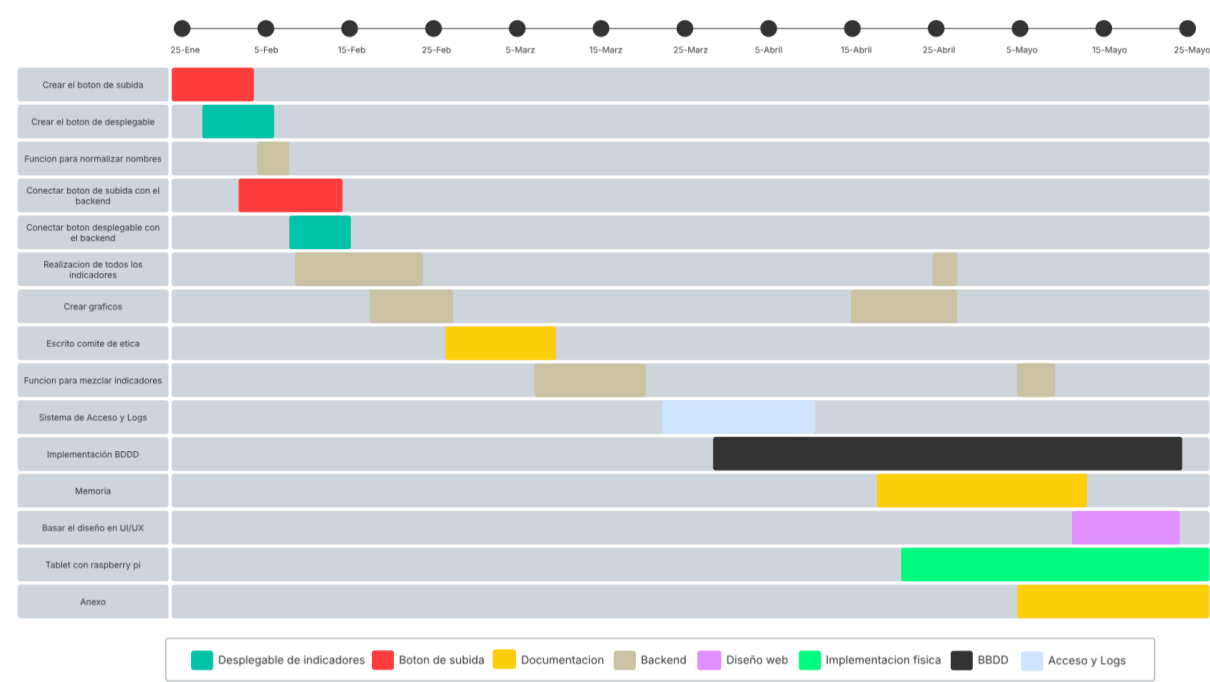


Figura A.2: Diagrama de *Gantt* final.

A.3. Planificación económica

Para llevar a cabo un análisis de viabilidad económica riguroso, es imperativo cuantificar la totalidad de los costes directos e indirectos asociados a la ejecución del proyecto. Esta estimación debe abarcar desde la valoración de licencias de *software* y la amortización de equipos informáticos, hasta la adquisición de componentes físicos y los honorarios correspondientes al capital humano involucrado.

En el presente apartado se desglosa el presupuesto detallado de la solución tecnológica desarrollada, contemplando de forma integral tanto los costes derivados de la ingeniería de *software* para la plataforma clínica *web*, como los recursos *hardware* requeridos para el ensamblaje del terminal portátil.

Coste de Personal

El diseño, desarrollo y despliegue integral de la plataforma han sido ejecutados por un único integrante. A efectos de la estimación presupuestaria, este rol se ha asimilado al perfil de un Ingeniero de la Salud o Ingeniero Bio-

médico en categoría junior (recién graduado). Según los datos salariales del sector publicados por la Universidad Alfonso X el Sabio (*UAX*) [[UAX, 2025](#)], la retribución media en España para este perfil oscila entre los 22.000 y 28.000 euros brutos anuales. Para el presente análisis de viabilidad, se ha adoptado una base conservadora de 25.000€ anuales.

Considerando que la carga temporal del proyecto equivale a un contrato de duración determinada de cuatro meses a jornada completa, el devengo salarial bruto del empleado asciende a 8.333,33€. A este importe es imperativo añadirle las cotizaciones sociales de carácter obligatorio que asume la empresa contratante. El cálculo de dichas partidas, basado en los tipos impositivos vigentes estipulados por el Régimen General de la Seguridad Social, se desglosa al detalle en la Tabla [A.1](#).

Concepto	Importe (€)
Sueldo bruto mensual	2.083,33
<i>Costes del Régimen General de la Seguridad Social (Empresa)</i>	
Contingencias Comunes (23,60 %)	491,67
Desempleo - Duración determinada (6,70 %)	139,58
FOGASA (0,20 %)	4,17
Formación Profesional (0,60 %)	12,50
Coste total mensual	2.731,25
Coste total del proyecto (4 meses)	10.925,01

Tabla A.1: Desglose de costes de contratación y cotizaciones sociales actualizado.

Coste de *Software*

Un aspecto fundamental en la viabilidad económica de este proyecto ha sido la adopción de un ecosistema tecnológico basado íntegramente en *software* libre y de código abierto (*Open Source*¹). Esta decisión estratégica ha permitido anular por completo los costes derivados de la adquisición de licencias comerciales o suscripciones de uso. Entre las tecnologías que han conformado este entorno de desarrollo gratuito destacan el lenguaje *Python* como núcleo del sistema, el *framework* *Reflex* para la orquestación de la plataforma clínica, *PostgreSQL* para la gestión de la base de datos relacional, librerías avanzadas de análisis y exportación de datos (*Pandas*, *Scikit-Learn*,

¹Es software cuyo código fuente es accesible, modificable y distribuible libremente por cualquiera

FPDF2), así como la plataforma *GitHub* para el control de versiones y el entorno *Texmaker* para la composición en *LaTeX* de la presente memoria.

Amortización de Equipos Informáticos

Para el desarrollo de la arquitectura de *software* ha sido indispensable el uso de equipamiento informático propio durante los cuatro meses que ha durado la ejecución del proyecto. Específicamente, el trabajo de programación se ha dividido en dos fases y plataformas: durante los dos primeros meses se empleó un ordenador portátil de la marca *MSI* (modelo *GL62 6QF*), cuyo precio de compra aproximado fue de 1.100€; mientras que durante los dos meses restantes el desarrollo se trasladó a un equipo informático de sobremesa valorado en 1.400€.

En el análisis presupuestario de un proyecto de ingeniería no se debe imputar el coste total de adquisición de las herramientas físicas, sino únicamente la fracción de su valor que se ha depreciado durante el tiempo efectivo de uso. Asumiendo una vida útil estimada de 5 años, que es el estándar contable habitual para equipos informáticos [ING, 2022], la depreciación se calcula mediante la fórmula de amortización lineal expresada en la Ecuación A.1:

$$\text{Amortización del Periodo} = \left(\frac{\text{Precio de Compra}}{\text{Vida Útil en Años}} \right) \times \left(\frac{\text{Meses de Uso}}{12} \right) \quad (\text{A.1})$$

Aplicando este modelo matemático al *hardware* utilizado, se desglosan los siguientes costes de desgaste imputables directamente al presupuesto del proyecto:

- **Amortización Portátil *MSI*:** $(1,100/5) \times (2/12) = 36,67 \text{ €}$
- **Amortización PC Sobremesa:** $(1,400/5) \times (2/12) = 46,67 \text{ €}$

En consecuencia, el coste total derivado de la amortización de las herramientas de desarrollo físico asciende a la cantidad de **83,34€**.

Costes de *Hardware* y Ensamblaje Físico

Para la materialización del terminal clínico portátil ha sido necesaria la adquisición de diversos componentes físicos y periféricos. Dada la naturaleza

del proyecto y su destino en un entorno hospitalario, se priorizó la fiabilidad, la originalidad de los componentes y los tiempos de entrega. Por ello, se descartó la importación a través de distribuidores internacionales de bajo coste con logísticas inestables, optando exclusivamente por proveedores oficiales y plataformas de comercio electrónico con garantías operativas en España como *Amazon* [[Amazon](#),] o distribuidores autorizados de *Raspberry Pi* [[shop](#),].

Cabe destacar un aspecto fundamental en la reducción de costes estructurales: en lugar de adquirir una carcasa comercial genérica o un chasis industrial (cuyo precio suele ser elevado), se optó por el diseño y la fabricación aditiva del encapsulado. Utilizando filamento de impresión 3D, se ha logrado construir una carcasa completamente a medida que integra a la perfección la placa base y la pantalla táctil, garantizando la ergonomía necesaria para su uso en la unidad de Reanimación (*URCCPQ*).

El desglose presupuestario de los elementos físicos que conforman el dispositivo se detalla en la Tabla [A.2](#):

Componente	Coste (€)
Placa base <i>Raspberry Pi 3 Model B</i>	38,50
Pantalla Oficial <i>Raspberry Pi LCD 7" Táctil</i>	75,00
Tarjeta de memoria <i>SanDisk MicroSD Ultra 32GB</i>	8,50
Set de cables puente (hembra-hembra)	3,00
Interruptor <i>SPST</i>	5,50
Batería de 6000 <i>mAh</i> a 3,7v	22,99
<i>Adafruit PowerBoost 1000C</i>	28,65
Bobina de filamento de impresión 3D (para carcasa)	15,00
Coste Total del Terminal Portátil	197,14

Tabla A.2: Desglose de costes de los componentes físicos del terminal.

A.4. Viabilidad legal

Todo desarrollo tecnológico exige un análisis exhaustivo de su encaje legal para asegurar la protección de los activos intangibles y el acatamiento de las regulaciones actuales. La legislación aplicable a esta índole de proyectos debe abarcar los estándares de uso y la protección integral de los datos del usuario. La adecuación a estos preceptos jurídicos es indispensable para certificar la seguridad clínica del producto final. En consecuencia, tomando

como base la jerarquía normativa europea y estatal, este apartado expone las directivas fundamentales que condicionan la viabilidad del presente trabajo.

Protección de datos personales y respeto de la privacidad

El proceso de seudonimización se llevará a cabo de forma estricta en origen y de manera exclusiva por el propio personal clínico responsable de la atención a los pacientes. Durante la fase de recolección y exportación de la información desde el sistema de la unidad, serán los propios facultativos quienes descarten y eliminen cualquier variable directa de identificación (tales como nombre, apellidos, DNI) antes de introducir la información en la base de datos implementada. De este modo, el software y el equipo investigador recibirán y procesarán únicamente un conjunto de datos completamente ciego, garantizando la privacidad absoluta desde el primer eslabón de la cadena.

Los registros clínicos seudonimizados son procesados exclusivamente dentro de la infraestructura segura de la *intranet* del hospital, garantizando que la información nunca sea transferida a redes externas. Tras la finalización del estudio, los datos permanecen almacenados como respaldo persistente en los servidores del centro, protegidos mediante estrictos protocolos de control de acceso y claves cifradas. Este sistema de almacenamiento asegura la disponibilidad histórica de la información para futuras auditorías médicas, manteniendo en todo momento la integridad y confidencialidad exigidas por la *LOPD* (Ley Orgánica 3/2018) [[Orgánica, 2018](#)] y en cumplimiento con el *RGPD* (Reglamento 2016/679 del Parlamento europeo) [[\(UE\), 679](#)].

Se respetarán los principios de la Declaración de *Helsinki* de 2024 de la Asociación Médica Mundial sobre principios éticos para las investigaciones en seres humanos. También se respetará la Ley 14/2007, de 3 de julio, de Investigación Biomédica que compete al tratarse de un estudio observacional sin medicamentos ni productos sanitarios, [[Biomedica, 2007](#)] así como la Ley 41/2002, de 14 de noviembre, básica reguladora de la autonomía del paciente y de derechos y obligaciones en materia de información y documentación clínica [[Paciente, 2002](#)].

También compete en este caso el Decreto 1093/2010, por el que se regula el uso y acceso a la historia clínica electrónica [[Decreto, 2010](#)]. El investigador principal garantizará que no se produzcan incumplimientos ni modificaciones del protocolo sin la aprobación por escrito del Comité de Ética de la Investigación con medicamentos (*CEIm*). Además, el investigador

principal asegurará que todo el personal que intervenga en la realización de este estudio sea informado de la obligación del cumplimiento de los citados compromisos.

Consentimiento Informado

Por último, en cuanto al Consentimiento Informado, se hará la exención de éste, fundamentada tanto en la relación beneficio/riesgo como en los siguientes aspectos:

1. **Uso de datos seudonimizados:** Los registros almacenados en *PostgreSQL* serán proporcionados en formato seudonimizado, garantizando que no contengan datos de filiación, lo que elimina cualquier riesgo de reidentificación durante el estudio. Los autores se comprometen al cumplimiento estricto de la ley de protección de datos *LOPD*, así como con el *RGPD* y garantizan su seguridad y confidencialidad.
2. **Ausencia de impacto directo en los pacientes:** El desarrollo y validación de la herramienta se realizará únicamente sobre informes clínicos históricos existentes, lo que no provocará ninguna intervención sobre el cuidado médico de los pacientes ni modificación de sus datos. Es un estudio observacional.
3. **Imposibilidad Numérica:** Dificultad existente para recabar el consentimiento informado debido al amplio número de pacientes.

Clasificación como Producto Sanitario y Comercialización

Dada la alta sensibilidad del entorno clínico en el que opera el terminal portátil, la plataforma debe someterse a estrictos controles que garanticen su seguridad, eficacia y fiabilidad. A nivel comunitario, el proyecto se enmarca dentro del **Reglamento 2017/745** sobre los productos sanitarios [(UE), 7745]. Esta normativa establece que los programas informáticos (*software*) pueden ser catalogados como productos sanitarios si están destinados a fines de diagnóstico, prevención, seguimiento o tratamiento de enfermedades. Dado que el sistema desarrollado automatiza el cálculo de indicadores *SEMICYUC* para el apoyo en la toma de decisiones médicas, su futura comercialización exigiría su certificación bajo este reglamento.

La obtención de esta conformidad se materializa a través del **Marcado CE**², requisito indispensable para la libre circulación y venta del dispositivo en la Unión Europea. A nivel nacional, la transposición de estas directivas se refleja en el **Real Decreto 192/2023** [Decreto, 2023], el cual otorga a la Agencia Española de Medicamentos y Productos Sanitarios (*AEMPS*) la potestad y responsabilidad de evaluar la clasificación del producto, así como de conceder o revocar las licencias de comercialización pertinentes.

Propiedad Intelectual y Derechos de Autor

La protección de los activos intangibles generados durante el desarrollo del *software* constituye un aspecto fundamental del marco normativo. Tanto la jurisdicción española como las directivas comunitarias europeas establecen un marco jurídico robusto para salvaguardar la autoría y los derechos de explotación del código fuente desarrollado:

- **Directiva (UE) 2009/24/CE:** Relativa a la protección jurídica de programas de ordenador [CE, 24CE]. Esta normativa consagra que el *software*, independientemente de su forma de expresión o de si se encuentra integrado en un componente *hardware*, goza de protección íntegra mediante derechos de autor. Asimismo, estipula los criterios de originalidad requeridos para su amparo legal.
- **Real Decreto Legislativo 1/1996:** Por el que se aprueba el texto refundido de la Ley de Propiedad Intelectual (*LPI*) [Decreto, 1996]. Constituye la norma central a nivel estatal en esta materia, tipificando las bases legales que regulan la autoría, los derechos morales inalienables y las condiciones de explotación patrimonial o cesión de los programas de ordenador desarrollados en territorio nacional.

Licencias de *Software* de Terceros

Una licencia de *software* es un instrumento jurídico que establece los derechos, restricciones y obligaciones que el autor original de un programa otorga a los usuarios que desean utilizar, modificar o distribuir su código. En el ámbito de la ingeniería informática, comprender la compatibilidad entre licencias es fundamental para garantizar que el ensamblaje de distintas librerías no vulnere la propiedad intelectual de sus creadores.

²Etiqueta obligatoria que indica que un producto cumple con los requisitos de seguridad, salud y protección medioambiental de la Unión Europea

Como se ha justificado en apartados anteriores, la totalidad de la arquitectura de este proyecto se sostiene sobre un ecosistema de código abierto (*Open Source*). En la Tabla A.3 se detallan las herramientas, librerías y *frameworks* que conforman la plataforma clínica, junto con la licencia bajo la cual operan y han sido integradas.

Categoría	Herramienta / Paquete	Tipo de Licencia
Entorno del Sistema	<i>Raspberry Pi OS</i>	GPL / Debian Free Software
Lenguaje Base	<i>Python 3.x</i>	PSF License
Infraestructura	<i>Docker / Docker Compose</i>	Apache License 2.0
<i>Frameworks</i> y Servidor	<i>Reflex</i>	Apache License 2.0
	<i>FastAPI / Uvicorn</i>	MIT License
Ciencia de Datos y <i>ML</i>	<i>Pandas, NumPy</i>	BSD License
	<i>Scikit-Learn</i>	BSD License
Gráficos e Informes	<i>Matplotlib</i>	PSF / BSD-like
	<i>FPDF2</i>	GNU LGPL v3
Persistencia y Seguridad	<i>PostgreSQL</i>	PostgreSQL License
	<i>Psycopg2-Binary</i>	LGPL v3
	<i>Alembic</i>	MIT License
	<i>Bcrypt</i>	Apache License 2.0
Desarrollo y Redacción	<i>Visual Studio Code</i>	MIT License
	<i>Texmaker (LaTeX)</i>	GNU GPL v2

Tabla A.3: Desglose de herramientas *software* empleadas y sus respectivas licencias.

Licenciamiento del Proyecto Desarrollado

La elección de la licencia para el *software* resultante de este Trabajo de Fin de Grado debe responder a dos premisas fundamentales: permitir su libre utilización y evolución por parte del Hospital Universitario de Burgos (*HUBU*), y proteger al desarrollador original frente a posibles contingencias médico-legales.

Bajo este contexto, se ha determinado liberar el código fuente de la plataforma bajo la **Apache License 2.0** [Licence, V 2]. Las razones que justifican esta decisión son las siguientes:

- **Carácter permisivo y fomento de la innovación:** Permite al hospital y a la comunidad científica utilizar, modificar y distribuir el

software con total libertad, sin obligar a que futuras integraciones (por ejemplo, con el sistema privado *ICCA*) tengan que ser necesariamente de código abierto.

- **Limitación explícita de responsabilidad:** El ámbito de la unidad de Reanimación (*URCCPQ*) es altamente crítico. La licencia *Apache 2.0* incluye una cláusula robusta que exime al autor de cualquier responsabilidad por daños derivados del uso de la herramienta, aclarando que el *software* se proporciona “tal cual” (*as is*) y sirve como apoyo estadístico, no sustituyendo el criterio médico del facultativo.
- **Protección contra litigios de patentes:** A diferencia de licencias más sencillas como la *MIT*, la directiva *Apache* protege a los contribuyentes frente a posibles reclamaciones de patentes en el sector del *software* médico, garantizando un marco seguro para la transferencia tecnológica.

Apéndice *B*

Manual de especificación de diseño

B.1. Planos

El montaje inicial consta simplemente de la conexión de la pantalla *LCD* a la *Raspberry Pi* mediante su respectivo *flex* y las conexiones indicadas por el fabricante. Se desarrolló para la prueba del dispositivo portátil y se presenta en la Figura [B.1](#).

Tras la comprobación de la viabilidad del dispositivo portátil, se realizaron ciertas mejoras con el objetivo principal de dirigir el dispositivo hacia un enfoque totalmente autónomo. Se implementó una batería de $6000mAh$, dicho método de almacenamiento, está regulada por un controlador *Adafruit Powerboost 1000C* destinado a la gestión y recarga de la energía del dispositivo. Todo ello emprende gracias a la inclusión de un interruptor *SPDT* conectado al *pin EN* del *Powerboost 1000C*. Las conexiones son extremadamente simples, tomando como partida el montaje anterior, solo se necesita conectar el controlador y la pantalla (con su respectivo *flex*) a las fuentes de $5V$ de la placa. Se representa en la Figura [B.2](#).

El prototipo final, ilustrado en la Figura [B.3](#), integra todos los componentes de *hardware* esenciales para facilitar su manejo.

B.2. Diseño arquitectónico

Dada la naturaleza colaborativa de la *URCCPQ* y la necesidad de cumplir con la estricta normativa de protección de datos (*LOPD*), la arquitectura de

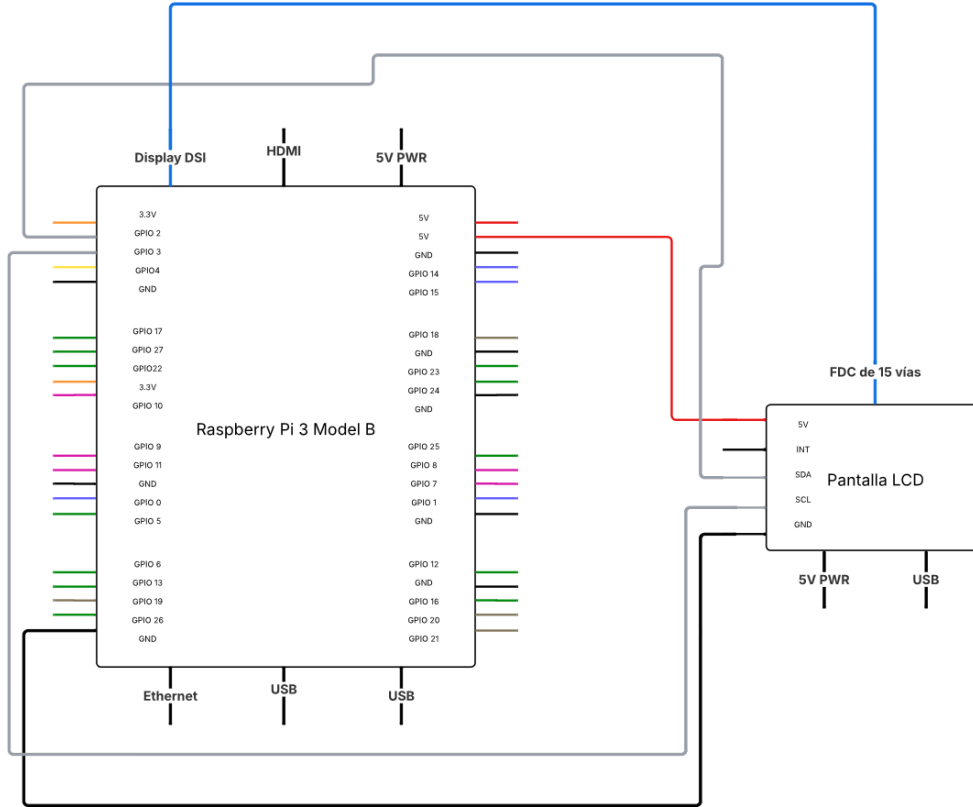


Figura B.1: Montaje inicial para pruebas del prototipo. El esquema de la placa se basa en [YoungWonks,].

permisos del sistema se ha diseñado bajo un modelo de Control de Acceso Basado en Roles (*RBAC*). A diferencia de los sistemas sin autenticación, esta herramienta implementa una segmentación jerárquica mediante perfiles diferenciados (como *Administrador* y *Facultativo*). Esta decisión técnica, gestionada a través de la base de datos *PostgreSQL*, garantiza que los médicos residentes mantengan un acceso ágil y sin fricciones para la inserción y visualización de registros clínicos, mientras que las operaciones críticas como la exportación masiva del histórico o la gestión de credenciales, quedan restringidas al personal coordinador. De esta forma, se equilibra la operatividad en un entorno de alta demanda con la trazabilidad y la seguridad institucional.

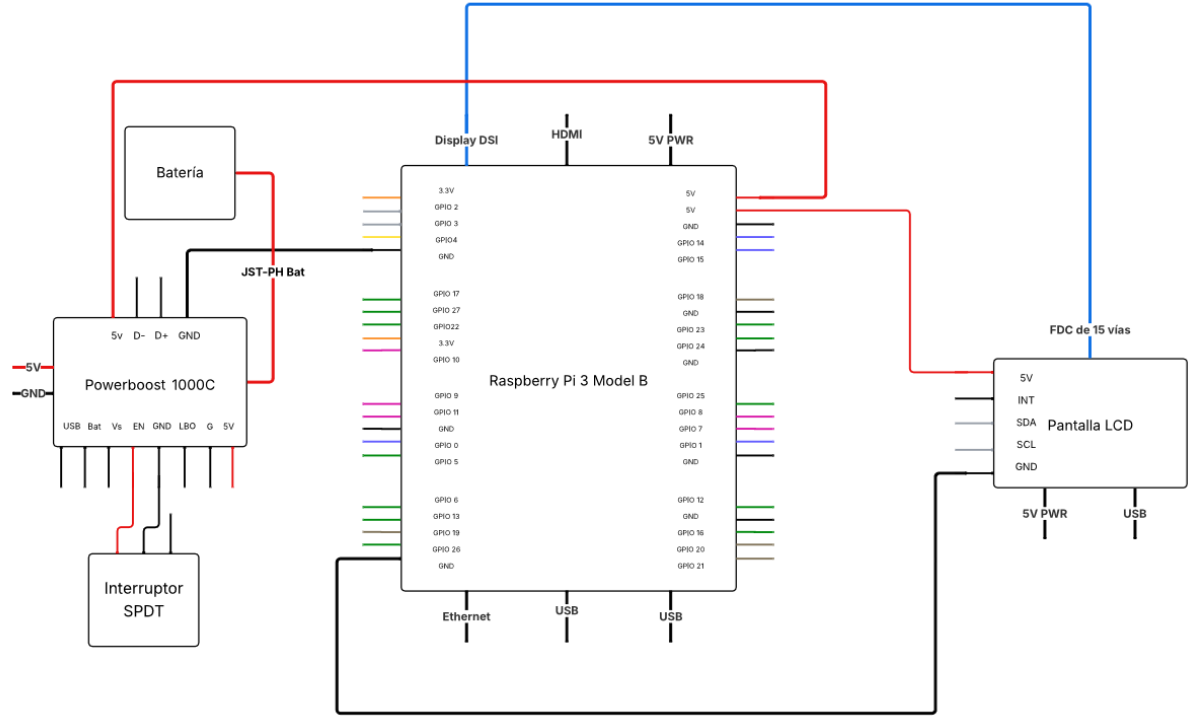


Figura B.2: Esquema del montaje final.

Para documentar la Experiencia de Usuario (*UX*) [Yablonski, 2022] y las rutas de navegación de forma clara y legible, se ha descartado la elaboración de un único diagrama de flujo monolítico, el cual resultaría visualmente incomprensible debido a la alta densidad de opciones de la plataforma. En su lugar, el mapa de interacciones se ha estructurado mediante un diseño modular, implementando una distinción semántica específica para clarificar las decisiones: los **rombos** representan decisiones lógicas automatizadas por el motor del sistema (validaciones de formato, conteo de archivos), mientras que los **nodos circulares de bifurcación** ($\oplus$) representan los puntos de interacción donde el facultativo médico ejerce una decisión de enrutamiento a través de la interfaz (selección de herramientas o menús). Dividiendo el sistema en los siguientes cinco flujos funcionales de alto nivel:

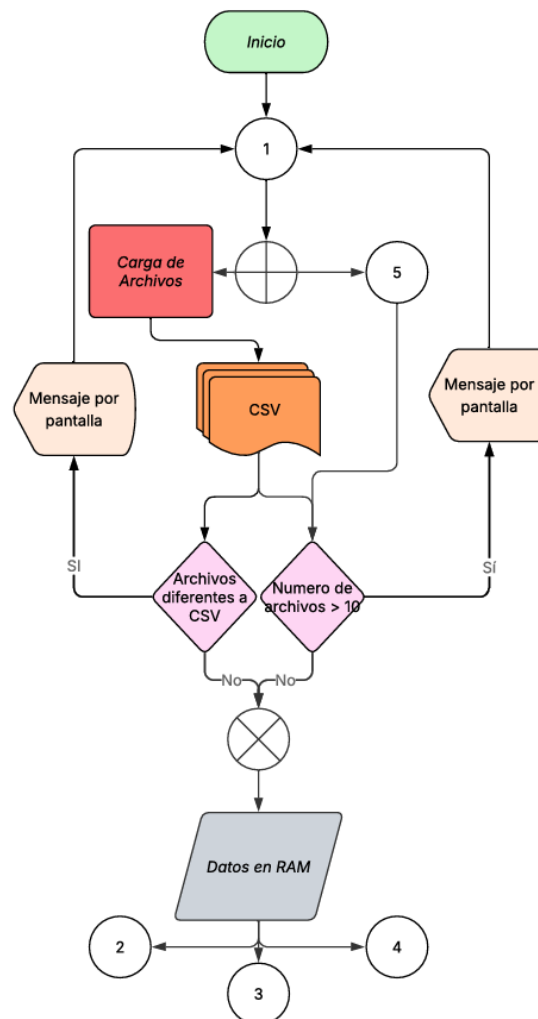
- **B.4 - Carga y Validación (Nodo 1):** Constituye la puerta de entrada a la plataforma. El sistema requiere la ingesta de registros ya sean en archivos físicos *CSV* o desde la base de datos. Superadas



Figura B.3: Prototipo final con todos sus elementos implementados.

las validaciones, la información se almacena en la memoria *RAM* del equipo, habilitando el acceso a las tres herramientas analíticas principales (Nodos 2, 3 y 4).

- **B.5 - Análisis Individual (Nodo 2):** Este flujo permite al usuario limpiar la sesión o profundizar en un indicador clínico específico. Una vez seleccionado el indicador, el usuario puede bifurcar la visualización hacia un Gráfico Lineal o un Gráfico de Barras, culminando el proceso con la exportación y descarga de los cálculos estadísticos.
- **B.6 - Resumen de Indicadores (Nodo 3):** Orientado a la gestión global, esta rama genera un resumen de la unidad clínica. El flujo guía al usuario a través de una tabla resumen y una distribución de

Figura B.4: Diagrama de flujo - Inicio de la *web*.

ingresos por especialidad médica, permitiendo finalmente descargar el reporte final en formato documento o extraer las tablas específicas.

- **B.7 - Comparativa Avanzada (Nodo 4):** Representa el módulo más complejo de interacción. El flujo *Mezcla* permite al facultativo añadir métricas iterativamente mediante un bucle de control (hasta un máximo de tres indicadores superpuestos) o eliminarlos en tiempo real. Esta iteración finaliza en la selección del tipo de gráfico comparativo, facilitando el análisis cruzado de datos.

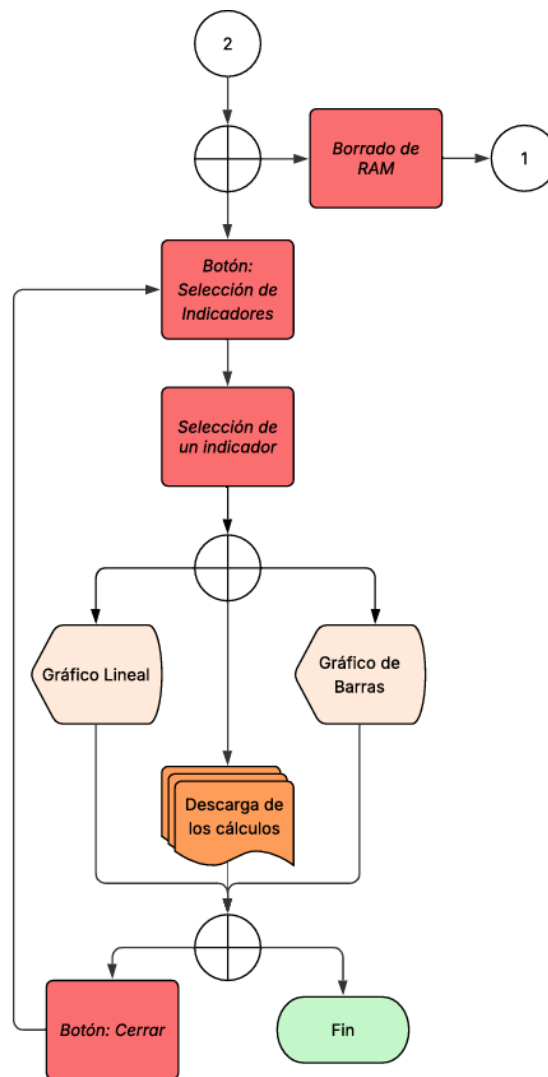


Figura B.5: Diagrama de flujo - Selección de indicadores.

- B.8 - Gestión y Persistencia (Nodo 5):** Constituye el núcleo de interacción con el sistema de almacenamiento *PostgreSQL*. A partir de este punto, el flujo lógico se bifurca en tres ramificaciones operativas independientes. La primera permite la adición directa de nuevos registros clínicos al servidor. La segunda ruta exige obligatoriamente una búsqueda previa del paciente, garantizando la consistencia antes de habilitar las funciones de edición o borrado que modifican la base de datos. Finalmente, la tercera ramificación gestiona la consulta del histórico mediante la selección de años, abriendo una nueva compuerta

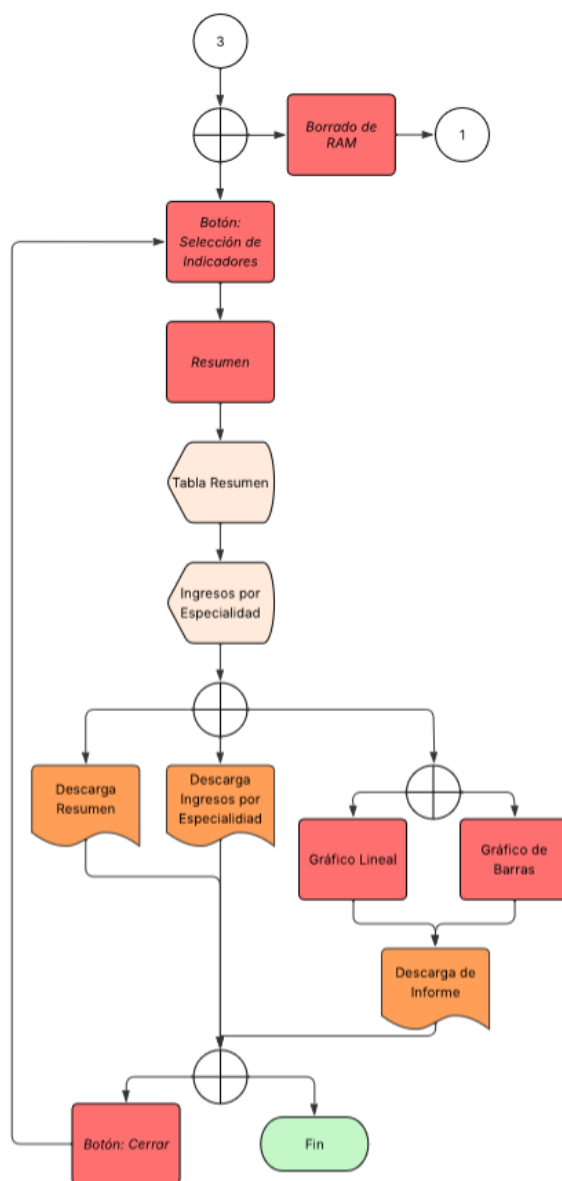


Figura B.6: Diagrama de flujo - Resumen.

que permite derivar la información hacia dos destinos: su carga volátil en la herramienta para el análisis estadístico, o su exportación masiva, dando por concluido el proceso.

Control de Versiones

Para el desarrollo del *software* se ha seguido una metodología iterativa e incremental, apoyada en el sistema de control de versiones *Git* y alojada en un repositorio de *GitHub*. A lo largo del ciclo de vida del proyecto, la plataforma ha evolucionado a través de varias versiones principales (ej. Releases v1.0, v2.0 y v3.0), añadiendo en cada iteración mayor robustez al motor matemático y nuevas funcionalidades a la interfaz gráfica. Los diagramas mostrados hacen referencia a las últimas versiones desarrolladas que permiten alcanzar la estabilidad de la arquitectura final.

B.3. Arquitectura de Despliegue en Entorno Hospitalario (*HUBU*)

El despliegue de la “Calculadora de Indicadores” se ha diseñado siguiendo una arquitectura de *microservicios* basada en contenedores, optimizada para integrarse de forma segura en la infraestructura interna de un centro sanitario (*intranet*). Este modelo garantiza el aislamiento de la aplicación, la seguridad de los datos clínicos y la facilidad de mantenimiento. Este despliegue requiere de un código alternativo al que se va a explicar mas adelante siendo accesible en el siguiente enlace: [TFG-2026-Nicolas-Garcia-Gomez-Produccion](#). La arquitectura se divide en tres capas funcionales descritas a continuación y representadas en el siguiente esquema B.9:

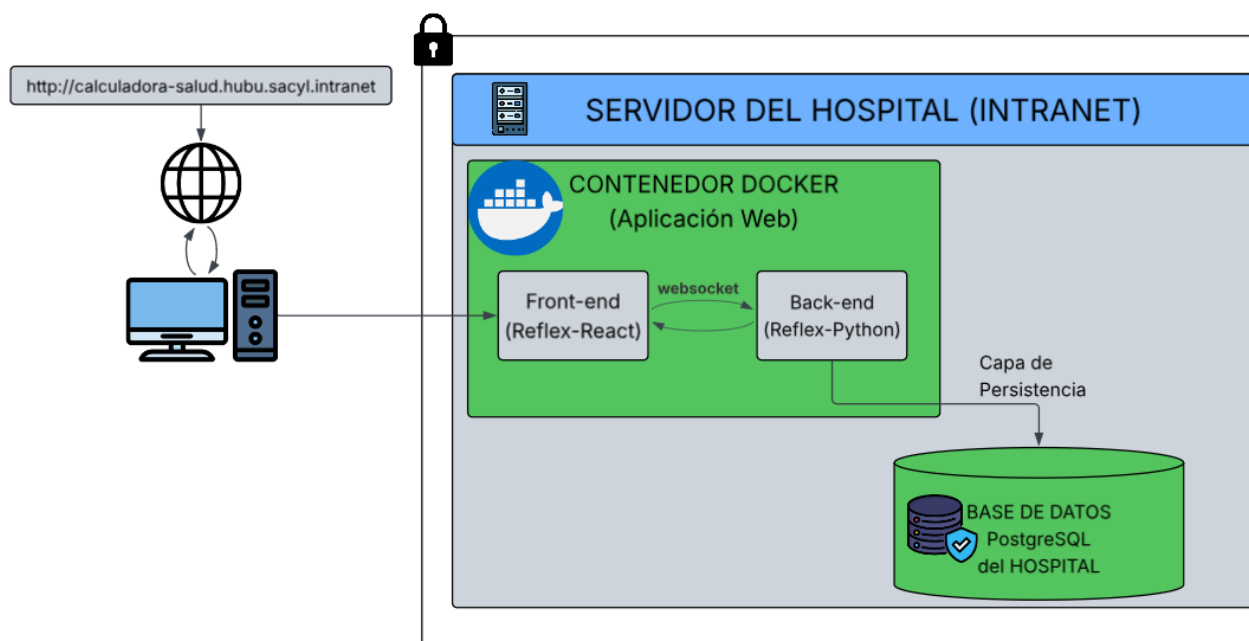


Figura B.9: Arquitectura de Despliegue.

1. **Capa de Acceso de Usuarios (Cliente):** El personal médico accede a la herramienta de forma transparente a través de un navegador *web* estándar, utilizando una *URL* interna proporcionada por la red del hospital (ej. `http://calculadora-salud.hubu.sacyl.intranet`). Este enfoque *zero-install* elimina la necesidad de instalar *software* local en los equipos de las consultas, centralizando las actualizaciones y reduciendo los puntos de vulnerabilidad. Las peticiones se dirigen a los puertos estándar del servidor del hospital.
2. **Capa de Aplicación (Contenedor *Docker*¹):** El núcleo lógico y visual de la calculadora reside en un único contenedor *Docker* alojado en el **Servidor del Hospital**. Este contenedor aísla el entorno de ejecución y encapsula las dos piezas fundamentales del *framework Reflex*:
 - **Frontend (*Reflex-React*):** Encargado de renderizar la interfaz de usuario de forma reactiva.

¹Permite empaquetar una aplicación con todo lo necesario (código, dependencias, librerías) para que funcione de manera idéntica en cualquier servidor o máquina.

- **Backend (*Reflex-Python*):** Encargado de procesar la lógica de negocio, las validaciones de grado clínico y la limpieza de datos.
 - Ambos componentes se comunican en tiempo real mediante un puente virtual interno, implementado a través del *WebSockets*, garantizando una latencia mínima en la interacción.
3. **Capa de Persistencia de Datos (*PostgreSQL*):** Para el almacenamiento robusto de los registros médicos, la aplicación no utiliza bases de datos volátiles ni locales dentro del contenedor. En su lugar, el *backend* de *Python* establece una conexión directa con la base de datos *PostgreSQL* oficial gestionada por el hospital.
- **Seguridad de Credenciales:** Para cumplir con el **Esquema Nacional de Seguridad (*ENS*)** [311/2022, 2022] y las normativas de protección de datos, las credenciales de acceso a la base de datos no están codificadas en el código fuente. Se inyectan de forma segura en el contenedor en el momento del arranque mediante variables de entorno.
 - **Integración *ORM*²:** Las operaciones de lectura, escritura y auditoría se realizan utilizando el *ORM* integrado, lo que previene ataques y asegura la coherencia e integridad de los tipos de datos introducidos por los facultativos.

B.4. Arquitectura de Despliegue en Entorno Local

Para las fases de desarrollo, validación técnica y demostración de la “Calculadora de Indicadores”, se ha diseñado una arquitectura de despliegue local ejecutada sobre un ordenador personal. Este modelo replica fielmente la lógica del entorno de producción hospitalario mediante tecnología de contenedores, pero condensa los servicios para permitir una ejecución autónoma y controlada. La arquitectura se divide en tres capas funcionales representadas en el siguiente esquema B.10:

²Capa de abstracción que traduce dinámicamente los objetos y funciones de Python en consultas seguras de PostgreSQL, eliminando la necesidad de escribir código SQL nativo.

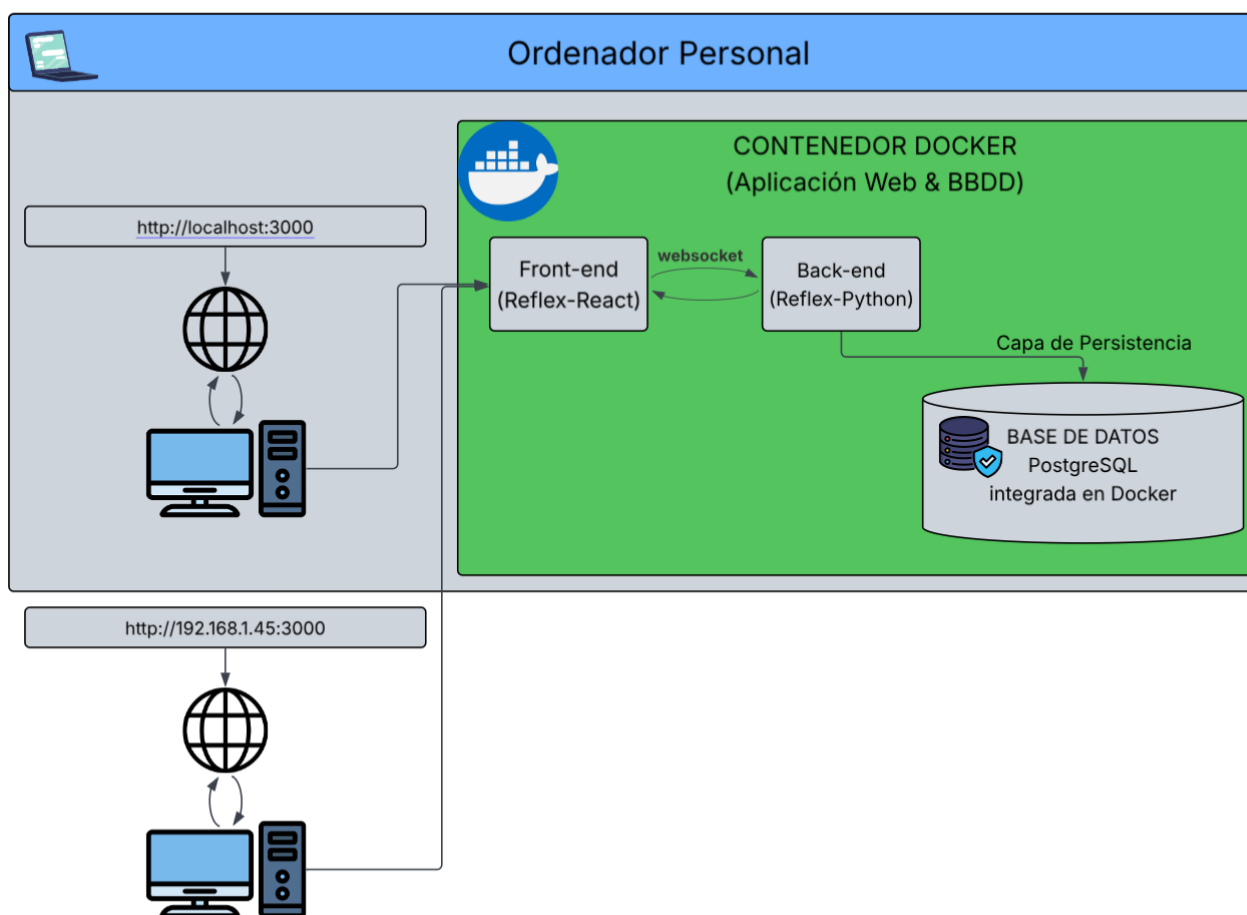


Figura B.10: Arquitectura de Despliegue en Entorno Local.

1. **Capa de Acceso de Usuarios (Cliente):** En este entorno, el acceso a la interfaz se realiza a través de un navegador *web* estándar. Al tratarse de una implementación local, existen dos vías de acceso principales:

- **Acceso Local (*Loopback*³):** Desde el propio equipo anfitrión donde se ejecutan los contenedores, utilizando la dirección `http://localhost:3000`.
- **Acceso en Red Local (*LAN*):** Desde otros dispositivos autorizados conectados a la misma red (como una *tablet* médica o portátil de planta), introduciendo la dirección *IP* privada del equipo (ej. `http://192.168.1.45:3000`).

³Término que se utiliza en diferentes áreas de la tecnología para describir un proceso en el que una señal o dato se envía de vuelta a su origen.

2. **Capa de Aplicación (Contenedor *Docker*):** El núcleo de la aplicación se ejecuta de forma aislada dentro de un contenedor. Este encapsula las dos piezas fundamentales del *framework Reflex*.
3. **Capa de Persistencia de Datos (*PostgreSQL* Integrada):** A diferencia del entorno hospitalario, donde la base de datos es externa y gestionada por el centro, en el entorno de pruebas el motor de *PostgreSQL* se despliega de forma integrada en el ecosistema *Docker* del equipo.
 - **Aislamiento y Pruebas de Estrés:** Esta configuración permite disponer de un entorno de datos controlado para realizar migraciones de tablas (*Alembic*), pruebas de carga y simulaciones de ingesta de archivos *CSV* sin riesgo de alterar sistemas externos o comprometer datos reales.
 - **Integración *ORM*:** El *backend* interactúa con esta base de datos local de forma transparente, asegurando que el código desarrollado sea idéntico al que se utilizará finalmente en la infraestructura del hospital.

B.5. Arquitectura de Despliegue en Terminal Portátil (*Raspberry Pi*)

Como variante orientada a la movilidad y la autonomía asistencial, se ha diseñado un despliegue específico sobre un dispositivo *Raspberry Pi*. Esta arquitectura está optimizada para funcionar como un terminal de diagnóstico rápido o “punto de atención”, donde prima la inmediatez y la sencillez de uso sobre la persistencia de datos a largo plazo. A diferencia de los modelos anteriores, esta configuración prescinde de un motor de base de datos relacional, operando en un modo de análisis volátil. Este despliegue requiere de un código alternativo siendo accesible en el siguiente enlace: [TFG-2026-Nicolas-Garcia-Gomez-Portatil](#) y su arquitectura se representa en el siguiente esquema B.11.

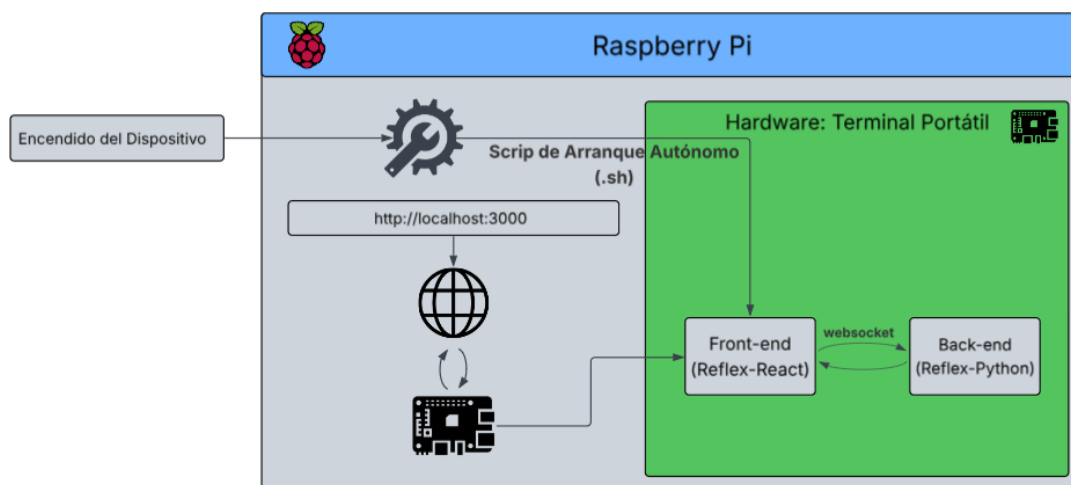


Figura B.11: Arquitectura de Despliegue en Entorno Portátil.

1. **Automatización y Gestión del Ciclo de Vida (*Script .sh*):** Para garantizar una experiencia de usuario *Plug & Play*, el sistema cuenta con un proceso de arranque automatizado. Mediante un *script* de *shell* (*.sh*)[de *shell*,] configurado en el gestor de servicios del sistema operativo (*Raspberry Pi OS*), la aplicación se inicia de forma transparente al encender el dispositivo. Este *script* se encarga de:
 - Verificar la integridad del entorno virtual de *Python*.
 - Levantar los servicios de *Frontend* y *Backend* sin intervención humana.
 - Lanzar el navegador nativo en modo “kiosco” (pantalla completa) apuntando a la dirección local.
2. **Procesamiento Efímero y Análisis en Memoria (*RAM*):** En esta arquitectura, la aplicación opera de forma puramente transaccional. El flujo de trabajo se centra exclusivamente en la subida de archivos *CSV* por parte del facultativo.
 - Los datos se cargan directamente en la memoria *RAM* del dispositivo para su procesamiento estadístico instantáneo.
 - Se mantiene la potencia analítica completa del motor de *Python*, permitiendo la generación de gráficas de control y el cálculo de indicadores de seguridad en tiempo real.

3. **Gestión de Datos sin Persistencia:** Al no contar con una capa de base de datos (*PostgreSQL*), el sistema garantiza la máxima privacidad de la información clínica en entornos fuera de la *intranet* hospitalaria segura. Una vez finalizada la sesión o reiniciado el dispositivo, los datos procesados desaparecen de la memoria volátil.
- Este enfoque elimina los riesgos de seguridad física asociados al robo o pérdida del dispositivo portátil, ya que no se almacena información sensible en el disco de la *Raspberry Pi*.
 - El resultado final del análisis puede ser exportado por el médico mediante la generación del informe *PDF*, sirviendo este como el único entregable permanente de la sesión.

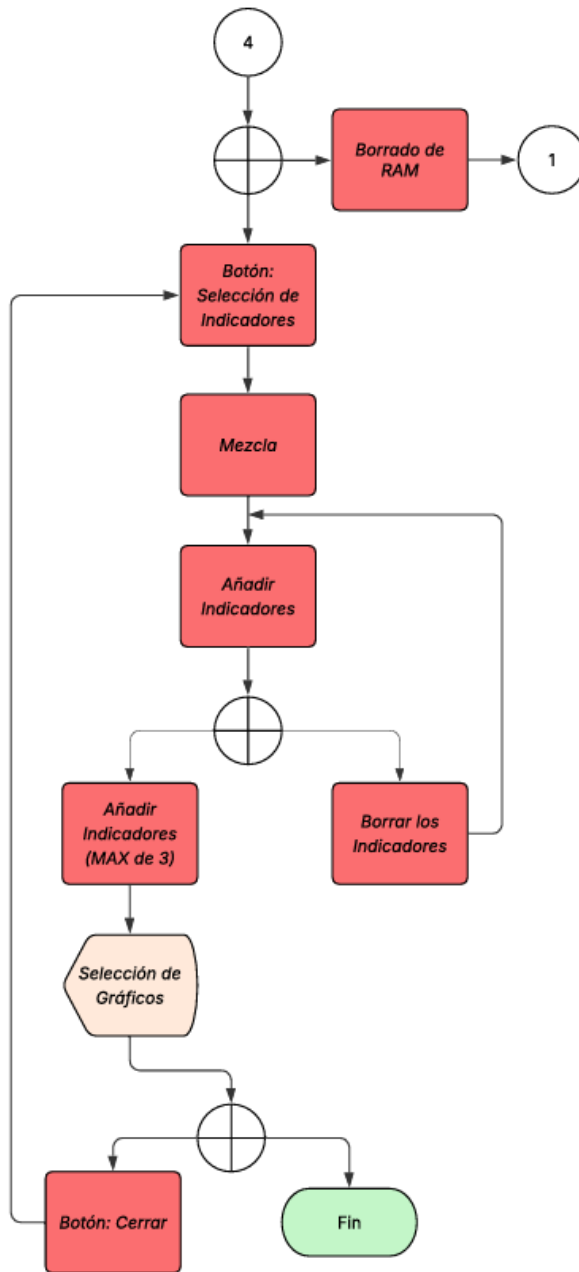


Figura B.7: Diagrama de flujo - Mezcla.

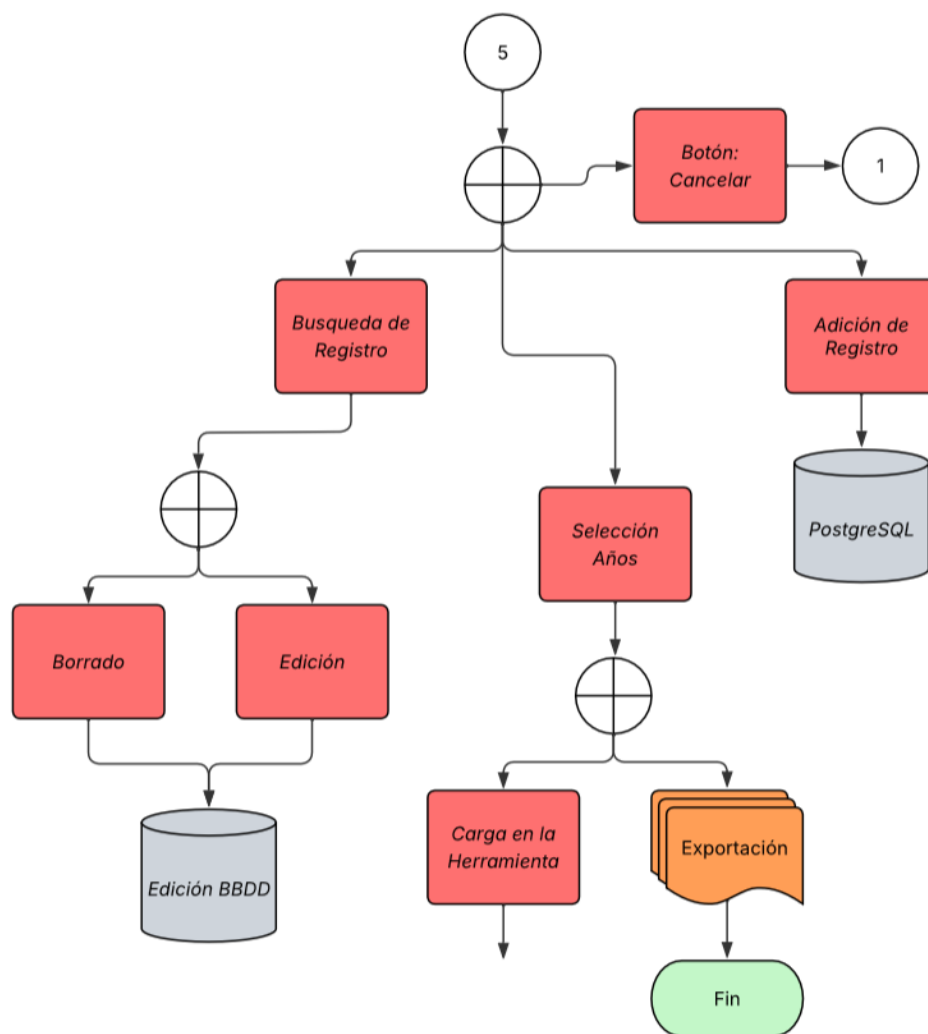


Figura B.8: Diagrama de flujo - Interfaz de la BBDD.

Apéndice C

Especificación de Requisitos

En la disciplina de la Ingeniería de *Software*, un actor o *stakeholder* se define como cualquier entidad, persona u organización que interactúa de forma directa o indirecta con el sistema para alcanzar un objetivo específico [Sommerville, 2005]. La correcta identificación de estos actores es el paso preliminar esencial para la especificación de los requisitos.

A diferencia de los planteamientos iniciales basados en arquitecturas de acceso unificado, el sistema desarrollado para la *URCCPQ* implementa un modelo de **Control de Acceso Basado en Roles (RBAC)**. Esta segmentación es fundamental para garantizar la trazabilidad clínica, proteger la integridad de la base de datos y cumplir estrictamente con la *LOPD*, definiendo tres perfiles de usuario estructurados jerárquicamente:

- **Nivel 1 - Visualización y Análisis:** Destinado al personal sanitario general. Permite el acceso de solo lectura a los registros clínicos, así como la ejecución de los algoritmos estadísticos y la visualización de los indicadores *SEMICYUC*, asegurando la consulta ágil de la información sin riesgo de alteración accidental de los datos.
- **Nivel 2 - Edición Clínica:** Dirigido a los médicos y residentes encargados de la recolección activa de datos. Hereda los permisos del nivel 1 e incorpora privilegios completos de escritura, posibilitando la búsqueda, adición, modificación y borrado de registros de pacientes directamente en la base de datos *PostgreSQL*.
- **Nivel 3 - Administración (Jefatura e Informática):** Constituye el perfil de máximo privilegio, reservado para la jefatura de la unidad

y el departamento de soporte técnico. Además de poseer control total sobre las herramientas operativas, es el único rol autorizado para ejecutar la exportación masiva del histórico clínico en formato *CSV* completamente anonimizado, posibilitando la auditoría y el análisis externo de forma segura.

Un caso de uso representa una descripción estructurada de las interacciones entre el sistema y sus actores, modelando las funciones o servicios que la plataforma proporciona desde el punto de vista del usuario [Jacobson, 1992]. En lugar de atomizar la documentación en micro-interacciones de interfaz, el diseño de este proyecto se ha enfocado en modelar los procesos críticos de valor clínico. Se han definido **Siete Casos de Usos** de alto nivel que abarcan la totalidad del flujo de trabajo en la plataforma.

C.1. Diagrama de casos de uso

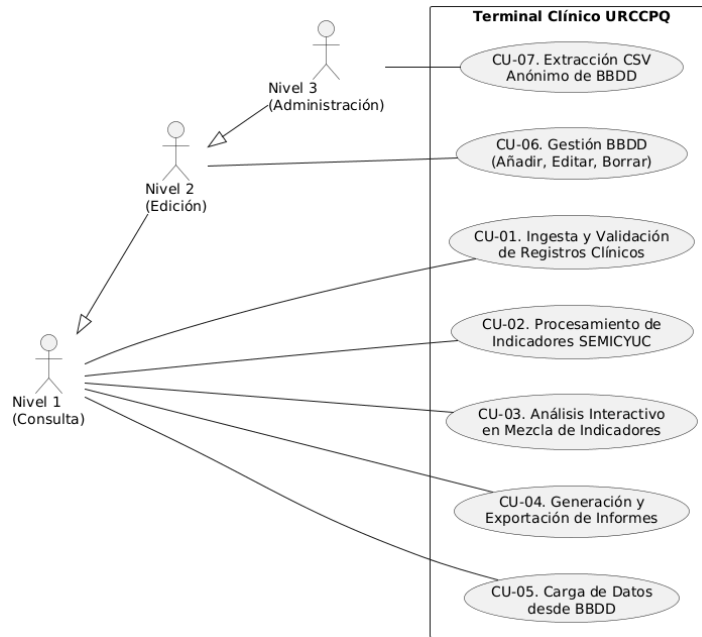
Un diagrama *UML* [Mediavilla,] de casos de uso ofrece un gran beneficio al definir con claridad los requisitos funcionales y el alcance del sistema desde la perspectiva del usuario. Facilita la comunicación entre equipos técnicos y clientes, simplifica sistemas complejos, ayuda en la estimación de proyectos y reutiliza funcionalidades, mejorando la mantenibilidad. En la Figura C.1 se representa el diagrama *UML* de la herramienta desarrollada en este proyecto.

C.2. Explicación casos de uso.

Las **tablas de explicación de casos de usos** [Vega, 2010] sirven para detallar el funcionamiento de un sistema, yendo más allá del diagrama gráfico para definir el flujo paso a paso, actores, condiciones y excepciones. Son cruciales en la fase de análisis para asegurar que desarrolladores y clientes tengan la misma visión antes de programar, facilitando la creación de pruebas.

Requisitos

Los requisitos funcionales describen el comportamiento esperado del *software* frente a las interacciones del facultativo y las necesidades de procesamiento de datos clínicos.

Figura C.1: Diagrama *UML* del proyecto.

- **RF-01:** El sistema debe permitir la carga de archivos de monitorización clínica desde el almacenamiento local del dispositivo.
- **RF-02:** El sistema debe validar estructuralmente los datos de entrada, requiriendo estrictamente formato *.csv* y limitando la ingesta a un máximo de 10 archivos simultáneos para prevenir desbordamientos de memoria.
- **RF-03:** El sistema debe integrar un motor algorítmico capaz de procesar la información y calcular los indicadores de calidad y seguridad definidos por la *SEMICYUC*.
- **RF-04:** El sistema debe proveer un cuadro de mandos (*dashboard*) interactivo que renderice los resultados matemáticos mediante gráficos lineales y de barras.
- **RF-05:** El sistema debe habilitar una herramienta de mezcla visual que permita la superposición paramétrica de hasta tres indicadores estadísticos para su análisis comparativo cruzado.
- **RF-06:** El sistema debe facultar la exportación de la sesión clínica, permitiendo empaquetar y descargar los cálculos y reportes generados al disco duro local.

- **RF-07:** El sistema debe implementar un modelo de **Control de Acceso Basado en Roles (RBAC)**, definiendo y restringiendo operativamente los niveles de visualización (Nivel 1), edición clínica (Nivel 2) y administración (Nivel 3).
- **RF-08:** El sistema debe permitir la consulta y carga de registros clínicos directamente desde la base de datos relacional (*PostgreSQL*) hacia la memoria volátil de la aplicación, habilitando el análisis del histórico para todos los niveles de usuario.
- **RF-09:** El sistema debe exigir obligatoriamente la ejecución de una búsqueda previa de pacientes en la base de datos, evaluando su pre-existencia antes de habilitar cualquier acción de modificación.
- **RF-10:** El sistema debe facultar a los usuarios con privilegios de Nivel 2 o superior para gestionar activamente la información, permitiendo añadir nuevos registros, así como editar o borrar pacientes existentes asegurando la integridad referencial.
- **RF-11:** El sistema debe autorizar exclusivamente a los administradores (Nivel 3) a realizar extracciones masivas del histórico clínico desde la base de datos hacia formato *CSV*, aplicando un proceso de anonimización algorítmico automático para garantizar el cumplimiento estricto de la *LOPD*.

Requisitos de *Hardware* y Autonomía

Para garantizar la operabilidad del terminal en los distintos boxes de la unidad de Reanimación, se han definido los siguientes requisitos físicos y de gestión energética:

- **RF-12 (Autonomía Energética):** El dispositivo debe ser completamente autónomo, permitiendo el desplazamiento del facultativo sin dependencia de cables. Para ello, debe integrar una batería de polímero de litio (*LiPo*¹) de gran capacidad (6000mAh a 3.7V).
- **RF-13 (Gestión de Carga e Interrupción):** El sistema debe incorporar un módulo de gestión de energía (*Adafruit PowerBoost*

¹Es una batería recargable de alta densidad energética y peso ligero

1000C) que permita la recarga de la batería mediante un puerto *Micro-USB* y garantice un flujo estable de 5.2V a la *Raspberry Pi*, incluso durante el proceso de carga (*load-sharing*²).

- **RF-14 (Protección Física y Fabricación Aditiva):** Todos los componentes electrónicos (placa, batería, gestor de carga y cableado) deben estar protegidos por una carcasa diseñada a medida y fabricada mediante impresión 3D, asegurando el aislamiento eléctrico y la ventilación pasiva de los componentes.
- **RF-15 (Control de Alimentación Manual):** El terminal debe disponer de un interruptor físico conectado al pin *Enable* del gestor de carga para permitir el encendido y apagado completo del sistema, evitando el drenaje innecesario de la batería cuando no esté en uso.

Requisitos No Funcionales (RNF)

Los requisitos no funcionales establecen las métricas de calidad, rendimiento, ergonomía y seguridad que condicionan el diseño de la arquitectura.

- **RNF-01 (Seguridad y Privacidad):** Dada la sensibilidad de los datos procesados, el sistema debe operar de manera aislada (*web* local en la *Raspberry Pi* y despliegue en la *intranet* del *HUBU*), asegurando que ningún historial clínico sea transmitido a redes externas o servicios en la nube.
- **RNF-02 (Ergonomía y Usabilidad):** La interfaz gráfica (*UI*) debe ser completamente *responsive* y sus elementos de interacción (botones, selectores) deben estar dimensionados y optimizados para el uso en máquina de sobremesa o en la pantalla táctil de 7 pulgadas del terminal.
- **RNF-03 (Rendimiento):** El tiempo de respuesta del motor matemático de *Python* para el cálculo y renderizado de un indicador clínico no debe penalizar la agilidad del facultativo durante una urgencia médica.

A continuación se detallan las siete tablas desarrolladas para este proyecto:

- La Tabla C.1 desarrolla el proceso de carga y procesamiento de los datos.

²Es una técnica de gestión de energía que permite a un dispositivo electrónico funcionar utilizando la energía de la fuente externa mientras simultáneamente carga su batería.

CU-1	Ingesta y Validación de Registros Clínicos
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-01, RF-02
Descripción	Proceso mediante el cual el usuario carga los archivos brutos y el sistema verifica su integridad y formato.
Precondición	El usuario debe disponer de los archivos de monitorización exportados en formato <i>CSV</i> .
Acciones	1. El usuario accede al módulo de carga. 2. El usuario selecciona los archivos desde su almacenamiento local. 3. El sistema evalúa la extensión y el volumen de los archivos. 4. El sistema almacena los datos en la memoria <i>RAM</i> y muestra un mensaje de éxito.
Postcondición	Los datos validados quedan listos para su procesamiento algorítmico.
Excepciones	Formato inválido o exceso de archivos: Si un archivo no es <i>.csv</i> o se seleccionan más de 10, el sistema aborta la carga y muestra un error.
Importancia	Alta

Tabla C.1: Especificación del Caso de Uso CU-1.

- La tabla C.2 detalla el método de selección de indicadores individuales
- La Tabla C.3 desglosa el método que permite el análisis gráfico de hasta tres indicadores de forma conjunta.
- La Tabla C.4 explica todo el proceso relativo a la generación del resumen de los indicadores y su descarga en forma de informe *PDF*.
- La Tabla C.5 detalla el proceso de conexión y extracción de información clínica histórica directamente desde la *BBDD*.
- La Tabla C.6 expone las operaciones de administración directa sobre los registros clínicos, garantizando la integridad de la información.

CU-2	Procesamiento de Indicadores <i>SEMICYUC</i>
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-03, RF-04
Descripción	Interacción para la selección de métricas específicas y ejecución del motor matemático.
Precondición	Deben existir datos validados en la memoria volátil del sistema (Postcondición CU-1 o CU-5).
Acciones	1. El usuario navega hasta el desplegable de selección de indicadores. 2. El usuario selecciona un indicador de calidad clínica del menú. 3. El motor de <i>Python</i> procesa los registros buscando los cálculos correspondientes. 4. El sistema presenta los datos estadísticos por pantalla. 5. El sistema permite visualizar dos tipos de gráficos
Postcondición	El indicador estadístico queda calculado y listo para descargar sus cálculos.
Excepciones	Datos insuficientes: Si los registros cargados no contiene la columna requerida para ese indicador, el sistema notifica la imposibilidad del cálculo.
Importancia	Alta

Tabla C.2: Especificación del Caso de Uso CU-2.

- La Tabla C.7 describe el flujo de exportación masiva y el proceso de protección de datos personales antes de su almacenamiento externo.

C.3. Prototipos de interfaz o interacción con el proyecto

En el desarrollo de *software* con aplicación clínica, el diseño de la **experiencia de usuario** (*UX*) y la **interfaz de usuario** (*UI*) es tan crítico como el propio motor algorítmico. Una interfaz mal diseñada en un entorno

CU-3	Análisis Interactivo en Mezcla de Indicadores
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-05
Descripción	Uso de las herramientas de visualización, filtrado y comparativa gráfica (<i>Dashboard</i> entre indicadores).
Precondición	Deben existir datos validados en la memoria volátil del sistema (Postcondición CU-1 o CU-5).
Acciones	1. El usuario utiliza la herramienta de “Mezcla” para superponer hasta 3 indicadores. 2. El usuario selecciona el tipo de visualización. 3. El sistema renderiza la gráfica en la interfaz interactiva. 4. El sistema actualiza la gráfica en tiempo real.
Postcondición	La información se muestra de manera visual y comprensible para la toma de decisiones.
Excepciones	Límite de superposición: Si se intentan mezclar más de 3 indicadores, el sistema bloquea la acción para preservar la legibilidad.
Importancia	Media

Tabla C.3: Especificación del Caso de Uso CU-3.

de alta presión como una unidad de Reanimación (*URCCPQ*) puede inducir a errores operativos o retrasar la toma de decisiones.

Por ello, de forma previa a la codificación de las vistas en el *framework Reflex*, se ha llevado a cabo una fase de prototipado de baja fidelidad (*wireframing*). Esta práctica es un estándar en la ingeniería de *software* porque permite:

- Definir la distribución espacial de la información minimizando la carga cognitiva del facultativo.
- Detectar de forma temprana problemas de usabilidad o cuellos de botella en la navegación.

CU-4	Generación y Exportación de Informes
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-06
Descripción	Empaquetado de los resultados y descarga en formatos estandarizados para la historia clínica.
Precondición	Deben existir datos validados en la memoria volátil del sistema (Postcondición CU-1 o CU-5).
Acciones	1. El usuario pulsa el botón de descarga correspondiente al reporte deseado. 2. El sistema recopila los cálculos y gráficas de la sesión actual. 3. Se genera un archivo en formato estructurado (<i>PDF</i> o tabla resumen). 4. El archivo se descarga en el disco duro local del dispositivo.
Postcondición	El facultativo obtiene un documento físico/digital para archivar en el historial.
Excepciones	Error de escritura: Si el disco está lleno o no hay permisos, se alerta al usuario del fallo en la descarga.
Importancia	Alta

Tabla C.4: Especificación del Caso de Uso CU-4.

- Validar que la disposición de los elementos (botones, selectores, gráficas) se ajusta a los requisitos ergonómicos de las unidades computacionales de la *URCCPQ* y de una pantalla táctil de 7 pulgadas.

A continuación, se presentan los prototipos estructurales de las siete vistas fundamentales que componen la arquitectura visual del terminal:

- **Pantalla Principal de Ingesta (Figura C.2):** Representa la vista inicial del sistema. Su diseño es minimalista, centrando la atención del usuario en el área de carga (arrastrar y soltar) para los archivos *.csv* de monitorización o la apertura de la base de datos, cumpliendo con el Caso de Uso CU-01.

CU-5	Carga de Datos desde BBDD
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-07, RF-08
Descripción	Consulta y extracción de registros clínicos desde <i>PostgreSQL</i> hacia la memoria volátil del sistema para su análisis.
Precondición	El usuario debe tener acceso de Nivel 1 (o superior) y la base de datos debe estar operativa.
Acciones	1. El usuario accede al módulo de interacción con la <i>BBDD</i>. 2. El usuario selecciona el año o rango de años a analizar. 3. El sistema lanza una consulta estructurada a la base de datos. 4. El sistema aloja los resultados en la memoria <i>RAM</i> de la aplicación.
Postcondición	Los datos quedan listos en memoria para su procesamiento algorítmico (CU-2, CU-3 Y CU-4).
Excepciones	Ausencia de años: Si no se selecciona ningún año, el sistema alerta al usuario y detiene la carga.
Importancia	Alta

Tabla C.5: Especificación del Caso de Uso CU-5.

- **Menú de Navegación y Enrutamiento (Figura C.3):** Muestra la disposición del panel de control lateral o desplegable, diseñado con una tipografía y áreas de pulsación amplias para facilitar el salto rápido entre los distintos módulos analíticos sin requerir gran precisión.
- **Vista de Indicador Individual (Figura C.4):** Esboza la estructura del área de trabajo clínico. En la parte superior se ubican las métricas *SEMICYUC* (CU-02), mientras que el cuerpo central está reservado para la renderización en gran formato de las gráficas (lineal o barras).
- **Herramienta de Mezcla y Comparativa (Figura C.5):** Este prototipo ilustra la funcionalidad más avanzada del sistema. Muestra

CU-6	Gestión BBDD (Añadir, Editar, Borrar)
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-07, RF-09, RF-10
Descripción	Módulo de administración para la inserción de nuevos pacientes y la modificación o eliminación de registros existentes.
Precondición	El usuario debe poseer credenciales de Nivel 2 (Edición Clínica) o superior.
Acciones	1. El usuario accede al panel de gestión de registros. 2. Si desea editar o borrar, el sistema exige realizar una búsqueda previa del paciente. 3. El usuario introduce los nuevos datos o modifica los existentes en el formulario. 4. El sistema ejecuta la transacción de escritura en <i>PostgreSQL</i>.
Postcondición	La base de datos se actualiza manteniendo la consistencia de los historiales clínicos.
Excepciones	Datos erróneos: Si un usuario introduce datos sin sentido (ej. fecha de alta anterior a la de ingreso), el sistema bloquea la operación por seguridad.
Importancia	Alta

Tabla C.6: Especificación del Caso de Uso CU-6.

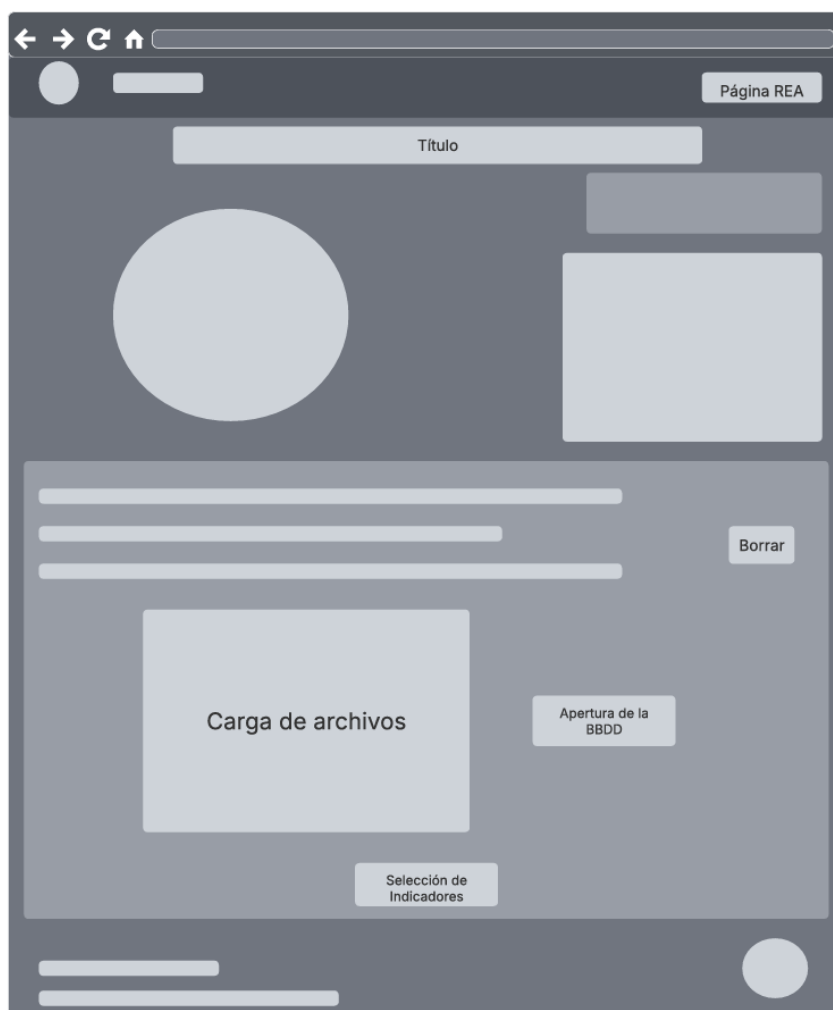
cómo el facultativo puede añadir iterativamente hasta tres indicadores, visualizando la leyenda cruzada y la superposición de datos en el lienzo principal (CU-03).

- **Panel de Resumen Global (Figura C.6):** Representa el cuadro de mandos final, donde se agrupan las métricas totales y la distribución por especialidades en formato tabular. Incluye los botones de acción (*Call to Action* o *CTA*) para la generación y exportación del informe final en *PDF* (CU-04).
- **Pantalla de Inicio de Sesión (Figura C.7):** Representa el portal de autenticación seguro de la aplicación. Su diseño diferencia claramente

CU-7	Extracción CSV Anónimo de BBDD
Versión	1.0
Autor	Nicolás García Gómez
Requisitos asociados	RF-07, RF-11
Descripción	Exportación en formato <i>CSV</i> del histórico clínico aplicando algoritmos de anonimización y borrado de identificadores personales.
Precondición	El usuario debe poseer credenciales exclusivas de Nivel 3 (Administración).
Acciones	1. El administrador accede a la interfaz de la <i>BBDD</i>, habilitándose el botón específico de exportación. 2. El sistema recupera el volumen de datos solicitado desde <i>PostgreSQL</i>. 3. El motor de la aplicación purga automáticamente la columna identificativa (Número de Historia Clínica). 4. Se empaqueta la información resultante en un archivo <i>CSV</i>. 5. El sistema descarga el archivo anonimizado en el dispositivo local.
Postcondición	Se obtiene un conjunto de datos legales aptos para investigación o auditoría externa.
Excepciones	Ausencia de años: Si no se selecciona ningún año, el sistema alerta al usuario y detiene la carga.
Importancia	Alta

Tabla C.7: Especificación del Caso de Uso CU-7.

dos áreas de entrada: un bloque principal destinado al acceso clínico habitual mediante correo y contraseña (Niveles 1, 2 y 3), y una sección inferior reservada exclusivamente para el acceso administrativo (Nivel 3). Esta pasarela es indispensable para garantizar la trazabilidad y asignar dinámicamente los módulos disponibles según el modelo jerárquico implementado.

Figura C.2: Pantalla principal de la *web*.

- **Interfaz de la Base de Datos (Figura C.8):** Ilustra el panel unificado de gestión clínica. Su diseño modulariza visualmente las funciones según los casos de uso: el bloque superior gestiona la selección por años (CU-05); el bloque central contiene los campos de búsqueda por número de historia para habilitar la adición, edición o borrado de pacientes (CU-06); y la barra inferior agrupa las acciones globales, destacando la carga a la memoria de la aplicación y el botón de exportación (CU-07), el cual solo será funcional si el rol del usuario lo permite.

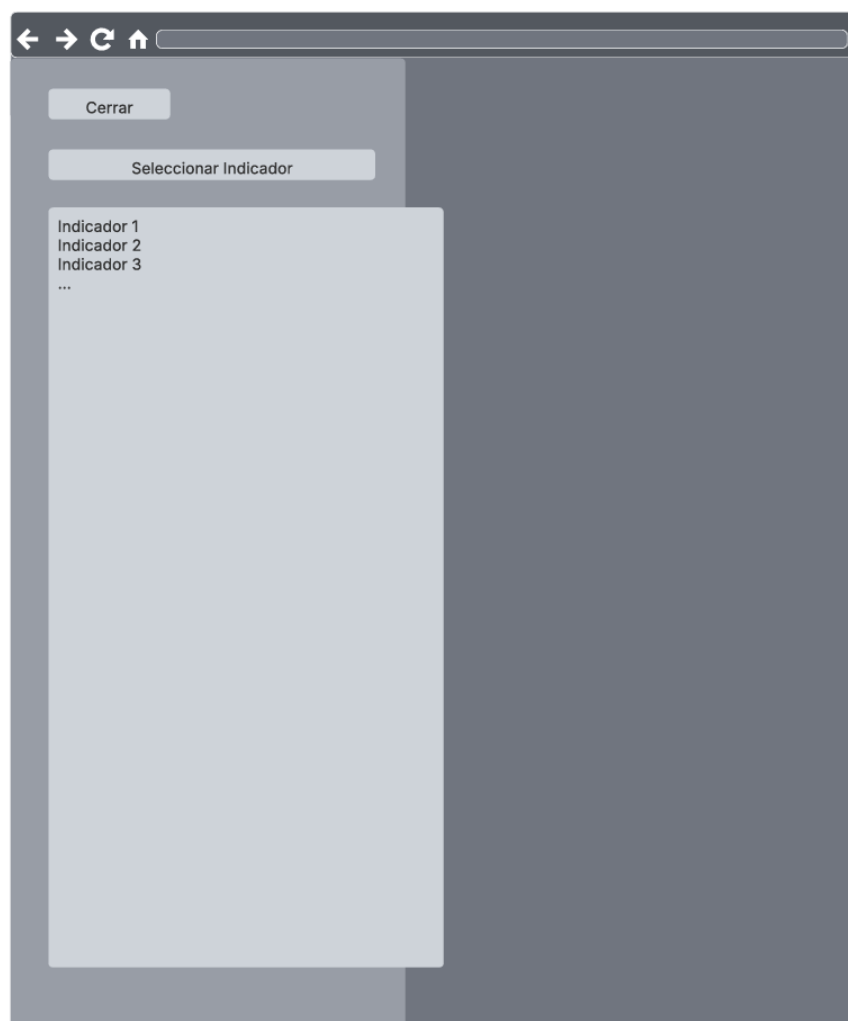


Figura C.3: Panel de selección lateral.

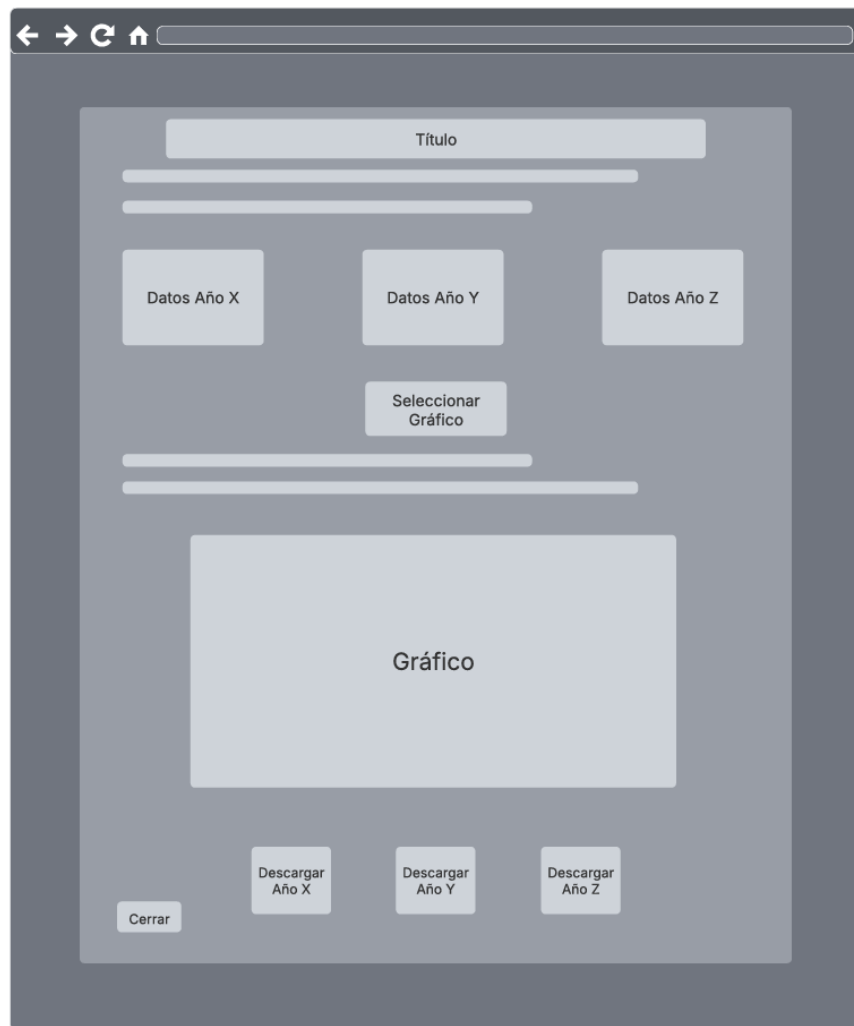


Figura C.4: Indicador individual.

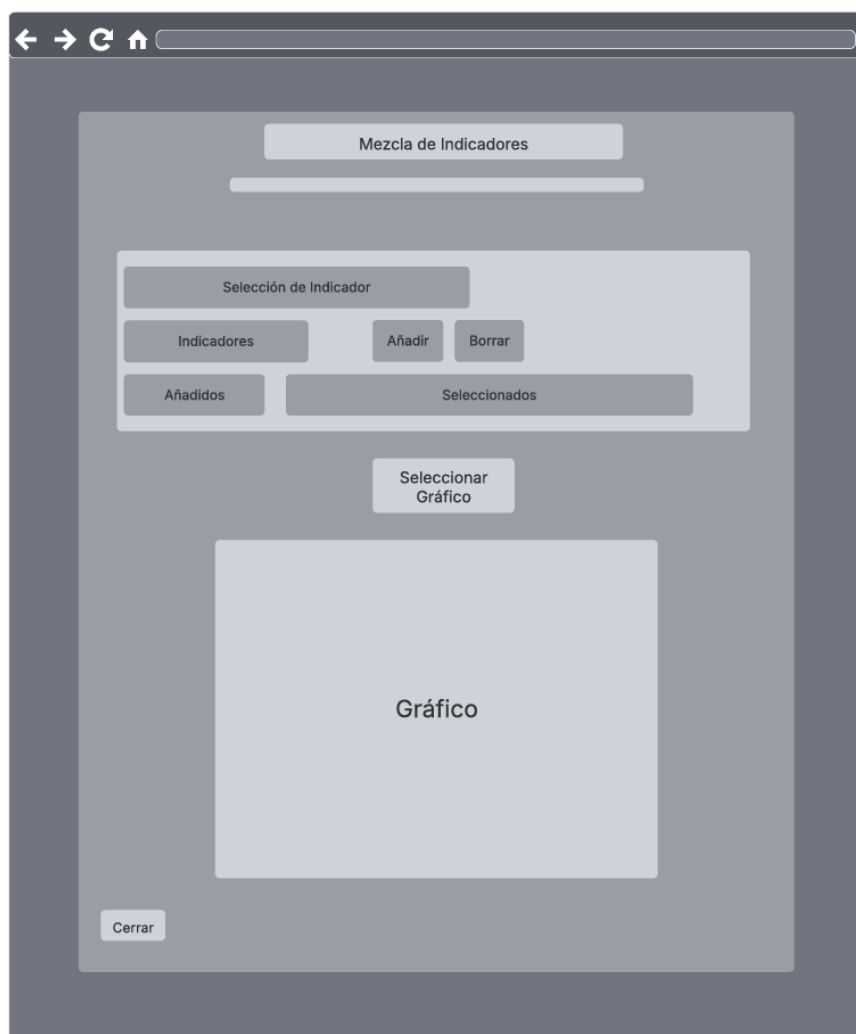


Figura C.5: Mezcla de indicadores.

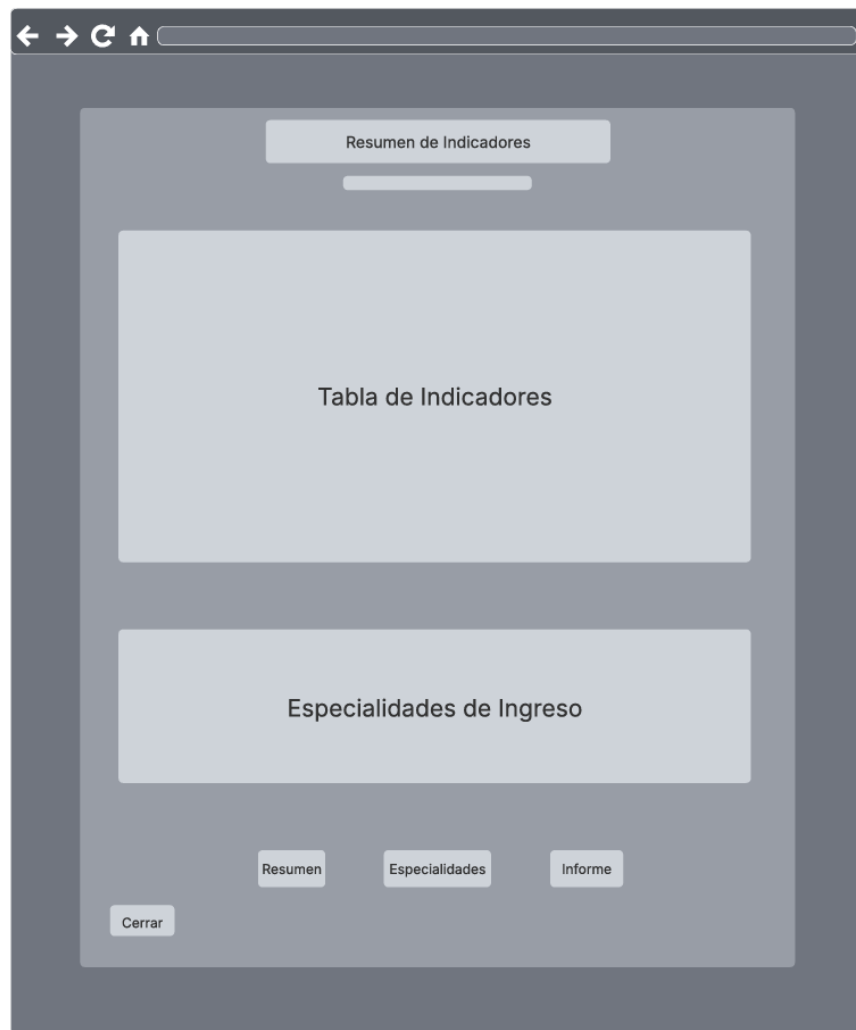


Figura C.6: Resumen final de los indicadores.



Figura C.7: Pantalla de inicio de sesión y validación de roles.



Figura C.8: Interfaz unificada para la gestión y consulta de la *BBDD*.

Apéndice *D*

Documentación de usuario

D.1. Requisitos software y hardware para ejecutar el proyecto.

A diferencia del entorno de desarrollo, el entorno de producción para el usuario final (el facultativo médico) se ha diseñado bajo la filosofía de *Zero-Configuration* (Cero Configuración). El objetivo es que la curva de adopción técnica sea nula, evitando que el personal sanitario deba instalar librerías, configurar servidores locales o modificar variables de entorno.

Para garantizar el correcto funcionamiento y despliegue del terminal clínico, el usuario o la institución hospitalaria únicamente deben asegurar el cumplimiento de los siguientes requisitos mínimos:

Requisitos de *Software*

- **Sistema Operativo:** El sistema es multiplataforma, pero para la ejecución en los equipos de escritorio de la unidad se requiere un Sistema Operativo *Windows 10* o *Windows 11* (arquitectura de 64 bits). En el caso del uso del terminal portátil dedicado, este opera nativamente bajo *Raspberry Pi OS* (basado en *Debian*).
- **Navegador Web:** Al tratarse de una arquitectura basada en tecnologías web (*framework Reflex*), la interfaz gráfica de usuario (*UI*) se renderiza en el navegador. Se requiere una versión actualizada de navegadores modernos basados en *Chromium* (*Google Chrome*, *Microsoft Edge*), *Mozilla Firefox* o *Apple Safari*.

- **Despliegue Centralizado y Acceso Inmediato:** No se requiere la instalación de *Python*, dependencias externas ni *software* local por parte de los facultativos de la *URCCPQ*. La infraestructura de la aplicación y la base de datos se ejecutan de forma centralizada y transparente mediante contenedores *Docker* en la *intranet* del hospital. En consecuencia, el usuario interactúa con el sistema directamente a través de cualquier navegador web estándar, garantizando una adopción ágil y eliminando por completo la necesidad de mantenimiento o configuración en los equipos de la unidad.

Requisitos de *Hardware*

El sistema no requiere de capacidades de procesamiento avanzadas (como tarjetas gráficas dedicadas), dado que el motor matemático optimizado exige un bajo consumo de recursos. Los requisitos físicos se dividen según la modalidad de uso:

- **Uso en Equipos de Planta (*Desktop*):** Cualquier ordenador estándar con un procesador de 64 *bits*, un mínimo de 4GB de memoria *RAM*, conexión para periféricos de entrada (ratón y teclado) y espacio en disco suficiente para el almacenamiento de los archivos descargables.
- **Uso en el Terminal Portátil Dedicado:** Si se utiliza el dispositivo físico desarrollado en este proyecto, el *hardware* ya se encuentra encapsulado y preconfigurado. El módulo consta de una placa *Raspberry Pi 3 Model B*, una pantalla oficial *LCD* táctil de 7 pulgadas y una tarjeta *MicroSD* de 32GB. El único requisito por parte del usuario es disponer de una fuente de alimentación estándar *Micro-USB* conectada a la red eléctrica para recargar la batería.

D.2. Puesta en marcha

Esta sección proporciona una guía detallada para el despliegue y la primera puesta en funcionamiento del sistema. A diferencia de los métodos de instalación tradicionales que requieren la configuración manual de servidores, motores de bases de datos y entornos de ejecución, este proyecto implementa un modelo de **despliegue automatizado mediante contenedores *Docker* en la *intranet* del *HUBU***.

Este enfoque garantiza que el facultativo pueda iniciar la plataforma de forma inmediata y segura, minimizando el riesgo de errores de configuración técnica.

Acceso al Software desde los Equipos de Planta

Gracias a la nueva arquitectura centralizada basada en contenedores *Docker*, la plataforma opera como un servicio *web* interno (*On-Premise*¹). Por consiguiente, se elimina por completo la necesidad de instalar dependencias, descargar librerías o ejecutar *scripts* locales en los ordenadores de la *URCCPQ*. El procedimiento de acceso para cualquier facultativo se ha simplificado a los siguientes pasos:

1. **Verificación de red:** Comprobar que el ordenador del puesto de trabajo está conectado a la *intranet* del hospital. Este es un requisito técnico y legal indispensable, ya que el servidor opera en una red aislada sin salida a internet para garantizar la privacidad de los datos.
2. **Acceso al portal:** Abrir el navegador *web* institucional e introducir en la barra de direcciones la *IP* local asignada al servidor (por ejemplo, `http://192.168.X.X:3000`) o su dominio interno correspondiente (ver Figura D.1).

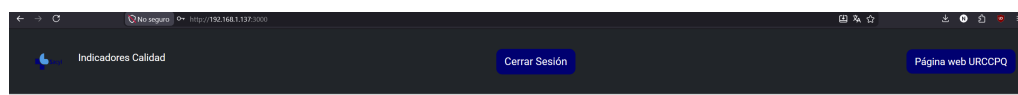


Figura D.1: Acceso a la plataforma clínica mediante la introducción de la *IP* local en el navegador.

3. **Autenticación de usuario:** Una vez cargada la pantalla de inicio, el facultativo deberá introducir sus credenciales seguras. El sistema evaluará automáticamente su nivel de privilegios (*RBAC*) y desplegará las herramientas y menús correspondientes a su rol operativo.

A diferencia de las versiones previas de desarrollo, no existen ventanas de terminal (*CMD*) ni procesos en segundo plano que el usuario deba gestionar, minimizar o monitorizar. Al finalizar la sesión clínica o la ingesta de datos, el

¹Modelo de infraestructura donde el software y los servidores se instalan y operan dentro de las instalaciones físicas de la propia empresa, en lugar de en la nube.

usuario únicamente debe cerrar la pestaña del navegador para desvincularse de la aplicación de manera segura, sin riesgo de detener el servidor para el resto de la unidad. Para garantizar esta accesibilidad y transparencia de cara al facultativo, toda la complejidad de la plataforma se delega a la infraestructura del servidor. El entorno operativo se fundamenta en una orquestación de contenedores independientes que encapsulan la interfaz, el motor estadístico y la base de datos relacional de forma aislada. Como se puede observar en la Figura D.2, el motor de *Docker* gestiona de forma centralizada el ciclo de vida de estos servicios, asegurando su ejecución ininterrumpida (*Up*) y replicando con exactitud el comportamiento real que el sistema tendría al ser desplegado en los servidores de la *intranet* hospitalaria.

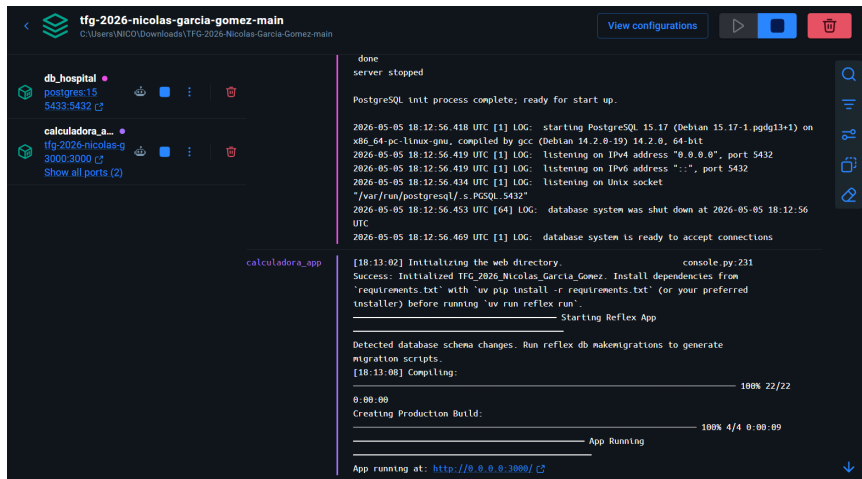


Figura D.2: Panel de *Docker* monitorizando el estado activo (*Up*) de los contenedores que dan soporte a la plataforma en la *intranet* simulada.

Guía de Operación del Terminal Físico

Dado que el *software* ha sido diseñado bajo una arquitectura de despliegue automatizado, la interacción del facultativo se centra principalmente en el manejo del dispositivo físico dedicado. A continuación, se detallan las pautas básicas para la correcta operación del terminal en la unidad de Reanimación (*URCCPQ*).

Encendido y Apagado

El dispositivo cuenta con un sistema de gestión de energía integrado en su carcasa.

- **Para encender el terminal:** Accione el interruptor físico situado en el lateral de la carcasa. La pantalla táctil se iluminará y el sistema operativo iniciará la plataforma clínica de forma automática en pocos segundos, mostrándose la pantalla de inicio lista para la carga de datos como se observa en la figura D.3.

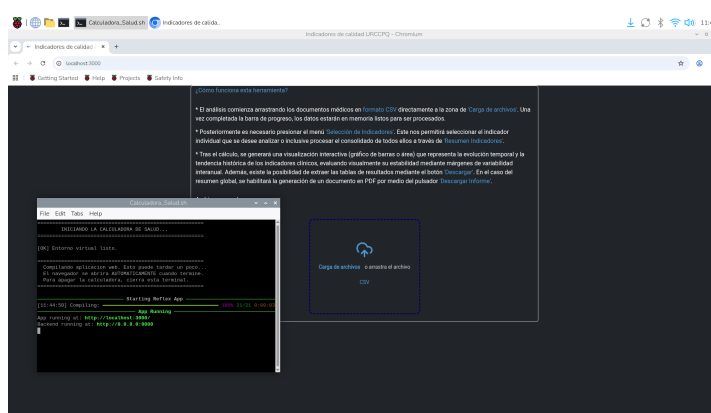


Figura D.3: Terminal portátil encendido mostrando la interfaz inicial tras el arranque automático.

- **Para apagar el terminal:** Con el fin de evitar la corrupción de la tarjeta de memoria interna, se recomienda pulsar primero el botón de “Apagar Sistema” situado en el menú de la interfaz gráfica. Una vez la pantalla se apague por completo, puede accionar el interruptor lateral para cortar el suministro eléctrico de la batería.

Autonomía y Recarga de la Batería

El terminal está equipado con una batería interna de alta capacidad que le otorga total movilidad entre los distintos boxes de la unidad.

- **Autonomía:** En condiciones de uso continuo y brillo de pantalla estándar, el dispositivo ofrece varias horas de autonomía operativa.
- **Proceso de carga:** Cuando el nivel de batería sea bajo, conecte un cable estándar con terminación *Micro-USB* al puerto de carga

habilitado en la carcasa, y conéctelo a un adaptador de red eléctrica de 5 V.

- **Uso ininterrumpido:** Gracias a su módulo de gestión avanzada, el terminal puede seguir siendo utilizado con total normalidad mientras se encuentra enchufado recargando su batería, sin sufrir interrupciones ni reinicios.

Cuidados y Limpieza

Al tratarse de un dispositivo destinado a un entorno clínico de cuidados críticos, es importante mantener unas pautas de higiene que no comprometan la electrónica interna:

- La carcasa, fabricada a medida mediante impresión *3D*, protege los componentes internos. Sin embargo, el dispositivo **no es sumergible**.
- Para su desinfección superficial, se recomienda utilizar toallitas con alcohol isopropílico (al 70 %) o paños de microfibra ligeramente humedecidos con soluciones desinfectantes de uso hospitalario, aplicándolos suavemente sobre la pantalla y la carcasa exterior.
- Evite la pulverización directa de líquidos sobre el terminal para prevenir filtraciones por las ranuras de ventilación o los puertos de conexión.

D.3. Manuales y/o Demostraciones prácticas

Para proceder a la utilización de la herramienta de calculo desarrollada, se requiere haber seguido las instrucciones de inicio automatizado descritas en el apartado anterior.

Manual de Uso Clínico

Este apartado resume las instrucciones necesarias para que cualquier facultativo pueda operar el sistema de forma autónoma, garantizando la integridad de los datos y la agilidad en la consulta.

1. **Preparación del Entorno de Trabajo:** El procedimiento de inicialización dependerá del dispositivo utilizado para realizar el análisis clínico:

- **Uso del Terminal Portátil (*Raspberry Pi*):** Asegúrese de que el interruptor lateral del dispositivo esté en posición de encendido. Esto desplegará el servidor de forma local y autónoma, permitiendo el análisis de los archivos *CSV* anonimizados extraídos previamente del hospital, sin necesidad de conexión a la red de la *URCCPQ*.
 - **Uso desde los Equipos de la Unidad:** Si se opera directamente desde los ordenadores de planta, no es necesaria la ejecución de ningún archivo o *script* local. El sistema ya se encuentra desplegado de forma continua en la *intranet*, por lo que bastará con acceder a la plataforma centralizada a través del navegador *web*.
2. **Navegación:** Una vez cargados los datos en la *RAM*, utilice el menú lateral desplegable para alternar entre la vista de “Resumen Indicadores” (indicadores agregados de la unidad), la vista de “Mezcla de Indicadores” (permitiendo la comparación directa entre los indicadores) o seleccionando los indicadores individuales (profundización en una métrica específica).
 3. **Interacción con Gráficas:** Las gráficas permiten el filtrado dinámico. Al pasar el ratón sobre la leyenda de un indicador, este se mostrará en el lienzo principal, facilitando la limpieza visual durante la sesión.

Demostraciones prácticas

Con el propósito de reducir el tiempo de aprendizaje del personal sanitario, se han realizado vídeos demostrativos accesibles desde los enlaces proporcionados:

- **Vídeo “Arranque de la Herramienta”:** Vídeo del arranque normal del servidor.
- **Vídeo “Uso Avanzado”:** Tutorial sobre la carga de múltiples archivos *.csv*, el uso de la *BBDD* y de las diferentes funcionalidades de la *web*.

Apéndice *E*

Manual del programador.

E.1. Estructura de directorios

El código fuente desarrollado a lo largo de este proyecto se encuentra estructurado y accesible a través de diferentes repositorios de *GitHub* [Nicolás, 2026], separados de acuerdo a las tres arquitecturas propuestas: el entorno de producción hospitalario, el terminal portátil (*Raspberry Pi*) y el entorno de desarrollo local.

No obstante, con el objetivo de facilitar la comprensión y el análisis técnico del trabajo, en este apartado se detallará exclusivamente la estructura y el contenido de los ficheros correspondientes al **despliegue local con la base de datos integrada mediante *Docker*: TFG-2026-Nicolas-Garcia-Gomez**. Se ha seleccionado esta versión como referencia principal para la memoria dado que engloba la totalidad de la lógica de negocio, las herramientas de migración y el entorno completo de pruebas del sistema.

- `README.md`
Pequeña introducción y explicación sobre el trabajo y los recursos disponibles en el repositorio.
- `LICENCE`
Documento de la licencia *Apache License 2.0* utilizada para este proyecto.
- `.gitignore`
Documento de configuración que le indica a *Git* qué archivos o carpetas (como los documentos *CSV* con datos médicos sensibles o el

entorno virtual) debe ignorar por completo para no subirlos nunca a tu repositorio de *GitHub*.

- **requirements.txt**
Listado que contiene todas las librerías externas que necesita el proyecto para funcionar.
- **rxconfig.py**
Documento de configuración central del *framework Reflex*, encargado de definir los parámetros globales de la aplicación, como su nombre, el puerto de ejecución o las conexiones, para que el sistema arranque con los ajustes correctos.
- **.env**
Archivo de configuración segura que almacena las variables de entorno y credenciales sensibles (como usuarios, contraseñas y la *URL* de conexión a la base de datos), separándolas del código fuente por motivos de seguridad.
- **.dockerignore**
Archivo de exclusión que indica a *Docker* qué directorios locales (como *cachés*, entornos virtuales o archivos ocultos) deben omitirse al construir la imagen, optimizando así el peso final del contenedor.
- **Dockerfile**
Documento de texto que contiene todas las instrucciones secuenciales para construir la imagen del entorno de la aplicación (define el sistema operativo base, instala las dependencias de *Python* y ejecuta el servidor de *Reflex*).
- **docker-compose.yml**
Archivo de orquestación en formato *YAML* que define y coordina la arquitectura completa del sistema. Levanta simultáneamente los servicios (*frontend/backend* y base de datos), estableciendo sus puertos, redes internas y volúmenes de persistencia.
- **alembic.ini**
Es el archivo de configuración principal de la herramienta de migraciones. Actúa como el centro de mando que le indica a *Alembic* dónde está la carpeta con los historiales, qué formato usar para generar los nuevos archivos y cómo debe comportarse al conectarse. Es básicamente el manual de instrucciones inicial para que el sistema de migraciones sepa cómo arrancar y trabajar con el proyecto.

- `alembic/`.

Es el corazón de las migraciones. Cada vez que se cambie algo en el código y se ejecute el comando `makemigrations`, aquí se genera un nuevo archivo *Python*. Estos archivos son los “planos de obra” secuenciales que contienen las instrucciones matemáticas exactas (ej. “crear tabla paciente”, “añadir columna edad”) que *PostgreSQL* debe ejecutar para actualizarse sin perder los datos clínicos que ya tiene guardados.

- `Datos/`.

Contiene todos los archivos utilizados para las demostraciones y pruebas de la aplicación.

- `Datos_2016_2025.csv`

Conjuntos de archivos *CSV* sintéticos utilizados para el desarrollo de la *web*

- `Datos.txt`

Archivos de texto que describe todos los datos a recoger y sus tipos en la unidad.

- `importador.py`

Script administrativo independiente diseñado para automatizar el proceso *ETL* (Extracción, Transformación y Carga), permitiendo limpiar, normalizar y migrar de forma masiva los historiales clínicos desde archivos *CSV* a la base de datos *PostgreSQL*.

- `Documentacion_Latex/`.

Carpeta que contiene todos los archivos empleados en el desarrollo de la memoria. Incluye los documentos pdf de la memoria y anexos.

- `/img/`.

Localización de todas las imágenes empleadas tanto en la memoria como en los anexos.

- `/tex/`.

Carpeta que almacena todos los capítulos de la memoria y anexos en sus respectivos documentos *LaTeX*.

- `Logica/`.

Almacena todos los archivos de *Python*(*.py*) necesarios para el correcto funcionamiento del *Backend*.

- `BBDD.py`

Archivo que administra la lógica relacionada con la base de datos.

Contiene las funciones que administran la carga de registros, así como su edición, modificación y visualización en pantalla.

- **Modelo.py**
Define los modelos que estructuran las tablas de la base de datos para almacenar los historiales clínicos, registrar los *logs* de auditoría y gestionar las credenciales de los usuarios.
 - **PDF.py**
Archivo que contiene toda la lógica relativa al procesamiento del informe final. Recibe todos los datos y gráficas calculadas y se encarga de su correcta colocación.
 - **Programa.py**
Contiene la gran mayoría del procesamiento y cálculo de indicadores. Recibe los registros en la *RAM* cargados por el usuario y realiza la computación de todos los indicadores, el resumen, y la mezcla.
 - **State.py**
Se encarga de la recepción de los archivos y el control de la lista *FIFO*. Controla que el máximo de años almacenados sea de diez y que el formato sea el admitido por la herramienta. Además almacena todos los métodos de borrado.
 - **Usuarios.py**
Constituye el núcleo de seguridad y control de acceso de la aplicación web. Se encarga de gestionar de forma íntegra la autenticación de los usuarios mediante cifrado unidireccional (*Bcrypt*), la validación de credenciales frente a la base de datos y la administración de los diferentes roles y permisos.
- **TFG_2026_Nicolas_Garcia_Gomez/.**
Directorio que almacena todos los componentes relativos al *frontend*, por que en todos ellos se utiliza código de *Reflex*.
- **TFG_2026_Nicolas_Garcia_Gomez.py**
Funciona como el módulo orquestador. Su única responsabilidad es instanciar el objeto *rx.App*, definir el árbol de navegación (rutas) e importar los diferentes componentes modulares de la interfaz para compilarlos.
 - **constantes.py**
Contiene *URL* constantes de las diferentes referencias y *links* utilizados en la *web*.

- `/componentes/`.
Contiene todos los diferentes componentes gráficos utilizados en la herramienta.
 - `boton_subida.py`
Archivo que despliega el botón de subida.
 - `graficos.py`
Archivo que contiene todos los gráfico utilizados en *Reflex*.
 - `link_icon.py`
Contiene un botón interactivo para el uso de enlaces externos.
 - `mezcla.py`
Archivo que se utiliza para el despliegue del método mezcla. Al tener tantos componente visuales diferentes al resto de indicadores se separó por limpieza y pulcritud.
 - `renderizar_campo_formulario.py`
Define una función que genera visualmente un campo de formulario dinámico, adaptando automáticamente el tipo de control (desplegable *booleano*, entrada de fecha, número o texto) y su diseño según la categoría a la que pertenezca el dato.
 - `seleccion.py`
Fichero que almacena todos los componentes visuales del menú desplegable de indicadores. Además controla la llamada a cada uno de los método una vez seleccionado la opción deseada.
 - `tabla.py`
Registro que contiene todos los componentes para la creación de tablas. En un inicio se encontraba en el archivo `vent_flotante.py` separándose por limpieza.
- `/estilos/`.
Carpeta que contiene todos los archivo relacionados a la fuente, colores y estilado de la *web*.
 - `colores.py`
Almacena los colores como enumerados (*Enum*).
 - `estilos.py`
Contiene tamaños estándar utilizados, hojas de estilo [Christian Robertson, b] [Bootstrap,] y el estilo base para la *web*.
 - `fuentes.py`
Contiene la fuente utilizada *Roboto* [Christian Robertson, a].

- `Roboto-Regular.ttf`
Archivo con la fuente *Roboto* utilizada en el informe *PDF* para utilizar caracteres especiales.
- `/views/`.
Directorio, el cual contiene todas las vistas y partes de la página.
 - `acceso.py`
Contiene los tres componente gráficos de la validación de usuarios. El inicio de sesión, el cambio de contraseña y el panel de gestión para los administradores.
 - `cabecera.py`
Almacena la primera parte de la página donde se da la bienvenida al usuario y se dan las diferentes instrucciones de uso.
 - `instrucciones.py`
Parte de la *web* que contiene las instrucciones de su funcionamiento, así como el botón de subida y el disparados del menú de selección de indicadores.
 - `navbar.py`
Componente que muestra la barra fija superior, donde se encuentra el enlace a la página de la *URCCPQ* y el título de la *web*.
 - `pie.py`
Muestra el pie de la página, conteniendo el nombre del autor y el escudo de la *UBU*.
 - `vent_flotante.py`
Componente principal de la herramienta. Se almacena toda la lógica que permite mostrar diferentes componentes para cada opción del menú multiselección. Por ejemplo: a través de condicionales en *Reflex* (`rx.cond`) este componente es capaz de mostrar los indicadores individuales con sus tarjetas (`rx.card`) de datos o mostrar las tablas del resumen.
 - `vistas_bbdd.py`
Despliega toda la lógica de visualización de la interfaz de la *BBDD*. Muestra u oculta las diferentes funcionalidades que permite la base de datos en función del rol del usuario.
- `assets/`.
Directorio que contiene todas las imágenes y el icono de la *web*.

- `favicon.ico`
Icono de la *web* con el logo del *SaCyl*.
- `urccpq.png`
Imagen de la unidad.
- `sacyl.png`
Logo del *SaCyl*.
- `ubu.png`
Logo de la Universidad de Burgos *UBU*.

E.2. Pruebas del sistema

La validación de la plataforma clínica trasciende la mera comprobación de la interfaz gráfica. El plan de pruebas se ha diseñado bajo un enfoque de *Defensive Programming* (Programación Defensiva), asumiendo por defecto que las entradas del usuario o de las fuentes externas pueden contener errores de formato, omisiones o inconsistencias lógicas. A continuación, se detallan los bloques de pruebas críticas realizados, abarcando desde el despliegue del entorno hasta la validación transaccional de los datos médicos.

Despliegue de Infraestructura y Puesta en Marcha

El primer nivel de pruebas garantiza que el sistema es reproducible y puede ser desplegado de manera consistente en cualquier equipo u hospital, simulando las condiciones de la *intranet* clínica.

- **Objetivo de la prueba:** Verificar la correcta construcción de los contenedores *Docker* a partir del repositorio de código, la inicialización de la base de datos y la aplicación automática de las migraciones estructurales, se proporciona un vídeo que explica todo el proceso descrito en este apartado: **Primer Arranque de la Herramienta**.
- **Ejecución y Resultado:**
 1. **Instalación de los requisitos previos:** Antes de operar con el código fuente, el equipo anfitrión debe contar con las herramientas base para la gestión de versiones y la contenerización. Se requiere la descarga e instalación de *Docker Desktop*.
 - **Nota importante para entornos *Windows*:** Para garantizar el correcto arranque del motor de *Docker*, es indispensable

verificar que la virtualización por *hardware* se encuentre habilitada en la configuración de la *BIOS* del equipo. Asimismo, se requiere que el Subsistema de *Windows* para *Linux* ([WSL 2](#)) esté correctamente instalado y configurado. En caso de presentar errores de inicialización, se recomienda ejecutar los comandos de actualización del núcleo en la terminal (`wsl --install` o `wsl --update`) y reiniciar el sistema.

2. **Obtención del código fuente:** Es necesario obtener el código de la aplicación eligiendo entre las dos siguientes vías:

- Se debe abrir una terminal o consola de comandos, navegar hasta el directorio de destino deseado y clonar el repositorio del proyecto ejecutando la siguiente instrucción: `git clone https://github.com/ngg1017/TFG-2026-Nicolas-Garcia-Gomez`.
- Descargarlo directamente desde el repositorio *online* [[Nicolás, 2026](#)]

3. **Configuración de las variables de entorno:** Por motivos de seguridad y buenas prácticas de desarrollo (definidas en los archivos `.gitignore` y `.dockerignore`), las credenciales sensibles no se deben de incluir en el repositorio público. En este proyecto se ha decidido subir un archivo `.env` a modo de ejemplo directamente a *GitHub* para facilitar la configuración inicial. Si se desea se puede editar el archivo de texto plano (`.env`) facilitado. Este archivo debe contener la cadena de conexión a la base de datos, siguiendo este formato: `DATABASE_URL=postgresql://medico:secreto123@db_hospital:5432/hosp`

4. **Construcción y levantamiento de la infraestructura:** Una vez configuradas las credenciales, se ordena a *Docker* la construcción de las imágenes y el despliegue de los servicios (el servidor *web* y el motor de base de datos) en segundo plano. Para ello, se ejecuta: `docker-compose up --build -d` en el directorio raíz. En la imagen [E.1](#) se muestra el resultado de la consola de este primer comando.

```
[+] up 22/22g provenance for metadata file
✓Image postgres:15                               Pulled          19.8s
✓Image tfg-2026-nicolas-garcia-gomez-main-calculadora-app Built          110.0s
✓Network tfg-2026-nicolas-garcia-gomez-main_default Created          0.1s
✓Volume tfg-2026-nicolas-garcia-gomez-main_pgdata Created          0.3s
✓Container postgres_tfg Started          2.2s
✓Container reflex_app_tfg Started          1.6s
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker-compose up --build -d
```

Figura E.1: Trazas de consola confirmando la creación de las imágenes

5. **Migración de la estructura de la base de datos:** Tras el levantamiento de los contenedores, el gestor de base de datos se encuentra vacío. Para que la herramienta *Alembic* traduzca los modelos de datos implementados en *Python* a sentencias *DDL* y construya la estructura de tablas y columnas correspondientes, se debe ejecutar la orden `docker exec reflex_app_tfg reflex db migrate`, si este comando no devuelve ningún tipo de log como se muestra en la imagen E.2 se puede pasar al siguiente punto o dar por finalizada la configuración.

```
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker exec reflex_app_tfg reflex db migrate
```

Figura E.2: Consola ejecutando la migración exitosa de la base de datos.

6. **Carga inicial de datos históricos (Opcional):** Si se requiere que la aplicación parta con información previa, se procederá a ejecutar el *script* administrativo encargado del proceso de **Extracción, Transformación y Carga (ETL)**. Este volcará los registros de los archivos locales a la base de datos recién creada. En la figura E.3 se muestra el proceso completo.
 - La primera parte de este volcado consiste en activar *Python* o algún entorno virtual disponible.
 - Posteriormente se hace un desplazamiento mediante el comando `cd` a la carpeta **Datos**.
 - Finalmente se ejecuta el *scrip* de importación cargando la *BBDD* con una serie de registro generados aleatoriamente para la prueba de la herramienta.

```
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> c:\Users\NICO\Desktop\Universidad\Practicas\ent_practicas\Scripts\Activate.ps1
(ent_practicas) PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> cd Datos
(ent_practicas) PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main\Datos> python importador.py
Tabla 'modelo' vaciada correctamente...
Procesando Datos2016.csv...
Procesando Datos2017.csv...
Procesando Datos2018.csv...
Procesando Datos2019.csv...
Procesando Datos2020.csv...
Procesando Datos2021.csv...
Procesando Datos2022.csv...
Procesando Datos2023.csv...
Procesando Datos2024.csv...
Procesando Datos2025.csv...
¡Importacion finalizada con exito total!
```

Figura E.3: Consola mostrando todos los pasos necesarios para la importación de los registros iniciales.

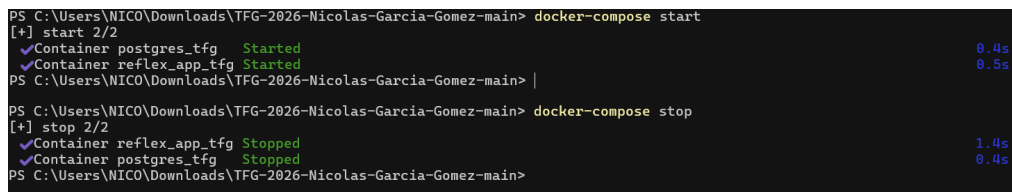
7. **Comprobación de acceso local:** Una vez finalizados los pasos anteriores, la aplicación estará completamente operativa. Se puede verificar su funcionamiento accediendo a través del navegador web del equipo anfitrión a la dirección: `http://localhost:3000`.

Gestión del Ciclo de Vida del Servidor

Una vez realizada la instalación y el primer despliegue, no es necesario repetir los pasos descritos anteriormente. El encendido y apagado rutinario del sistema se puede gestionar de dos formas equivalentes:

1. **Apagado Rutinario (Uso Diario Recomendado):** Para pausar el servidor al final de la jornada laboral sin destruir la infraestructura, se recomienda utilizar el apagado en suspensión. Los contenedores quedan “dormidos” y listos para arrancar inmediatamente al día siguiente:

- Ejecutar el comando `docker-compose stop` para detener el servicio, y `docker-compose start` para reanudarlo. El resultado de la consola de ambos comandos se refleja en la siguiente imagen E.4.



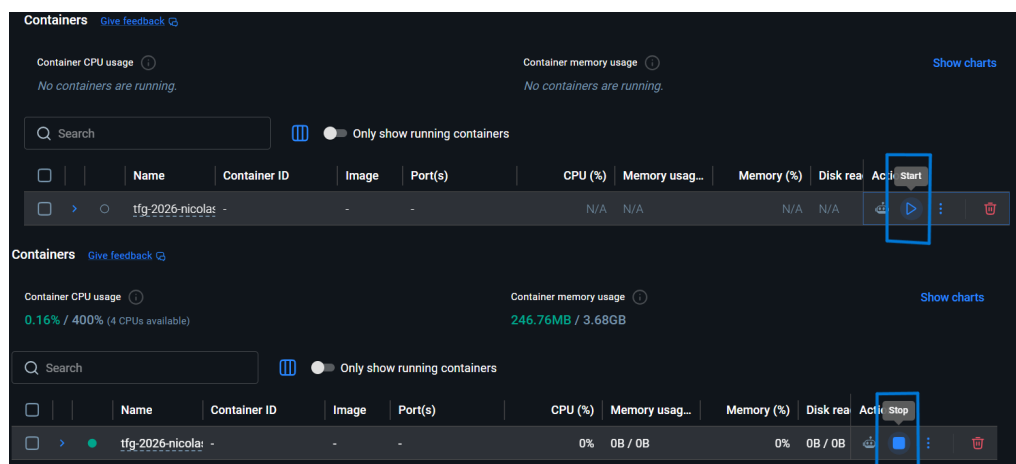
```

PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker-compose start
[+] start 2/2
✓Container postgres_tfg Started 0.4s
✓Container reflex_app_tfg Started 0.5s
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> |
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker-compose stop
[+] stop 2/2
✓Container reflex_app_tfg Stopped 1.4s
✓Container postgres_tfg Stopped 0.4s
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main>

```

Figura E.4: Consola mostrando el arranque y detención correcto.

- **Mediante interfaz gráfica (GUI):** Abriendo la aplicación *Docker Desktop*, ubicando el contenedor correspondiente al proyecto en la pestaña *Containers* y utilizando los botones de acción integrados (*Play* para arrancar y *Stop* para detener). En la siguiente imagen se muestran ambos botones E.5.
2. **Apagado de Mantenimiento (Destrucción de Contenedores):** Si se necesita hacer una limpieza profunda del sistema, liberar memoria en el servidor o aplicar actualizaciones estructurales, se debe proceder a la destrucción de los contenedores actuales (los datos clínicos de los pacientes no se pierden, ya que están blindados en un volumen

Figura E.5: Botones en la interfaz de *Docker Desktop*.

persistente). En la siguiente imagen se muestra la destrucción y el reinicio E.6.

- Ejecutar el comando: `docker-compose down`.
- Al usar este comando, para volver a encender el servidor será necesario ejecutar `docker-compose up -d` para que el sistema vuelva a construir los contenedores a partir de los datos guardados.

```
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker-compose down
[+] down 3/3
✓Container reflex_app_tfg Removed 3.0s
✓Container postgres_tfg Removed 0.6s
✓Network tfg-2026-nicolas-garcia-gomez-main_default Removed 0.2s
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main>
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main> docker-compose up -d
[+] up 3/3
✓Network tfg-2026-nicolas-garcia-gomez-main_default Created 0.1s
✓Container postgres_tfg Started 0.8s
✓Container reflex_app_tfg Started 1.1s
PS C:\Users\NICO\Downloads\TFG-2026-Nicolas-Garcia-Gomez-main>
```

Figura E.6: Borrado y reinicio de los contenedores.

Acceso Remoto mediante Red Local (LAN)

La arquitectura de red del proyecto permite que la aplicación actúe como un nodo centralizado, posibilitando el acceso desde cualquier dispositivo externo (como pueden ser otros ordenadores, *tablets* o dispositivos móviles

del personal médico) que se encuentre conectado a la misma red local que el servidor principal. Para establecer esta conexión:

1. Se debe identificar la dirección *IPv4* local del equipo que aloja el servidor (ejecutando el comando `ipconfig` en la terminal de *Windows*, o `ifconfig` en sistemas *Unix/Linux*).
2. En el navegador web de cualquier dispositivo cliente de la red, se introducirá dicha dirección *IP* seguida del puerto de exposición de la aplicación. Suponiendo, a modo de ejemplo, que la *IP* del servidor sea 192.168.1.45, el acceso se realizará a través de la URL: `http://192.168.1.45:3000`.

Cabe destacar que, debido a las políticas de seguridad restrictivas de los sistemas operativos modernos (basadas en arquitecturas *Zero Trust*), el cortafuegos o *firewall* del equipo anfitrión bloquea por defecto cualquier intento de conexión entrante que no utilice puertos estándar. Por consiguiente, si no se realiza una configuración adicional, los dispositivos cliente de la red local obtendrán un error de tiempo de espera (*timeout*) al intentar acceder a la dirección *IP* del servidor.

Para que el acceso a la herramienta sea efectivo, resulta indispensable configurar el perímetro de seguridad del nodo principal. En el siguiente vídeo **Configuración Firewall**, se documenta paso a paso la resolución técnica de este bloqueo, ilustrando el procedimiento necesario para transformar un entorno local hermético en un servidor accesible.

En la demostración práctica se detalla la configuración del *Firewall* de *Windows* con seguridad avanzada, donde se establece una nueva **Regla de Entrada**. Esta directiva se configura para autorizar explícitamente el tráfico de red bajo el protocolo *TCP* dirigido al puerto local 3000. Esta apertura perimetral, específica y controlada, es el requisito fundamental que habilita la comunicación bidireccional segura entre los dispositivos del hospital y el contenedor *Docker* que aloja la aplicación principal.

Control de Ingesta Volátil (Archivos *CSV*)

Para el análisis de sesiones clínicas aisladas, el sistema permite la carga de archivos exportados. Para evitar el desbordamiento de memoria y asegurar el rendimiento, se implementan políticas estrictas de restricción.

Control de Ingesta: Restricción de Formato y Gestión *FIFO*

Dadas las limitaciones visuales, de almacenamiento y de procesamiento del terminal portátil o equipo de la *URCCPQ*, el sistema implementa una política estricta de gestión de archivos para evitar el desbordamiento de memoria y asegurar el rendimiento.

- **Objetivo de la prueba:** Verificar que el sistema solo admite archivos con extensión *.csv* y que gestiona un máximo de 10 registros simultáneos mediante una cola tipo *FIFO* (*First-In, First-Out*).
- **Ejecución y Resultado:**
 1. Al intentar cargar un archivo de formato distinto (ej. *.pdf* o *.xlsx*), el sistema rechaza la entrada y emite un mensaje de error por pantalla (Figura E.7).

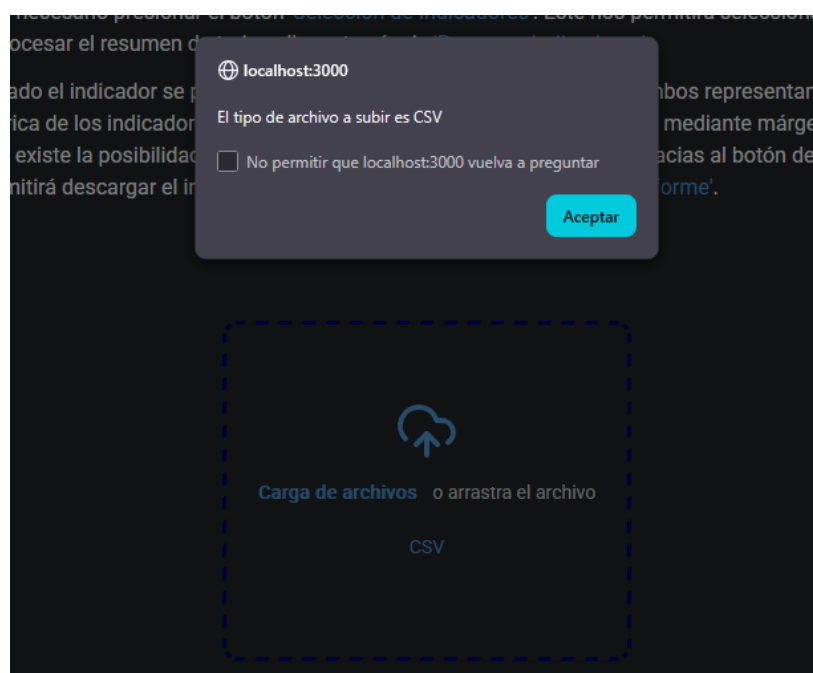
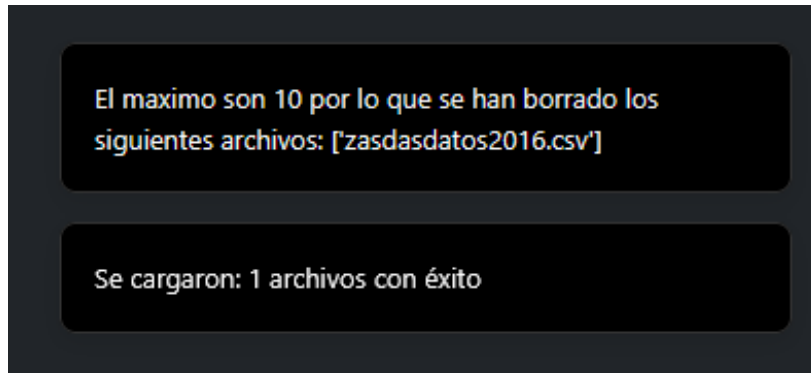


Figura E.7: Rechazo de formatos no válidos.

2. Al superar el límite de 10 archivos cargados, el sistema elimina automáticamente el registro más antiguo (el primero que entró) para alojar el nuevo. Un mensaje por pantalla confirma esta rotación dinámica, garantizando que el usuario siempre disponga de los datos más recientes sin saturar la memoria (Figura E.8).

Figura E.8: Gestión de la cola *FIFO*.

Validación de Estructura: Control de Nomenclatura de Columnas

Cada indicador clínico de la *SEMICYUC* requiere variables específicas. El sistema debe validar que el archivo cargado contiene la información necesaria para realizar los cálculos matemáticos.

- **Objetivo de la prueba:** Garantizar que el motor de procesamiento detecta la ausencia de columnas obligatorias y detiene la ejecución antes de generar un error de sistema.
- **Ejecución y Resultado:** Se procedió a la carga de un archivo *CSV* con cabeceras alteradas. Al seleccionar un indicador para su análisis, el sistema detecta la inconsistencia y proyecta un mensaje específico en pantalla indicando: “Error crítico: Falta la columna “X” en “Archivo”. Esta validación evita resultados nulos o *crashes* inesperados (Figura E.9).

Validación de Transacciones y Gestión de Base de Datos

La incorporación de la base de datos interactiva exige un nivel de validación superior para garantizar la integridad referencial y clínica del historial médico antes de persistirlo en el disco. El formulario de inserción y edición filtra los datos mediante múltiples capas de seguridad:

- **Tipado y Límites Aritméticos:** El sistema bloquea el guardado si el *Nº de Historia* contiene caracteres no enteros E.10. Asimismo,

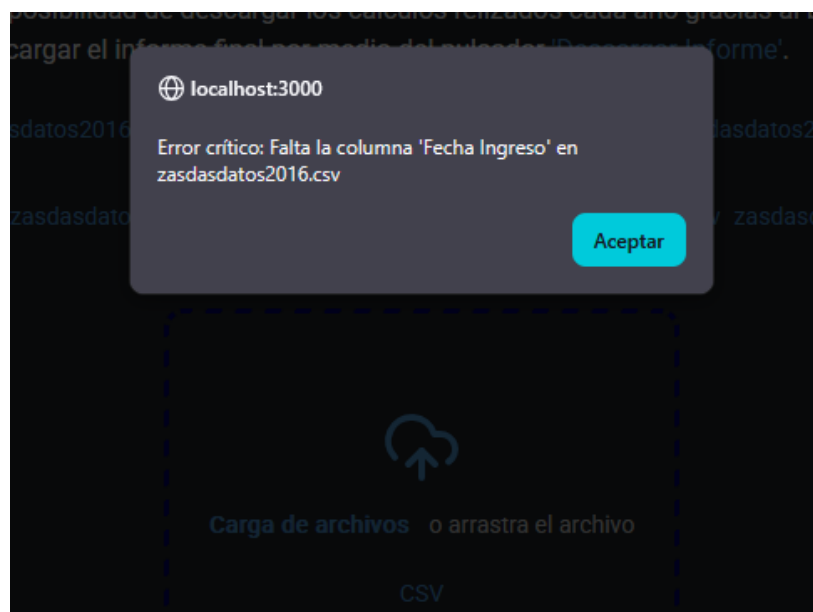


Figura E.9: Interrupción controlada del sistema ante la ausencia de columnas requeridas para el cálculo.

los parámetros biométricos se evalúan aritméticamente, reemplazando comas por puntos en números decimales y abortando la operación si se detectan valores atípicos absurdos (ej. parámetros con valores superiores a 999).

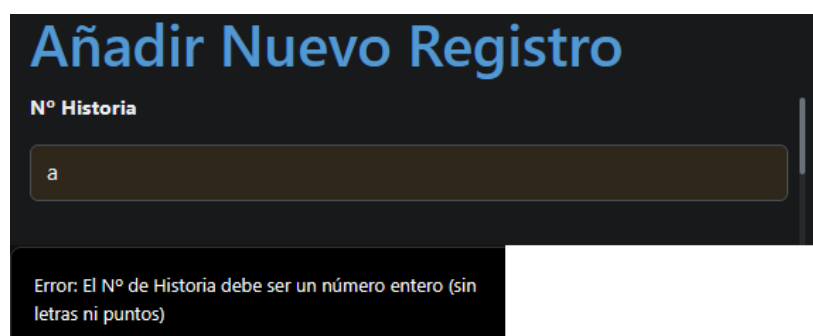


Figura E.10: Número de historia incorrecto.

- **Coherencia Cronológica:** Se emplean algoritmos para evitar imposibilidades clínicas. El sistema asegura que las fechas existan en el calendario (rechazando entradas como “345/12/2026”) y verifica mediante lógica relacional que la fecha de *Alta* no sea anterior a la

de *Ingreso* E.11, o que el inicio de la Nutrición Enteral (*NE*) ocurra dentro de los márgenes de estancia en la unidad.

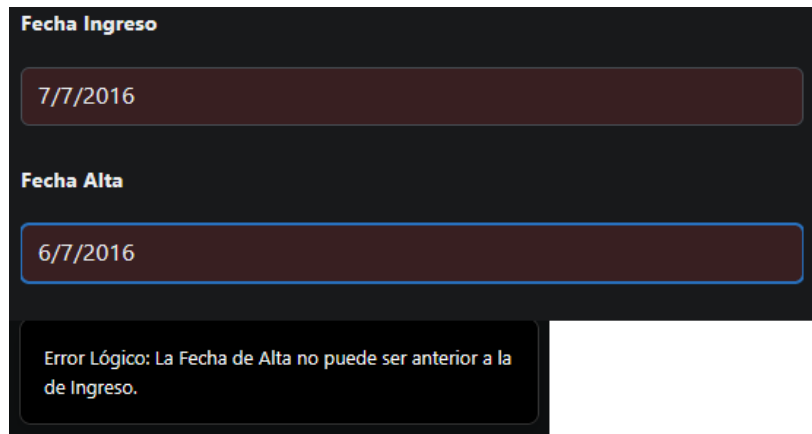


Figura E.11: Fechas introducidas erróneas.

- **Normalización y Limpieza de Cadenas:** Antes de inyectar texto a *PostgreSQL*, el sistema descompone los caracteres, elimina tildes, purga espacios en blanco residuales y capitaliza la primera letra de cada palabra, asegurando que búsquedas posteriores no fallen por errores tipográficos de los facultativos E.12.

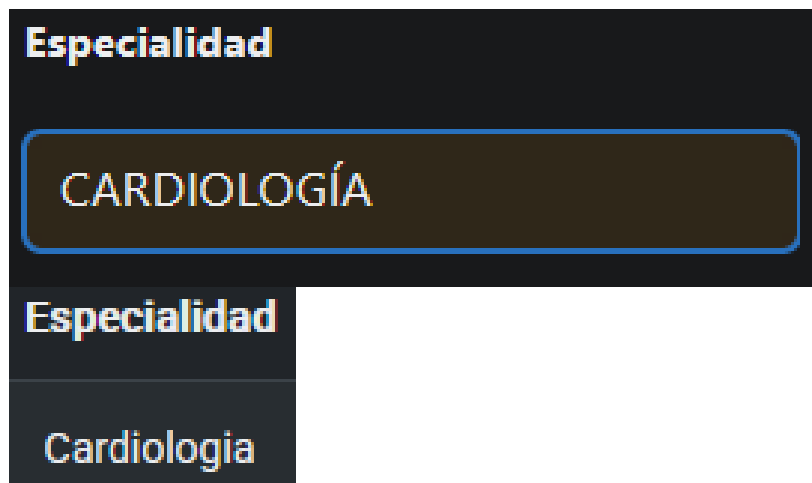


Figura E.12: Cadena de texto corregida.

- **Auditoría y Anonimización:** Todas las transacciones de escritura, borrado o extracción disparan un registro en una tabla de *Auditoría*,

guardando el correo del facultativo y la acción realizada E.13. A su vez, se verificó que la rutina de extracción de datos para investigación en formato *ZIP* elimina automáticamente la columna del *Nº de Historia*, garantizando que los archivos exportados cumplan estrictamente con la protección de datos personales.

105	105	a	BORRAR_PACIENTE	Se eliminó el N° Historia: 1	2026-04-28 14:28:07.566187
106	106	a	EDITAR_PACIENTE	N° Historia editado: 2	2026-04-28 14:28:10.735428
107	107	a	BORRAR_PACIENTE	Se eliminó el N° Historia: 2	2026-04-28 14:28:15.962156
108	108	a	CERRAR_SESION	El usuario cerró sesión	2026-04-28 14:28:19.634895
109	109	a	INICIAR_SESION	El usuario inició sesión	2026-04-29 11:34:03.521943
110	110	a	INICIAR_SESION	El usuario inició sesión	2026-04-30 11:11:41.872777

Figura E.13: Registro de auditoria.

E.3. Instrucciones para la modificación o mejora del proyecto

Tal y como se expone en el apartado de “*Líneas de trabajo futuras*” de la memoria principal, la arquitectura de este sistema ha sido diseñada bajo principios de escalabilidad y alta cohesión. Esto permite que futuras ampliaciones se integren como nuevos módulos sin necesidad de alterar el código base o reestructurar el motor de renderizado de *Reflex*.

A continuación, se proporcionan las directrices técnicas dirigidas a los futuros desarrolladores encargados de continuar la escalabilidad del sistema hacia un entorno hospitalario integral.

Ampliación del Catálogo de Indicadores *SEMICYUC*

Uno de los pilares de la arquitectura de este proyecto es su escalabilidad. El sistema ha sido diseñado de forma modular, permitiendo la adición de nuevos cálculos clínicos sin necesidad de alterar la lógica central del *software*. Para implementar un nuevo indicador, el desarrollador debe seguir un flujo estandarizado descrito a continuación.

Fase 1: Creación del Método Base en el *Backend*

Primero el desarrollador tiene que crear el nuevo método del indicador en el archivo `Programa.py`, constando de cinco pasos:

1. **Definición del método e inicialización de datos E.14:** El proceso comienza con la declaración de un nuevo método dentro de la clase principal (por ejemplo, `def nuevo_indicador(self, ocultar=False):`). Esta primera fase consta de un bloque de código estandarizado que se encarga de iterar sobre los *DataFrames* cargados en memoria (`self._dataframes_memoria`) y aplicar una limpieza preventiva (como la normalización de cadenas booleanas y la eliminación de columnas duplicadas). En este paso, el desarrollador solo debe modificar el nombre de la función.

```
def bacteriemia(self, ocultar = False):
    resultado = self.limpieza()

    try:
        for (dataframe_memoria, nombre) in zip(self._dataframes_memoria, self.nombres_archivos):
            df = dataframe_memoria.copy()
            df = df.replace({"True": True, "true": True, "False": False, "false": False, "None": pd.NA, "" : pd.NA})

            df.columns = [self.normalizar_frame(col) for col in df.columns]
            if df.columns.duplicated().any():
                df = df.loc[:, ~df.columns.duplicated()]
```

Figura E.14: Definición del Método.

2. **Validación de variables clínicas (Control de errores) E.15:** Antes de proceder con cualquier operación matemática, es imperativo garantizar la integridad de los datos. Se debe implementar un bloque condicional que verifique si las columnas necesarias para el cálculo (las variables clínicas) existen en el *DataFrame*. En caso de ausencia, el sistema debe levantar una excepción controlada (`raise ValueError`) informando al usuario de la variable y el archivo donde radica el problema.

```
if "NUMERO_EPISODIOS_BACTERIEMIA" not in df.columns:
    raise ValueError(f"Falta la columna 'Numero Episodios Bacteriemia' en {nombre}")

if "NUMERO_TOTAL_DE_DIAS_CVC" not in df.columns:
    raise ValueError(f"Falta la columna 'Numero total de días CVC' en {nombre}")
```

Figura E.15: Validación de Variables Clínicas.

3. **Lógica de cálculo del indicador E.16:** En esta etapa se programa el núcleo matemático del indicador. Las series de datos extraídas se

fuerzan a un formato numérico seguro mediante herramientas como `pd.to_numeric` (utilizando la coerción de errores para evitar fallos de ejecución por caracteres anómalos). A continuación, se aplican las operaciones necesarias (sumatorios, medias, ratios por cada 1000 días, etc.) y se añade el valor resultante a la lista global de resultados del método.

```
#Logica de calculo
bacteriemia = pd.to_numeric(df["NUMERO_EPISODIOS_BACTERIEMIA"], errors="coerce")
cvc = pd.to_numeric(df["NUMERO_TOTAL_DE_DIAS_CVC"], errors="coerce")

#Numero bacteriemia
num_bacteriemia = bacteriemia.sum()
#Numero de dias CVS
num_cvc = cvc.sum()

valor_final = (num_bacteriemia/num_cvc)*1000 if num_cvc != 0 else 0.0
resultado.append(float(valor_final))
```

Figura E.16: Cálculo del Indicador.

4. **Configuración de la exportación de datos (CSV) E.17:** Para garantizar la trazabilidad clínica y permitir auditorías posteriores, se debe estructurar la salida de datos. El desarrollador debe crear un diccionario (`data_resumen`) que relacione los nombres de las variables con sus valores calculados. Finalmente, se invoca al método de exportación (`self.csv_metodo`), pasándole los metadatos necesarios para que la plataforma genere automáticamente el archivo descargable con el desglose de las operaciones.
5. **Renderizado final y documentación clínica E.18:** El último paso consiste en encapsular toda la lógica anterior en un bloque `try-except`. Si se captura el `ValueError` definido en el segundo paso, se renderiza una alerta visual en la interfaz del usuario (`rx.window_alert`). Si el proceso es exitoso, se concluye llamando al método `self.final`, al cual se le inyectan los resultados numéricos obtenidos y una cadena de texto descriptiva. Esta descripción contiene la definición clínica formal del indicador y su impacto, siendo el texto que el facultativo leerá en la interfaz *web*.

```

data_resumen = {
    "EPISODIOS_BACTERIEMIA": ["RESUMEN GLOBAL bacteriemia relacionada a CVC"],
    "NUMERO_EPISODIOS_BACTERIEMIA": [f"Total: {num_bacteriemia}"],
    "NUMERO_TOTAL_DE_DIAS_CVC": [f"Total: {num_cvc}"],
    "INDICADOR_FINAL": [f"Total: {float(valor_final)}"]
}
self.csv_metodo(
    data_resumen, ["NUMERO_EPISODIOS_BACTERIEMIA", "NUMERO_TOTAL_DE_DIAS_CVC"], [bacteriemia, cvc],
    f"bacteriemia_CVC_{self.encontrar_año(nombre)}.csv"
)

```

Figura E.17: Exportación de los Datos.

```

except ValueError as e:
    return rx.window_alert(f"Error critico: {e}")

self.final(resultado, "Bacteriemia relacionada a CVC:Mide los episodios de infección del torrente sanguíneo " \
    "relacionados con el uso de catéteres venosos centrales. Es una de las complicaciones más importantes, " \
    "asociada a un incremento en la mortalidad y días de estancia.", ocultar=ocultar)

```

Figura E.18: Renderizado Final.

Fase 2: Integración en el Resumen Global e Informes

Para que el nuevo indicador forme parte del análisis global y se incluya automáticamente en la exportación masiva y en el informe en formato *PDF*, el desarrollador debe registrarlo en el método encargado de la consolidación de resultados, denominado `tabla_resumen`.

Tal y como se establece en la lógica de dicho método, el sistema automatiza la ejecución en bloque de todos los indicadores clínicos. Para integrar el nuevo cálculo en este ciclo, únicamente es necesario modificar dos estructuras de control [E.19](#):

- **Estructura de metadatos (`data_resumen`):** Se debe añadir el nombre descriptivo del nuevo indicador al final de la lista asociada a la clave "RESUMEN_INDICADORES". Esta cadena de texto será la empleada para identificar la métrica en la tabla visual y en la columna del archivo *CSV* resumen.
- **Lista de ejecución (`claves`):** Se debe incorporar la referencia al método recién creado (por ejemplo, `self.nuevo_indicador`) dentro de esta lista. El motor de la aplicación itera sobre esta estructura, invocando y ejecutando cada función de manera silenciosa mediante

```
#Definimos la columna de indicadores
data_resumen = {'RESUMEN_INDICADORES': ["Mortalidad Estandarizada", "Reingresos no programados", "Incidencia de barotrauma",
    "Posicion semiincorporada con VMI", "Incidencias Úlcera por presión", "Valoración diaria de la interrupción de la sedación",
    "Prevención de la enfermedad tromboembólica", "Mantenimiento de niveles de glucemia", "Resucitación precoz de la sepsis",
    "Traslado intrahospitalario", "Tratamiento empírico adecuado en infección", "Neumonía asociada a ventilación mecánica",
    "Reintubación", "Profilaxis de la úlcera por estrés en enfermos con NE", "Sedación adecuada", "Ingresos urgentes",
    "Eventos adversos durante el traslado intrahospitalario", "Nutrición enteral precoz",
    "Sobrettransfusión de concentrados de hematies", "Retirada accidental del tubo endotraqueal", "Bacteriemia relacionada a CVC"]}

df_resumen = pd.DataFrame(data_resumen)

#Crea una lista de listas. Cada sublista comienza con el año, se usa para acumular los resultados de ese año específico
lista_valores = [[self.encontrar_año(nombre)] for nombre in self.nombres_archivos]

claves = [self.mortalidad_estandarizada, self.reingresos_no_programados, self.incidencia_de_barotrauma, self.posicion_semiincorporada_vmi,
    self.incidencia_ulceras_presion, self.valoracion_interrupcion_sedacion, self.prevision_enfermedad_tromboembolica, self.mantenimiento_niveles_glucemia,
    self.resucitacion_precoz_sepsis, self.traslado_intrahospitalario, self.tratamiento_empirico_infeccion, self.neumonia_asociada_vmi,
    self.reintubacion, self.profilaxis_ulcera_enfermos_NE, self.sedacion_adecuada, self.ingresos_urgentes, self.adversos_traslado, self.ne_precoz,
    self.sobrettransfusion_hematies, self.retirada_accidental, self.bacteriemia]
```

Figura E.19: Inclusión en el Resumen.

el parámetro `ocultar = True`. Esto permite extraer y recopilar sus valores estadísticos en segundo plano sin alterar la interfaz gráfica del facultativo.

Con la adición de estos dos únicos elementos, la arquitectura del sistema vincula dinámicamente el nuevo cálculo al flujo principal de la plataforma, garantizando su disponibilidad y trazabilidad en el informe clínico unificado.

Fase 3: Integración en el Módulo de Análisis Personalizado (Mezcla)

Si se desea que el nuevo indicador esté disponible en el módulo de análisis combinado (conocido internamente como *mezcla*), el cual permite al facultativo seleccionar y cruzar diferentes gráficas en una misma vista, es necesario registrar el indicador en dos archivos distintos que separan la lógica del renderizado visual:

- **Maapeo lógico en Programa.py:** Dentro de este archivo, que gestiona el *backend* de la aplicación, se debe localizar el método `mezcla`. En su interior existe un diccionario denominado `claves`. El desarrollador debe añadir una nueva entrada donde la clave sea el nombre en texto plano del indicador (ej. "Nuevo Indicador Clínico") y el valor sea la referencia a su método correspondiente (ej. `self.nuevo_indicador`). Este diccionario actúa como un enrutador interno que asocia la selección del usuario con la ejecución del cálculo [E.20](#).
- **Renderizado en la interfaz gráfica en mezcla.py:** Para que el indicador sea visible y seleccionable por el usuario en el *frontend*, se

```
#Diccionario con todos los metodos
claves = {"Mortalidad Estandarizada": self.mortalidad_estandarizada, "Reingresos no Programados": self.reingresos_no_programados,
          "Incidencia de Barotrauma": self.incidencia_de_barotrauma, "Posición Semiincorporada con VMI": self.posicion_semiincorporada_VMI,
```

Figura E.20: Método mezcla en Programa.py.

debe modificar el archivo `mezcla.py`. Dentro de la estructura que genera los botones de selección, se encuentra un componente iterador `rx.foreach` E.21. El desarrollador únicamente debe añadir el nombre exacto del indicador (el mismo texto plano utilizado como clave en el paso anterior) a la lista de cadenas de texto que procesa este componente. De este modo, *Reflex* generará automáticamente el elemento visual con los estilos predefinidos de la plataforma.

```
#Generamos las opciones una a una para poder darles estilo
rx.foreach(
    [
        "Mortalidad Estandarizada", "Reingresos no Programados", "Incidencia de Barotrauma",
        "Posición Semiincorporada con VMI", "Incidencias Úlcera por Presión UPP",
```

Figura E.21: Método mezcla en mezcla.py.

Esta separación de responsabilidades entre el mapeo de la lógica (`Programa.py`) y la generación de la vista (`mezcla.py`) obedece al patrón de diseño del *framework*, manteniendo el código limpio y facilitando la inyección de nuevos componentes en la interfaz reactiva.

Fase 4: Accesibilidad Individual en el Menú de Navegación

Para concluir la integración y permitir que el facultativo pueda ejecutar y visualizar el nuevo indicador de forma aislada, es indispensable vincularlo a la interfaz de usuario principal de la aplicación. Esta acción se lleva a cabo en el archivo encargado de renderizar el menú de opciones del *frontend*, denominado `seleccion.py`.

Dentro de este archivo, se ubica el contenedor del menú desplegable definido por el componente `rx.menu.content`. Para incorporar la nueva funcionalidad, el desarrollador debe instanciar un nuevo elemento dentro de este contenedor utilizando `rx.menu.item`, configurando dos parámetros fundamentales E.22:


```
#Define el contenedor de la lista de opciones
rx.menu.content(

    #Opciones del menu
    rx.menu.item("Mortalidad Estandarizada", on_click=Programa.mortalidad_estandarizada(ocultar = False)),
```

Figura E.22: Método `seleccion` en `seleccion.py`.

- **Etiqueta visual:** Una cadena de texto con el nombre clínico del indicador (por ejemplo, "Nuevo Indicador"), que servirá como la opción visible y descriptiva para el usuario.
- **Evento de invocación (`on_click`):** Se debe enlazar este disparador de eventos al método correspondiente alojado en la clase principal del *backend*. Para ello, se pasa como argumento la llamada directa a la función (ej. `Programa.nuevo_indicador(ocultar = False)`).

Es crucial mantener el parámetro `ocultar = False` en esta llamada. A diferencia de la ejecución masiva en segundo plano vista en los resúmenes globales, este booleano instruye al motor de la aplicación para que procese los datos y, acto seguido, renderice las gráficas y estadísticas en la pantalla principal del usuario.

Con la adición de este botón, el nuevo indicador queda plenamente integrado en todas las capas de la arquitectura del *software*: motor de cálculo interno, resúmenes globales, vistas combinadas y consulta individual, dando por finalizado el proceso de extensión de la herramienta.

Fase 5 (Opcional): Renderizado de Gráficos

En caso de que el nuevo indicador requiera un análisis temporal evolutivo a través de la interfaz visual (generado mediante la librería *Matplotlib*), es altamente recomendable configurar la semántica de colores de su gráfica. Para ello, el desarrollador debe dirigirse al método `grafico` alojado en el archivo `Programa.py` E.23 y añadir el nombre de la variable al diccionario de control interno denominado `claves`.

Este diccionario asocia la cadena de texto identificativa del indicador con un valor booleano (`True` o `False`) que define la lógica de renderizado de la línea de tendencia:

- **Valor `True` (Tendencia descendente favorable):** Se asigna a aquellos indicadores clínicos donde una reducción en su valor numérico

```
#Funcion para realizar los graficos con matplotlib
def grafico(self, eje_x, eje_y, texto, media, desviacion, error_tipico, tend, y_tendencia, r2):
    #Creamos el objeto io
    grafico_bytes = io.BytesIO()

    #Claves que determinan el color
    claves = {
        "Mortalidad Estandarizada": True,          #Bajar es bueno (menor mortalidad)
        "Reingresos no Programados": True,          #Bajar es bueno (menor tasa de fracaso al alta)
        "Incidencia de Barotrauma": True,           #Bajar es bueno (complicacion)
    }
```

Figura E.23: Claves para su Renderizado.

absoluto representa una mejora en la calidad asistencial. Bajo esta configuración, el sistema pintará automáticamente los descensos en la gráfica de color verde (positivo) y los incrementos en color rojo (alerta).

- **Valor False (Tendencia ascendente favorable):** Se reserva para indicadores donde un incremento temporal denota un escenario clínico positivo. En este caso, la lógica de colores se invierte en el renderizado de la imagen final.

Mantenimiento, Escalabilidad y Migración de la Base de Datos

En este apartado se detalla el protocolo técnico estandarizado para la adición de nuevas variables clínicas al sistema. Dada la arquitectura basada en *Reflex* y *PostgreSQL*, es imperativo seguir un flujo de trabajo riguroso que garantice la integridad referencial de los datos históricos y la correcta asimilación de los cambios estructurales tanto en la interfaz gráfica como en el despliegue mediante *Docker*. El procedimiento se articula en las tres fases consecutivas descritas a continuación.

Fase 1: Modificación del Esquema Lógico del Modelo (*Modelo.py*)

El primer paso consiste en declarar la nueva variable dentro de la clase *Registro* alojada en el archivo *Modelo.py* E.24, la cual hereda de *rx.Model* y actúa como el mapeador objeto-relacional (*ORM*).

La nomenclatura del nuevo campo debe ajustarse estrictamente a la convención **snake_case**¹. Asimismo, es un requisito arquitectónico ineludible

¹Convención de escritura que consiste en escribir palabras compuestas en minúsculas y separarlas mediante un guion bajo (__) en lugar de usar espacios.

```
#Al heredar de rx.Model, Reflex crea automaticamente
class Registro(rx.Model, table=True):
    __tablename__ = "registro"
    #Identificador unico (Obligatorio)
    num_historia: str

    #Fechas y Datos de Ingreso
    fecha_ingreso: Optional[str] = None
    fecha_alta: Optional[str] = None
```

Figura E.24: Tabla Registro en el archivo Modelo.py.

que la variable sea declarada mediante el tipado `Optional` y se le asigne el valor por defecto `None`. Esta práctica asegura la retrocompatibilidad del sistema, permitiendo que las consultas a los registros históricos (previos a la creación de este nuevo campo) no generen excepciones por ausencia de datos estructurales.

Fase 2: Configuración del Motor Lógico e Interfaz (BBDD.py)

Una vez actualizado el modelo físico de la base de datos, es necesario modificar el motor lógico para que el *frontend* procese adecuadamente la variable. Esto se logra alterando las listas maestras del Estado (*State*) de *Reflex*.

Mapeo de Variables y Sincronización de Índices: La nueva variable debe ser añadida obligatoriamente en tres listas principales ubicadas en la clase BBDD del archivo BBDD.py. Para evitar desajustes en la renderización de las celdas de la tabla de registros, el nuevo campo **debe ocupar exactamente la misma posición relativa (índice)** en todas ellas:

1. **cabeceras E.25:** Representa el nombre exacto de la columna tal y como consta en la base de datos (formato `snake_case`).

```
#Lista principal con los nombres exactos de la base de datos
cabeceras: list[str] = [
    "num_historia", "fecha_ingreso", "fecha_alta", "reingreso_48",
```

Figura E.25: Lista cabeceras.

2. `cabeceras_display` E.26: Representa el nombre amigable o etiqueta visual que consumirá la interfaz gráfica (utilizando *Title Case*² y respetando la acentuación ortográfica).

```
#Lista secundaria con abreviaturas legibles exclusivamente para la interfaz grafica
cabeceras_display: list[str] = [
    "N° Historia", "Fecha Ingreso", "Fecha Alta", "Reingreso 48h", "Especialidad",
```

Figura E.26: Lista cabeceras_display.

3. `cabeceras_exportacion` E.27: Define la cabecera exigida por los *scripts* analíticos construidos sobre *Pandas* al generar archivos *CSV* o volcados a unidades extraíbles. Son los nombres utilizados para el cálculo en el archivo `Programa.py`.

```
#Lista terciaria exclusiva para exportar los CSV con los nombres que exige el script de Pandas
cabeceras_exportacion: list[str] = [
    "Num Historia", "Fecha Ingreso", "Fecha Alta", "Reingreso 48", "Especialidad de ingreso",
```

Figura E.27: Lista cabeceras_exportacion.

Seguidamente, la variable utilizada en la lista `cabeceras_display` debe ser incluida en la lista `orden_formulario` E.28, lo cual dictará su posición visual exacta en el formulario modal dinámico de inserción y edición de pacientes.

Clasificación de Tipos para Renderizado Dinámico: Para que la función del *frontend* denominada `renderizar_campo_formulario` sea capaz

²La mayúscula inicial o mayúscula inicial es un estilo de mayúsculas utilizado para representar los títulos de obras publicadas o de arte en inglés.

```
#Lista exclusiva para ordenar visualmente el formulario de añadir paciente
orden_formulario: list[str] = [
    "N° Historia", "Fecha Ingreso", "Fecha Alta", "Reingreso 48h", "Barotrauma",
    "VMI", "Días VMI", "RASS Objetivo", "RASS", "BIS Objetivo", "BIS",
```

Figura E.28: Lista orden_formulario.

de pintar el componente adecuado (*input* de texto, numérico o *select*) y valide correctamente las restricciones, el parámetro debe clasificarse (utilizando la misma cadena de texto que en la lista `cabeceras_display`) incluyéndolo en una de las siguientes listas de control de tipos. Si la variable es texto libre, no es necesario incluirla en ninguna de estas listas; el sistema generará un *input* de texto genérico por defecto:

- `campos_display_fecha` o `campos_display_fecha_multiple` E.29: Utilizadas para gestionar la inclusión de fechas o conjunto de fechas.

```
#Lista de control del formulario
campos_display_fecha: list[str] = ["Fecha Ingreso", "Fecha Alta", "Fecha Inicio NE"]

campos_display_fecha_multiple: list[str] = ["Registro Intubación"]
```

Figura E.29: Lista campos_display_fecha y campos_display_fecha_multiple.

- `campos_display_numerico` E.30: Contiene todos los campos que se basan en cifras numéricas.

```
campos_display_numerico: list[str] = [
    "APACHE", "Días VMI", "Grados Inclinación", "PAM Ingreso", "PAM 6h", "PAM 24h",
    "Diuresis Ingreso", "Diuresis 6h", "Diuresis 24h", "Lactato Ingreso", "Lactato 6h",
    "Lactato 24h", "Bacteriemia", "Días CVC", "RASS", "BIS", "RASS Objetivo", "BIS Objetivo",
    "Ventana Sedación", "Glucemia", "Cant. Transfusiones"
]
```

Figura E.30: Lista campos_display_numerico.

- `campos_display_booleano` E.31: Almacena los registros booleanos.

```
campos_display_booleano: list[str] = [
    "Reingreso 48h", "Ingreso Urgencia", "Fallecimiento", "VMI", "Barotrauma", "Episodios NAV",
    "Extub. Programada", "Extub. Maniobras", "Sepsis", "Shock Séptico", "Trat. Adecuado",
    "Cobertura Empírica", "Resist. Antibióticos", "Omeprazol", "Trat. Corticoides",
    "Hemorragia GI", "Coagulopatía", "Insuf. Renal", "Insuf. Hepática", "UPP", "Profilaxis TVP",
    "TVP", "Trat. Insulina", "Traslado Intra.", "Listado Verific.", "Eventos Traslado",
    "Transfusiones", "Sangrado Activo"
]
```

Figura E.31: Lista campos_display_booleano.

Restricciones Lógicas en el Procesamiento de Datos: Finalmente, en el método `guardar_nuevo_paciente`, se deben añadir las comprobaciones lógicas oportunas. Por defecto el sistema valida el tipo numérico del número de historia [E.32](#), la validez y sentido lógico de las fechas, comprueba el tipo de las variables numéricas y las limita a no superar el máximo de 1000 unidades. Al añadir nuevas columnas, será decisión del desarrollador incluir las verificaciones que considere oportunas.

```
#1. Validacion de N° Historia
n_historia_raw = str(self.nuevo_paciente_dict.get("N° Historia", "")).strip()
if n_historia_raw == "":
    return rx.toast("Error: El N° de Historia es obligatorio")
try:
    #Si tiene letras o decimales fallara
    self.nuevo_paciente_dict["N° Historia"] = int(n_historia_raw)
except ValueError:
    return rx.toast("Error: El N° de Historia debe ser un número entero (sin letras ni puntos)")
```

Figura E.32: Verificación N° Historia.

Fase 3: Ciclo de Migración Estructural Segura de Datos (*Alembic* y *Docker*)

El último eslabón del mantenimiento implica alterar el esquema físico de *PostgreSQL*. Para ello, se utiliza el motor de migraciones *Alembic* (integrado nativamente en *Reflex*), el cual permite inyectar columnas en caliente sin provocar la pérdida de los registros clínicos ya consolidados en el sistema.

Para el procedimiento de despliegue es necesario abrir la terminal del entorno de desarrollo (en el directorio raíz) y ejecutar la siguiente secuencia de comandos:

1. **Generación del *script* automático de migración temporal:** El sistema compara el modelo lógico definido en *Python* con el esquema vivo de la base de datos para registrar las diferencias.

```
reflex db makemigrations --message "Agregada nueva variable clinica"
```
2. **Aplicación física de la migración:** Se consolida el cambio, inyectando la nueva columna en el esquema final de la tabla de *PostgreSQL*.

```
reflex db migrate
```
3. **Reconstrucción y despliegue del contenedor:** Se asimilan los nuevos archivos de migración a nivel de infraestructura reiniciando los servicios orquestados.

```
docker-compose up -d --build
```

Nota técnica de integridad referencial: Al ejecutar exitosamente el ciclo de migración, los registros antiguos (pacientes ingresados con anterioridad al parche) inicializarán automáticamente esta nueva columna con un valor nulo (NULL a nivel de base de datos, mapeado como *None* en *Python*). Este comportamiento por diseño garantiza la absoluta coherencia de los datos y previene caídas del sistema por desajustes posicionales durante la lectura.

Transición a Ingesta Automatizada (Integración con ICCA)

Sustituir el actual sistema de carga manual de registro en la *BBDD* por una conexión en tiempo real con el sistema *ICCA*, el desarrollador deberá modificar el módulo de ingesta de datos respetando la arquitectura de eventos de *Reflex*.

- **Directriz técnica:** Se debe implementar un cliente de interoperabilidad basado en estándares internacionales de salud (*HL7* [HL7,] o *FHIR*). El programador deberá reemplazar la función actual de ingesta por una tarea en segundo plano (*Background Task*³) que realice peticiones *API REST* periódicas (ej. mediante la librería *requests* o *httpx* en *Python*) al servidor *ICCA*, inyectando los datos estructurados directamente en el *DataFrame* del estado global de la aplicación.

³Proceso o rutina que se ejecuta sin necesidad de la interacción directa del usuario, permitiendo que la aplicación principal siga funcionando sin interrupciones.

Implementación de Modelos de *Machine Learning*

La integración de modelos predictivos de la librería *Scikit-Learn* requerirá un manejo cuidadoso de los recursos del servidor para no bloquear la interfaz de usuario durante el procesamiento algorítmico intensivo.

- **Directriz técnica:** El entrenamiento y la inferencia de los modelos (ej. árboles de decisión o regresiones logísticas para predecir alteraciones en los indicadores) deben programarse utilizando la asincronía nativa de *Python* (`async/await` [`async/await`,]). De este modo, mientras el modelo matemático calcula el riesgo del paciente, la interfaz gráfica (*Frontend*) seguirá respondiendo a las interacciones táctiles del facultativo sin sufrir bloqueos (*Thread blocking*⁴).

E.4. Vías de Implantación en la Infraestructura Hospitalaria

La arquitectura del sistema ha sido concebida bajo el paradigma de contenerización (*Docker*), lo que otorga a la plataforma un alto grado de flexibilidad, portabilidad y modularidad. Esta decisión tecnológica permite que la herramienta se integre en la *intranet* del hospital adaptándose a las políticas de seguridad y a los recursos del departamento de informática. Para su paso a producción, se contemplan dos vías principales de implantación operativa:

Vía A: Despliegue Íntegro sobre Infraestructura Dedicada

Este escenario es el más directo y asume la provisión de una **Máquina Virtual (VM)**⁵ limpia y dedicada exclusivamente al proyecto por parte de los servicios informáticos del hospital. Bajo este enfoque, se replica exactamente la arquitectura orquestada y validada durante la fase de desarrollo.

- **Despliegue Técnico:** Se ejecuta el archivo `docker-compose.yml` en su totalidad, levantando de forma simultánea e interconectada el

⁴Concepto de programación concurrente donde un hilo de ejecución se detiene y queda en espera (“bloqueado”) hasta que se cumpla una condición específica

⁵Entorno informático basado en software que emula un ordenador físico, ejecutando su propio sistema operativo y aplicaciones de forma aislada.

contenedor de la aplicación *web* (*Reflex*) y el contenedor del motor relacional (*PostgreSQL*), asegurando la creación de los volúmenes de persistencia (*pgdata*) en la propia máquina.

- **Acceso Clínico:** El departamento técnico asigna una *IP* estática a la *VM* dentro de la red local. Los facultativos de la *URCCPQ* acceden al sistema introduciendo dicha dirección en los navegadores de sus puestos de trabajo (por ejemplo, `http://10.X.X.X:3000`), garantizando un entorno autónomo, aislado y de baja latencia.

Vía B: Integración Desacoplada con la Base de Datos Corporativa

En entornos institucionales con protocolos de centralización de datos estrictos, es frecuente que el hospital exija alojar toda la información clínica en su propio clúster de servidores *PostgreSQL* preexistente para unificar las copias de seguridad (*backups*) y el cumplimiento normativo. La arquitectura del siguiente *GitHub*: [TFG-2026-Nicolas-Garcia-Gomez-Produccion](#). Permite un desacoplamiento nativo del despliegue local satisfaciendo este requisito.

- **Adaptación de la Orquestación:** Se excluye el servicio de base de datos local del archivo `docker-compose.yml`, de forma que la *VM* del proyecto únicamente aloje y procese el contenedor correspondiente al *frontend* y *backend* de la aplicación.
- **Enrutamiento de Datos:** La persistencia se redirige modificando de forma segura el archivo oculto de credenciales (`.env`). Se actualiza la variable `DATABASE_URL` para inyectar la cadena de conexión proporcionada por el hospital. De este modo, la aplicación se comunica de forma transparente a través de la *intranet* con el servidor corporativo, delegando la gestión física del almacenamiento a la infraestructura central de la institución médica.

Apéndice *F*

Descripción de adquisición y tratamiento de datos

F.1. Descripción Formal de los Datos

A diferencia de las versiones preliminares del proyecto basadas en procesamiento efímero y archivos temporales, la arquitectura definitiva implementa un modelo de gestión transaccional (*OLTP*¹) respaldado por una base de datos relacional persistente (*PostgreSQL*).

La plataforma actúa como un repositorio centralizado de ciclo continuo: la información clínica reside de forma segura en el servidor y solo se vuelca a la memoria *RAM* (*In-Memory Analytics*²) el volumen exacto de datos que el facultativo solicita para su análisis estadístico. En lugar de destruir los archivos al finalizar la sesión, el sistema garantiza el estricto cumplimiento de la normativa de protección de datos (*LOPD*) mediante dos capas de seguridad: un **Control de Acceso Basado en Roles (*RBAC*)** que restringe la visualización y edición, y un **algoritmo de anonimización** automática que purga los identificadores personales (como el *Nº de Historia*) siempre que el Nivel 3 autorice una exportación externa.

A continuación, se detalla el ciclo de vida de los datos y su estructura de almacenamiento.

¹Sistema de gestión de bases de datos diseñado para procesar un alto volumen de transacciones cortas, rápidas y simultáneas en tiempo real.

²Técnica que permite procesar y analizar grandes volúmenes de datos directamente en la memoria principal (*RAM*) del sistema, en lugar de depender de los discos duros o unidades de almacenamiento tradicionales.

Origen y Gestión de la Información

- **Origen Híbrido:** Los datos que alimentan el sistema provienen de dos vías complementarias. Por un lado, la carga manual de archivos *CSV* proporcionados por la dirección o de exportaciones previas. Por otro, la recolección activa mediante la inserción y edición manual de nuevos pacientes por parte de los médicos (Nivel 2) directamente a través del formulario de la plataforma web.
- **Volumen y Escalabilidad:** La base de datos relacional está diseñada para centralizar múltiples ejercicios clínicos, gestionando un volumen aproximado de 1600 registros (filas) por año. La arquitectura permite escalar el almacenamiento histórico sin degradar el rendimiento del motor analítico de *Python*.
- **Formato y Nomenclatura:** Mientras que el almacenamiento interno se gestiona mediante esquemas relacionales (*SQLModel*), el sistema sigue generando archivos en formato estandarizado *.csv* para las exportaciones anonimizadas (uso en modo portátil). Estas operaciones exigen una concordancia algorítmica estricta en las cabeceras para el correcto mapeo de la información.

Para optimizar el procesamiento analítico y la estructuración en la base de datos, las 54 variables clínicas monitorizadas se han tipificado rigurosamente (fechas, booleanos, números enteros y decimales) y organizado en categorías principales. La descripción técnica detallada de estas 54 variables, incluyendo sus restricciones algorítmicas, se encuentra disponible para su consulta en el Anexo [I.1](#).

Estructura y Tipología de las Variables Clínicas

Cada registro de paciente consta de un máximo de 54 variables clínicas. Durante la fase de ingesta, el motor de *Python* (mediante la librería *Pandas*) tipifica (*casting*³) automáticamente estas variables en cuatro formatos computacionales principales: *Strings* (Texto Plano o Lista) *Booleans* (True/False), *DateTimes* (dd/mm/aaaa) y Numéricos (Enteros y Flotantes).

Para el manejo de valores nulos (pacientes a los que no aplica una métrica), el sistema está programado para interpretar las celdas vacías como

³Proceso de convertir un tipo de dato en otro.

variables NaN (*Not a Number*), evitando sesgos matemáticos que ocurrirían si se rellenaran con ceros.

El conjunto de datos (*Dataset*) se divide en siete dimensiones clínicas:

1. **Datos Generales y Fechas:** Control del flujo de estancias, incluyendo Fecha de Ingreso, Alta, Reingreso, Especialidad, estado de Fallecimiento y puntuación de gravedad *APACHE*.
2. **Ventilación Mecánica y Soporte Respiratorio:** Cuantificación de días de *VMI*, episodios de Barotrauma, Neumonía Asociada a Ventilación Mecánica (*NAV*) y registros de extubación programada o accidental.
3. **Sepsis y Monitorización Hemodinámica:** Variables evolutivas críticas registradas al ingreso, a las 6 horas y a las 24 horas, incluyendo la Presión Arterial Media (*PAM*), volumen de Diuresis y niveles de Lactato.
4. **Infectología:** Control del tratamiento antibiótico empírico, su adecuación, la aparición de resistencias, episodios de bacteriemia y días con catéter venoso central.
5. **Sedación y Neurología:** Comparativa entre las escalas neurológicas prescritas por el facultativo (*RASS* objetivo, *BIS* objetivo) y las métricas reales, así como la interrupción diaria de la sedación.
6. **Nutrición y Cuidados Gastrointestinales:** Fechas de inicio de nutrición enteral y control de complicaciones como hemorragias e insuficiencias orgánicas.
7. **Seguridad y Otros Cuidados:** Chequeos de seguridad orientados a la calidad asistencial, abarcando profilaxis de trombosis (*TVP*), aparición de úlceras (*UPP*), control de glucemia y seguridad en los traslados intrahospitalarios.

Datos de Salida (Exportación y Persistencia)

Aunque el sistema garantiza el almacenamiento persistente y centralizado de los registros en la base de datos, su verdadero valor analítico reside en la transformación de esta información bruta en conocimiento clínico aplicable y transportable. Dependiendo de los requerimientos de la sesión y del nivel de acceso del usuario, la plataforma proporciona cuatro vías de exportación estructurada:

- **Exportación A (Resultados Analíticos):** Un archivo *.csv* que contiene exclusivamente los datos que el algoritmo ha filtrado y utilizado para un cálculo específico, junto con los resultados numéricos de las tasas *SEMICYUC*. Ideal para auditorías internas de un indicador concreto.
- **Exportación B (Resumen de Cohorte):** Un archivo *.csv* tabular que condensa las métricas agregadas de todos los registros o años cargados en la sesión actual, ofreciendo una vista panorámica del rendimiento de la unidad.
- **Exportación C (Informe Visual Integral):** Un documento empaquetado en formato *.pdf*. Constituye el reporte final, integrando la consolidación de los datos analizados junto con la renderización de los gráficos interactivos (lineal y de barras) generados en la interfaz de usuario, listo para ser adjuntado a las actas del hospital.
- **Exportación D (Extracción Histórica Anonimizada):** Generación de un archivo comprimido (*.zip*) que agrupa el histórico clínico en formato *.csv* clasificado por años. Durante la extracción, el motor purga automáticamente los identificadores personales (*Nº de Historia Clínica*). Esta función es exclusiva del nivel de administración (Nivel 3) y está diseñada específicamente para volcar datos seguros que serán analizados de forma descentralizada y autónoma mediante el dispositivo portátil (*Raspberry Pi*), garantizando el cumplimiento normativo fuera de la *intranet*.

F.2. Descripción Clínica de los Datos

En el entorno de los cuidados críticos y la reanimación, los datos monitorizados trascienden la simple lectura de constantes vitales; constituyen un reflejo directo del estado fisiológico del paciente, de la agresividad de los tratamientos de soporte vital y, fundamentalmente, de la calidad y seguridad de la asistencia prestada.

Significado clínico de las variables estructuradas

La información procesada por la plataforma no se analiza de forma aislada, sino que conforma la base para el cálculo de los **Indicadores de Calidad de la Sociedad Española de Medicina Intensiva, Crítica y Unidades Coronarias** (*SEMICYUC*). A continuación, se enumeran los

22 indicadores clínicos y de gestión procesados por el motor analítico de la herramienta, fundamentados en los estándares de calidad y seguridad de la SEMICYUC [Delgado, 2017]

- Mortalidad Estandarizada
- Reingresos no programados
- Incidencia de barotrauma
- Posición semiincorporada con VMI
- Incidencias de úlcera por presión (UPP)
- Valoración diaria de la interrupción de la sedación
- Prevención de la enfermedad tromboembólica
- Mantenimiento de niveles de glucemia
- Resucitación precoz de la sepsis
- Traslado intrahospitalario
- Tratamiento empírico adecuado en infección
- Neumonía asociada a ventilación mecánica
- Reintubación
- Especialidades con mayores ingresos
- Profilaxis de la úlcera por estrés en enfermos con NE
- Sedación adecuada
- Ingresos urgentes
- Eventos adversos durante el traslado intrahospitalario
- Nutrición enteral precoz
- Sobretransfusión de concentrados de hematíes
- Retirada accidental del tubo endotraqueal
- Bacteriemia relacionada a CVC

Clínicamente, las dimensiones de datos empleadas tienen el siguiente significado:

- **Soporte Respiratorio (*VMI* y *NAV*):** La necesidad de ventilación mecánica invasiva (*VMI*) es uno de los factores que mayor riesgo de morbilidad⁴ aporta [Bosch Costafreda et al., 2014]. El registro de los días de ventilación y de episodios como la neumonía asociada a la ventilación (*NAV*) o los barotraumas permite evaluar la eficacia de los protocolos de “destete” (*weaning*⁵) y la protección pulmonar del paciente.
- **Hemodinámica y Sepsis (*Lactato* y *PAM*):** Variables evolutivas como la presión arterial media (*PAM*) o el aclaramiento de lactato a las 6 y 24 horas son marcadores biológicos y hemodinámicos críticos. Un descenso adecuado del lactato indica que la reanimación con fluidos y fármacos vasoactivos está revirtiendo el shock y evitando el fallo multiorgánico. [Procter, 2024]
- **Neuromonitorización (*RASS* y *BIS*):** El ajuste fino de la sedación es vital. Los datos comparan el nivel de sedación prescrito (objetivo) con el real. Una sobredosis prolonga innecesariamente la estancia y la ventilación mecánica, mientras que una infradosis eleva el riesgo de extubaciones accidentales (retiro del tubo por el propio paciente) y estrés postraumático. [Ana Yomaira Reina, 2009]
- **Seguridad del Paciente (*UPP* y *TVP*):** El registro de úlceras por presión (*UPP*) o trombosis venosa profunda (*TVP*) actúa como un “termómetro” de la calidad de los cuidados preventivos de enfermería y la adecuada profilaxis farmacológica durante la inmovilización prolongada en la cama de reanimación. [García Fernández et al., 2008]

Relación con intervenciones terapéuticas y gestión de la unidad

Mientras que en el seguimiento en planta los datos se utilizan principalmente para ajustar tratamientos a largo plazo, en la *URCCPQ* la recopilación sistemática de esta información tiene un doble impacto inmediato. Tras

⁴Indicador sanitario que combina las tasas de morbilidad (enfermedad) y mortalidad (muerte) en una población durante un periodo determinado.

⁵Procesos estandarizados para retirar la ventilación mecánica (*VM*) de forma segura y progresiva, evaluando la estabilidad hemodinámica, respiratoria y neurológica del paciente.

el análisis de las necesidades del servicio en el **Hospital Universitario de Burgos (HUBU)**, se confirma que la explotación de estos datos incide positivamente en:

1. **Auditoría Clínica y Mejora Continua:** Los indicadores agregados permiten a la jefatura de servicio evaluar objetivamente si la unidad cumple con los estándares nacionales de excelencia. Por ejemplo, detectar un aumento en la tasa de infecciones por catéter o una baja adecuación en el tratamiento antibiótico empírico dispara automáticamente la revisión de los protocolos de asepsia o las guías de infectología.
2. **Optimización de Recursos y Morbimortalidad:** La visualización de las tasas de reingreso (pacientes que son dados de alta a planta pero empeoran y deben volver a la unidad) o el análisis cruzado de la escala *APACHE* (gravedad al ingreso) con la mortalidad final, proporciona a los facultativos la justificación basada en datos (*Data-Driven*⁶) necesaria para modificar los criterios de alta o ajustar la ratio de personal sanitario por cama.

⁶Situar los datos en el centro de la toma de decisiones estratégicas, operativas y de diseño, superando la intuición.

Apéndice G

Estudio experimental

G.1. Cuaderno de Trabajo y Evolución Experimental

El desarrollo del *software* clínico no ha sido un proceso lineal. Durante la fase de ingeniería, se evaluaron diversas arquitecturas y herramientas que fueron sometidas a pruebas de estrés. En este cuaderno de trabajo se documentan los tres retos técnicos más relevantes del proyecto, detallando tanto las aproximaciones iniciales fallidas como las soluciones definitivas implementadas.

Experimento 1: Motor de Procesamiento (*Spark SQL vs Pandas*)

- **Objetivo:** Implementar un motor analítico capaz de procesar y filtrar grandes volúmenes de historiales médicos de forma ágil.
- **Aproximación inicial (Fallida):** En las primeras iteraciones se intentó adaptar *Apache Spark SQL*. Sin embargo, su despliegue en entornos *Windows* resultó ser excesivamente complejo y poco eficiente para este caso de uso. Requería la configuración de dependencias externas pesadas (*Hadoop, Java*).
- **Solución y Éxito:** Tras analizar el rendimiento, se pivotó hacia el uso de la librería *Pandas* de *Python*. Esta transición eliminó por completo las dependencias de *Java*, permitiendo un procesamiento vectorial

nativo y ultrarrápido, perfectamente adaptado al volumen de datos (*Datasets* de tamaño medio) que maneja la unidad.

Experimento 2: Optimización de *RAM* y Cumplimiento Normativo (*LOPD*)

- **Objetivo:** Ingestar y procesar múltiples registros anuales manteniendo la fluidez de la aplicación web, evitando la degradación del rendimiento por saturación de la memoria *RAM* y garantizando el estricto cumplimiento de la normativa de protección de datos (*LOPD*).
- **Aproximación inicial (Fallida):** Originalmente, para evitar el colapso del sistema, se implementó una arquitectura de almacenamiento temporal. El archivo clínico se escribía físicamente en la carpeta temporal del sistema operativo (*Temp*) para ser leído por el motor de *Pandas* y posteriormente eliminado. Sin embargo, esta estrategia fue auditada y descartada por motivos de seguridad: escribir datos clínicos con identificadores personales en el disco físico supone un riesgo crítico de vulneración de la *LOPD*, ya que un fallo eléctrico o un cierre inesperado del servidor podría dejar archivos residuales expuestos y sin borrar.
- **Solución y Éxito:** Se rediseñó el flujo hacia un modelo de procesamiento exclusivo en memoria (*In-Memory Analytics*) aprovechando la arquitectura subyacente del *framework Reflex*. La causa de la lentitud original no era la capacidad del servidor, sino que el sistema intentaba enviar (*serializar*¹) toda la información bruta al navegador del cliente. La solución técnica consistió en declarar las variables que almacenan los *DataFrames* como variables privadas del *backend* (anteponiendo un guion bajo en su nomenclatura, ej. `_dataframes_memoria`). Esto restringe el volumen de datos exclusivamente a la memoria volátil del servidor aislado, enviando al *frontend* únicamente los resultados matemáticos calculados. Esta optimización resolvió instantáneamente la latencia, eliminó las advertencias del compilador y garantizó la total privacidad de los historiales clínicos.

¹Proceso de convertir un objeto o estructura de datos en un formato (como una secuencia de *bytes* o una cadena de texto) que puede ser fácilmente almacenado, transmitido o reconstruido más tarde.

Experimento 3: Ciberseguridad, Despliegue y Antivirus

- **Objetivo:** Lograr un empaquetado del *software* que permitiera una distribución segura, sencilla y universal en los equipos del hospital.
- **Aproximación inicial (Fallida):**
 1. Se desplegó un prototipo funcional en la nube (*Reflex Cloud*). Sin embargo, bajo la recomendación de la tutorización académica, se descartó de inmediato debido a los estrictos protocolos de la **Ley de Protección de Datos (LOPD)**, que prohíben subir historiales clínicos a servidores externos.
 2. Se optó por un entorno local empaquetando el código en un archivo ejecutable (.exe) mediante *PyInstaller*. Esta vía también fracasó: el Antivirus nativo (*Windows Defender*) identificaba la apertura de puertos locales y el uso de *WebSockets* como una amenaza de red, bloqueando la comunicación entre el *Frontend* y el *Backend*.
 3. Se intentó automatizar el despliegue local mediante archivos de procesamiento por lotes (*scripts .bat* en *Windows*). La intención era ejecutar el entorno de *Python* y levantar el servidor local en el equipo de cada facultativo con un doble clic. No obstante, esta alternativa demostró ser ineficiente y propensa a fallos humanos en el entorno clínico: requería mantener abierta una ventana de terminal de comandos (*CMD*) que los usuarios solían cerrar accidentalmente, provocando la caída inmediata de la aplicación. Además, la ejecución de *scripts* no firmados a menudo era bloqueada por las políticas de seguridad del departamento de informática y mantenía un modelo descentralizado que dificultaba enormemente la actualización y el mantenimiento del *software*.
- **Solución y Éxito:** Se abandonó la compilación de binarios cerrados y la ejecución local en los ordenadores de la unidad. La solución definitiva consistió en migrar hacia una arquitectura de red centralizada basada en la **contenerización mediante Docker**. Al encapsular la aplicación web, las dependencias y la base de datos relacional en contenedores aislados alojados en el servidor central, el facultativo interactúa con el sistema exclusivamente a través de su navegador web estándar. Esta aproximación elimina por completo las interferencias y falsos positivos de *Windows Defender*, ya que el equipo del usuario final no ejecuta

procesos de *Python* en segundo plano ni necesita abrir puertos de comunicación de forma local. Para garantizar el uso autónomo del dispositivo portátil (*Raspberry Pi*) fuera de la red hospitalaria, se implementó un *script* en *Bash* (.sh) que orquesta automáticamente el levantamiento de los servidores en el propio dispositivo, logrando un despliegue *Plug & Play* seguro, robusto y libre de bloqueos.

G.2. Configuración y Parametrización del Sistema

En el contexto de esta plataforma de *In-Memory Analytics*, la parametrización no se refiere al ajuste de hiperparámetros de modelos de aprendizaje automático, sino a la configuración de los umbrales críticos del sistema, las reglas de gestión de memoria y las directivas de seguridad.

La elección de estos valores no ha sido arbitraria, sino que responde a un equilibrio calculado entre las limitaciones de *hardware* del terminal físico, los protocolos clínicos y la normativa legal. A continuación, se detallan los parámetros fundamentales establecidos en el entorno de producción.

Límite de Ingesta (Cola *FIFO*: $N = 10$)

- **Parámetro:** Capacidad máxima de la cola de archivos/registros cargados simultáneamente en memoria.
- **Valor configurado:** 10 archivos.
- **Justificación técnica y clínica:** Dado que cada archivo/registro anual contiene aproximadamente 1600 registros con 54 variables, la transformación algorítmica de estos datos a *DataFrames* de *Pandas* ocupa una porción significativa de la memoria *RAM*. El límite de $N = 10$ previene de forma determinista la saturación de la memoria y la aparición de errores *OOM* (*Out Of Memory*²). A nivel clínico, este umbral es óptimo, ya que permite al facultativo comparar hasta 10 cohortes anuales simultáneas, cubriendo con creces las necesidades de auditoría retrospectiva de la unidad de Reanimación.

²Ocurre cuando el sistema o una aplicación agotan la *RAM* o *VRAM* disponible.

Normalización de Cadenas y Supresión de Diacríticos

- **Parámetro:** Algoritmo de limpieza y normalización de codificación de texto (`unicodedata.normalize`) aplicado durante la fase de ingesta de datos.
- **Valor configurado:** Descomposición canónica *NFD*³, conversión a *ASCII* (con bandera de exclusión `ignore`) y decodificación final a UTF-8.
- **Justificación técnica y clínica:** La información exportada desde los sistemas hospitalarios o introducida de forma manual suele presentar inconsistencias tipográficas (ej. variaciones ortográficas como “Tensión” frente a “Tension”, o el uso de la “ñ”). Si el *software* intentara realizar el mapeo de columnas y el filtrado de variables de forma literal, se producirían errores de clave (*KeyError*) o pérdida de registros de pacientes en el análisis estadístico. Para garantizar la máxima robustez algorítmica, se ha parametrizado esta rutina estricta que elimina los diacríticos (tildes) y estandariza todo el *Dataset* a un alfabeto base (*ASCII*), asegurando que el motor de *Pandas* identifique las variables clínicas de forma inequívoca.

Política Matemática de Valores Nulos (*NaN Handling y Coerción*)

- **Parámetro:** Directivas algorítmicas combinadas (`fillna` y `errors="coerce"`) para el tratamiento de celdas vacías o con tipos de datos anómalos durante el cálculo estadístico con *Pandas*.
- **Valor configurado:** Imputación lógica por defecto (ej. `False` para variables booleanas vacías) y coerción a `NaN` para errores numéricos.
- **Justificación técnica y clínica:** En el análisis de *Datasets* médicos del mundo real, la ausencia de un registro requiere un manejo contextual. Si un programa ignorara ciegamente todos los nulos, perdería datos valiosos. En este sistema, se ha parametrizado una política híbrida:

³Proceso de *Unicode* que separa los caracteres complejos en sus piezas fundamentales, dividiendo, por ejemplo, la letra “á” en la vocal base “a” y el acento “´” como dos elementos independientes.

1. **Imputación segura:** Variables booleanas críticas que llegan vacías se parametrizan automáticamente a **False** mediante `fillna()`.
2. **Coerción a NaN:** Las variables cuantitativas que contienen caracteres inválidos se fuerzan explícitamente a NaN numérico (*Not a Number*) mediante el parámetro `errors="coerce"`. De este modo, al realizar agregaciones matemáticas, la librería *Pandas* excluye esos pacientes defectuosos del denominador, evitando el colapso del cálculo (*Runtime Error*) y asegurando que las tasas *SEMICYUC* no se desvirtúen.

Configuración de Red y *Binding* (*Host* = 0.0.0.0)

- **Parámetro:** Dirección *IP* de escucha del servidor contenedorizado (*Reflex* sobre *Docker*).
- **Valor configurado:** 0.0.0.0 (exposición a la red local) en lugar del bloqueo restrictivo en 127.0.0.1 (*localhost*).
- **Justificación técnica y clínica:** En las versiones preliminares de escritorio, el *binding*⁴ se forzaba exclusivamente en 127.0.0.1 para garantizar que los datos solo fueran accesibles desde el propio terminal físico. Sin embargo, al evolucionar hacia una arquitectura centralizada, es un requisito técnico indispensable configurar el *host* en 0.0.0.0. Esto permite que el contenedor exponga sus puertos (ej. 3000) a la *intranet* del hospital, posibilitando que los facultativos accedan simultáneamente desde cualquier equipo de la *URCCPQ*. La seguridad clínica, que antes dependía del aislamiento físico de red, ahora se garantiza de manera mucho más robusta e institucional mediante la pasarela de autenticación cifrada y el control de acceso basado en roles (*RBAC*), protegiendo la plataforma contra cualquier acceso no autorizado proveniente de la red corporativa.

G.3. Evaluación de Resultados y Usabilidad Clínica

Para validar la viabilidad de la plataforma, el proyecto no solo se ha sometido a pruebas de estrés técnico, sino a una evaluación integral que

⁴Es la acción de configurar un servidor o *socket* (puerto) para vincularlo a una dirección *IP* específica.

contempla tanto la usabilidad de la interfaz como la legibilidad y utilidad clínica de los resultados analíticos generados.

Análisis de Usabilidad (Cuestionario SUS)

Tras una primera interacción completa con el sistema, se solicitó a un perfil externo al desarrollo que evaluara la herramienta mediante la escala de usabilidad del sistema (*System Usability Scale, SUS*). Este estándar industrial consta de 10 ítems valorados mediante una escala *Likert* del 1 (Totalmente en desacuerdo) al 5 (Totalmente de acuerdo).

Para calcular la puntuación objetiva que determina el grado de usabilidad [Brooke, 1995], se normalizaron los valores sumando la puntuación de cada ítem (con un rango de 0 a 4) según su polaridad intrínseca, multiplicando posteriormente el sumatorio por 2.5. En esta evaluación se obtuvieron las siguientes respuestas G.1, G.2 y G.3.

System Usability Scale (SUS)

Muchas gracias por dedicar unos minutos a interactuar con la nueva plataforma clínica. El objetivo de este breve cuestionario es evaluar de forma objetiva la usabilidad y la experiencia de uso del sistema. A continuación, encontrarás 10 rápidas afirmaciones. Por favor, lee cada una e indica tu nivel de acuerdo utilizando la escala proporcionada, donde **1 significa Totalmente en desacuerdo** y **5 significa Totalmente de acuerdo**. No te tomará más de dos minutos y tus respuestas (totalmente anónimas) son fundamentales para ayudarnos a mejorar la herramienta. ¡Muchas gracias por tu tiempo y colaboración!

1-Creo que usaría este sistema con frecuencia. *

1 2 3 4 5
Totalmente en desacuerdo ☐ ☐ ☐ ☐ ☒ Totalmente de acuerdo

2-El sistema parece innecesariamente complejo. *

1 2 3 4 5
Totalmente en desacuerdo ☒ ☐ ☐ ☐ ☐ Totalmente de acuerdo

3-Encuentro que el sistema es fácil de usar. *

1 2 3 4 5
Totalmente en desacuerdo ☐ ☐ ☐ ☐ ☒ Totalmente de acuerdo

Figura G.1: Respuesta SUS.

Dando en consecuencia el siguiente resultado:

$$\text{Puntuación SUS} = ((4+4+4+3+4) + (4+3+4+4+4)) \times 2.5 = 95 / 100$$

Este valor supera de forma sobresaliente el umbral de referencia (68) establecido para considerar que una interfaz es aceptable, confirmando que

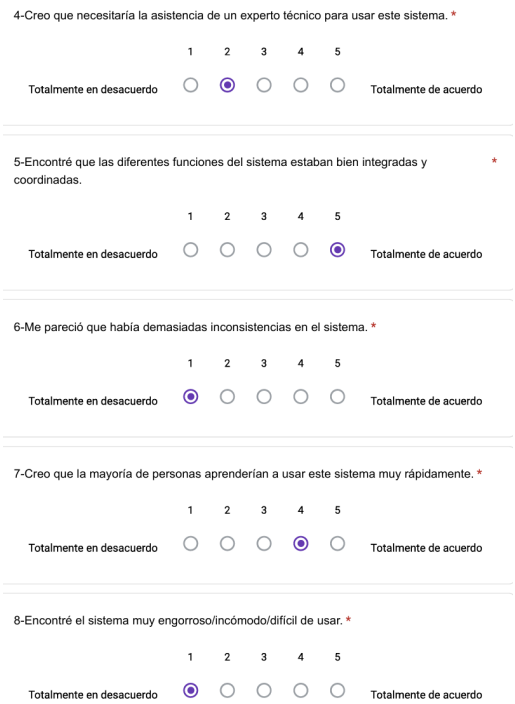


Figura G.2: Respuesta SUS.

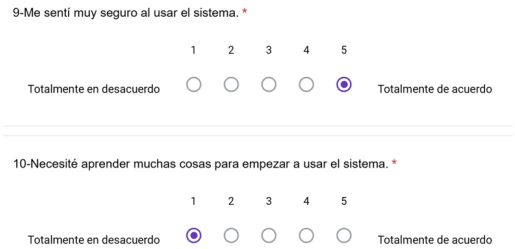


Figura G.3: Respuesta SUS.

la curva de aprendizaje es mínima y que la aplicación web cumple de forma excelente con su propósito en un entorno hospitalario.

Validación Visual y Formatos de Salida

En paralelo a la usabilidad, se validó la correcta representación de las diferentes herramientas de renderizado de la plataforma. La transformación de los datos en formatos visuales legibles es fundamental para agilizar la toma de decisiones del facultativo.

A continuación, se exponen los resultados visuales de los módulos de análisis:

- **Visualización Individual de Indicadores (Figura G.4):** Renderización de la gráfica de tendencias con su respectivo pasillo de confianza estadístico.

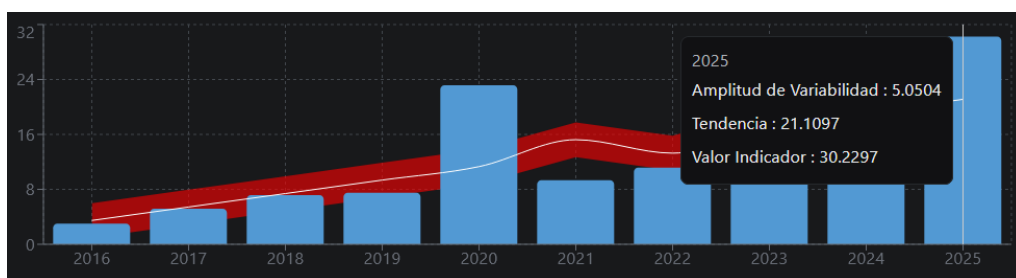


Figura G.4: Gráfico individual.

- **Herramienta de Mezcla Interactiva (Figura G.5):** Ejecución del lienzo de correlación, demostrando la capacidad del sistema para superponer hasta tres indicadores distintos simultáneamente. Esta vista permite a los médicos cruzar variables complejas visualmente.
- **Panel de Resumen Global (Figura G.6):** Condensación de todos los indicadores calculados durante la sesión en formato tabular. Ofrece una radiografía instantánea del estado de la unidad de Reanimación.
- **Generación del Informe Final (Figura G.7):** Consolidación automática de los cálculos, textos descriptivos y gráficos generados en la sesión en un documento estructurado en formato *PDF*. El motor aplica de forma automática la regla de las 2 *Sigmas* para colorear en rojo o verde aquellos eventos centinela que se desvían de la media histórica. Constituye el entregable final del sistema, diseñado para su archivo directo en las actas de calidad del hospital.



Figura G.5: Correlación de tres indicadores.

Resumen Indicadores										
Permite comparar y observar los datos numéricos de manera global de los indicadores de cada año. Concede la capacidad de descargar la tabla en formato CSV de los indicadores o de las especialidades de ingreso, además facilita la descarga del Informe Final, el cual incluye todos los datos así como los gráficos a elección del facultativo.										
RESUMEN_INDICADORES	2016	2017	2018	2019	2020	2022	2021	2023	2024	2025
Mortalidad Estandarizada	4.5233	4.2747	4.4494	4.3858	1.4055	4.0923	4.8846	4.3512	4.4533	2.0967
Reingresos no programados	0	0	0	0	0	0	0	0	0	0.2418
Incidencia de barotrauma	33.579	30.7762	33.3999	32.718	29.7793	33.0932	30.3808	30.6113	30.9866	3.9608
Posicion semiincorporada con VMI	65.9936	66.6615	66.87	66.9489	65.3512	67.9465	65.7593	64.503	67.795	46.2865
Incidenias úlcera por presión	3	5.1667	7.1667	7.5	23.1667	9.3333	11.1667	13.1667	13	30.2297
Valoración diaria de la interrupción de la sedación	44.9087	47.1377	47.5474	48.0966	50.1304	45.1351	48.7836	48.4682	49.855	55.568
Prevención de la enfermedad tromboembólica	16.25	18.0833	16.3333	17.6667	16.9167	16.8333	17.3333	16.3333	16.4167	7.6179
Mantenimiento de niveles de glucemia	30.101	34.6856	36.7793	29.8137	35.6415	35.8238	34.0816	37.3563	30.4904	27.1698

Figura G.6: Resumen tabular de los indicadores calculados.

Año 2016: 4.5233%	Año 2017: 4.2747%	Año 2018: 4.4494%
Año 2019: 4.3858%	Año 2020: 1.4055%	Año 2021: 4.0923%
Año 2022: 4.8846%	Año 2023: 4.3512%	Año 2024: 4.4533%
Año 2025: 2.0967%		

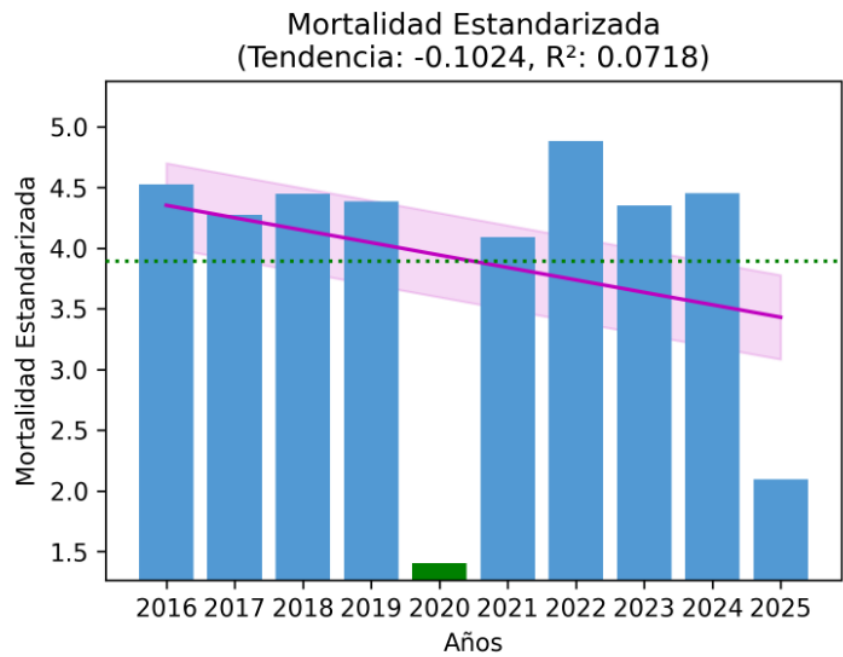


Figura G.7: Gráfico poseedor de la regla de las 2 *Sigmas* incrustado en el informe final.

Anexo de sostenibilización curricular

H.1. Introducción

El desarrollo de este Trabajo de Fin de Grado (*TFG*) ha supuesto una oportunidad inestimable para trascender la aplicación puramente técnica de los conocimientos adquiridos durante mi formación académica, permitiéndome integrar de forma transversal los principios fundamentales de la sostenibilidad. Como ingeniero, es imprescindible comprender que el diseño de *software* y la arquitectura de sistemas de información no son disciplinas neutras; tienen un impacto directo y profundo sobre la sociedad, la economía y el medio ambiente.

En consonancia con las directrices de la **CRUE** (**Conferencia de Rectores de las Universidades Españolas**) para la introducción de la sostenibilidad en el currículum universitario, este proyecto se ha concebido desde una visión holística. Se ha asumido la responsabilidad ética que conlleva el desarrollo de tecnologías aplicadas a un entorno tan crítico como la salud humana. A continuación, se detalla cómo esta plataforma analítica orientada a la *URCCPQ* aborda y satisface las tres dimensiones clásicas del desarrollo sostenible: social, económica y ambiental.

Dimensión Social: Ética, Privacidad y Calidad Asistencial

La dimensión social constituye el pilar central sobre el que se sustenta la motivación de este proyecto. El propósito último del sistema es la

transformación de datos brutos en conocimiento clínico accionable para la mejora continua de la atención al paciente crítico. Al automatizar y facilitar el cálculo de los indicadores de calidad y seguridad de la *SEMICYUC*, la herramienta empodera al personal médico para auditar su propia práctica, identificar áreas de mejora y prevenir complicaciones graves (como neumonías asociadas a ventilación mecánica, úlceras por presión o infecciones por catéter). En última instancia, esta tecnología contribuye directamente a la protección del derecho fundamental a la vida, promoviendo una asistencia sanitaria más segura y equitativa.

Asimismo, el diseño del sistema respeta escrupulosamente los derechos de privacidad y la normativa vigente de protección de datos (*LOPD*). La implementación de un control de acceso basado en roles (*RBAC*), el procesamiento aislado en la *intranet* hospitalaria (sin depender de nubes corporativas externas) y los algoritmos de anonimización para la extracción de datos destinados a la investigación, demuestran un fuerte compromiso ético. Se garantiza que la identidad de los pacientes permanezca protegida, mitigando los riesgos inherentes a la digitalización de historiales médicos.

Por otro lado, el desarrollo de una interfaz *web* intuitiva, operable desde cualquier navegador sin complejas instalaciones previas, promueve la inclusión y la accesibilidad universal para todos los facultativos, independientemente de sus competencias o habilidades tecnológicas previas.

Dimensión Económica: Software Libre y Soberanía Tecnológica

Desde una perspectiva de sostenibilidad económica, el proyecto ha apostado firmemente por la utilización de tecnologías de código abierto (*Open Source*). El empleo de lenguajes y herramientas como *Python*, *Reflex*, *PostgreSQL* y *Docker* elimina las barreras de entrada económicas asociadas a las costosas licencias de *software* privativo (como ocurre con programas estadísticos de pago tipo *SPSS*¹) o a las suscripciones recurrentes en la nube. Esta decisión democratiza el acceso a herramientas analíticas avanzadas, permitiendo que hospitales públicos o unidades clínicas con presupuestos limitados puedan beneficiarse de la innovación sin comprometer sus recursos financieros.

Además, la arquitectura orquestada mediante contenedores garantiza una alta mantenibilidad y viabilidad a largo plazo. El sistema no depende

¹IBM SPSS Statistics

de un proveedor tecnológico específico (*vendor lock-in*²), lo que asegura que el hospital pueda gestionar, replicar, modificar y actualizar la plataforma de forma autónoma. Esto fomenta la soberanía tecnológica de la institución, reduciendo la dependencia externa y los costes de mantenimiento.

Dimensión Ambiental: Eficiencia Energética y Reducción de la Huella Ecológica

Aunque con frecuencia se subestima el impacto ecológico del código informático, este proyecto ha integrado los principios de la computación verde (*Green Computing*³). La decisión de contenerizar la aplicación optimiza al máximo el uso de los recursos del servidor, evitando la necesidad de desplegar múltiples máquinas virtuales pesadas que consumen un exceso de memoria *RAM* y capacidad de procesamiento.

Específicamente en su variante de uso autónomo y portátil, el despliegue del sistema sobre un dispositivo *Raspberry Pi* representa un avance significativo en eficiencia energética. Mientras que un servidor tradicional o un ordenador de sobremesa en un hospital puede requerir un consumo de entre 200 y 400 vatios por hora, la *Raspberry Pi* opera de manera completamente funcional con un consumo inferior a los 5 vatios. Esta drástica reducción de la demanda eléctrica minimiza la huella de carbono asociada al procesamiento de datos.

Por último, la digitalización integral de los informes visuales y de las métricas clínicas de la unidad impulsa la transición institucional hacia una metodología sin papel. La capacidad del sistema para generar automáticamente archivos *PDF* estructurados y compartibles reduce de forma directa el consumo de recursos materiales (papel, tinta, almacenamiento físico) y la generación de residuos ofimáticos en la planta hospitalaria.

Conclusión

La conceptualización y el desarrollo de esta plataforma clínica han evidenciado que la ingeniería de *software*, cuando se alinea con propósitos sociales, posee un potencial transformador excepcional. Este proyecto no se limita a resolver un problema de procesamiento de datos médicos; refleja un compromiso activo con una evolución tecnológica innovadora que es

²Situación en la que una empresa depende tanto de un proveedor tecnológico específico que cambiar a otro resulta extremadamente difícil o costoso.

³Uso eficiente, sostenible y ecológico de los recursos informáticos

éticamente responsable, económicamente viable y respetuosa con el medio ambiente, respondiendo con rigor a las exigencias de la actual crisis climática y a las necesidades de la sociedad contemporánea.

Apéndice I

Registro de interacciones con sistemas de IA

I.1. Registro de interacciones con sistemas de IA

Este anexo tiene como objetivo garantizar la transparencia en el uso de herramientas de inteligencia artificial generativa durante la realización del Trabajo de Fin de Grado. En él se incluye un registro ordenado de las consultas formuladas en las interacciones con sistemas de *IA* que han sido más significativas en relación con el desarrollo del proyecto.

El propósito de este registro es documentar de forma clara el alcance, la finalidad y el impacto real de dichas herramientas en el proceso de aprendizaje, entendiendo su uso como una herramienta de apoyo al desarrollo técnico, la orientación conceptual y la depuración, y en ningún caso como un sustituto del razonamiento ingenieril o la toma de decisiones arquitectónicas.

A continuación, se detallan las interacciones más relevantes:

Interacción 1: Gestión de memoria efímera y borrado de archivos

- **Herramienta o modelo de *IA* utilizado:** *Gemini 3.1 Pro*.
- **Fecha aproximada de uso:** 23 de febrero de 2026.

- **Petición realizada:** “¿Cómo puedo implementar un sistema en *Python* que actúe como un *Watchdog* para monitorizar la desconexión de usuarios en una *app web* y borre automáticamente los archivos *.csv* guardados en la carpeta *Temp* de *Windows*?”
- **Resumen de la respuesta generada:** El modelo generó un bloque de código explicando el uso de la librería `tempfile` y los métodos del módulo `os` para identificar archivos huérfanos por marca de tiempo y purgar directorios temporales basándose en la caída de la conexión del *WebSocket*.
- **Uso posterior en el TFG:** Apoyo técnico y generación de código de prueba. Esta interacción sirvió para construir el primer prototipo de ingesta de datos. Sin embargo, como se documenta en la sección de **Pruebas del Sistema E.2**, este enfoque fue posteriormente **descartado** por motivos de rendimiento y cumplimiento de la *LOPD*, evolucionando hacia una arquitectura de procesamiento enteramente en *RAM* y variables locales que eliminó la necesidad de escribir en el disco físico.

Interacción 2: Integración de Base de Datos y *SQLModel*

- **Herramienta o modelo de IA utilizado:** *Gemini 3.1 Pro*.
- **Fecha aproximada de uso:** 15 de abril de 2026.
- **Petición realizada:** “Quiero migrar mi aplicación de *Reflex* para que use una base de datos relacional persistente en lugar de leer archivos *CSV*. A modo de guía inicial, ¿qué paquetes necesito instalar y cuál es la mejor forma de conectar *PostgreSQL* a *Reflex* usando *SQLModel*?”
- **Resumen de la respuesta generada:** El sistema proporcionó una guía de configuración detallada. Indicó la necesidad de añadir paquetes como `psycopg2` y `sqlmodel`, ofreció un ejemplo sobre cómo configurar la variable de entorno `DATABASE_URL` y mostró un patrón de diseño básico para crear clases de tablas heredando de `rx.Model`.
- **Uso posterior en el TFG:** Orientación conceptual. La respuesta sirvió como mapa de ruta para estructurar la conexión al motor relacional y plantear el diseño del *ORM* (*Object-Relational Mapping*), sentando las bases teóricas sobre las que posteriormente se programó toda la lógica transaccional y la clase central BBDD del proyecto.

Interacción 3: Evolución de las Estrategias de Despliegue (Secuencia de consultas)

Esta interacción engloba una serie de tres consultas sucesivas que reflejan fielmente la evolución arquitectónica del proyecto, desde los intentos iniciales de despliegue local hasta llegar a la infraestructura final contenerizada.

- **Herramienta o modelo de IA utilizado:** *Gemini 3.1 Pro*.
- **Fechas aproximadas de uso:** Entre el 10 de marzo y el 1 de mayo de 2026.
- **Secuencia de Peticiones y Resultados:**

1. Consulta 3.1 - Despliegue local (*Windows*):

- **Petición:** “¿Cómo puedo crear un *script .bat* en *Windows* para activar automáticamente el entorno virtual de *Python* y arrancar mi servidor de *Reflex* con un doble clic?”
- **Respuesta generada:** El modelo proporcionó la sintaxis básica de *Batch* para navegar entre directorios, activar el entorno (`venv\Scripts\activate`) y ejecutar el comando de arranque.
- **Uso posterior:** Generación de código. El *script* fue implementado en las primeras pruebas de usuario, pero fue rápidamente **descartado** debido a una mejora en el despliegue hacia la *intranet* del hospital.

2. Consulta 3.2 - Dispositivo portátil (*Linux/Raspberry Pi*):

- **Petición:** “Necesito adaptar la lógica anterior a un entorno *Linux* para implementarlo en un terminal portátil (*Raspberry Pi*). ¿Cómo estructuro un *script .sh* para levantar la *web*?”
- **Respuesta generada:** Se detallaron los comandos *Bash* necesarios, el uso de la cabecera *shebang* (`#!/bin/bash`).
- **Uso posterior:** Soporte técnico. Esta guía se aplicó con éxito para garantizar el arranque autónomo y *Plug & Play* de la aplicación dentro de la *Raspberry Pi*, posibilitando su uso seguro fuera de la red del hospital.

3. Consulta 3.3 - Infraestructura definitiva (*Docker*):

- **Petición:** “Dame una guía inicial y comandos para contenerizar la aplicación *web* y una base de datos *PostgreSQL* usando *Docker* y `docker-compose.yml`.”
- **Respuesta generada:** El modelo redactó un esqueleto básico para un `Dockerfile` (empleando la imagen `python:3.12-slim`) y un archivo `docker-compose.yml` configurado para orquestar los dos servicios, mapeando los puertos 3000 y 5432.
- **Uso posterior:** Toma de decisiones arquitectónicas y diseño de infraestructura. Esta consulta marcó el punto de inflexión del *TFG*, sirviendo como base estructural para desarrollar la arquitectura final y profesional de despliegue continuo en la *intranet* del hospital.

Diccionario de Variables Clínicas

Tabla I.1: Diccionario Detallado de Variables Clínicas del Sistema.

Nº	Nombre de la Variable	Tipo de Dato	Categoría Clínica
1	Fecha Ingreso	Fecha	Datos Generales y Fechas
2	Fecha Alta	Fecha	Datos Generales y Fechas
3	Reingreso 48	Booleano	Datos Generales y Fechas
4	Especialidad de Ingreso	Texto	Datos Generales y Fechas
5	Ingreso por Urgencia	Booleano	Datos Generales y Fechas
6	Fallecimiento	Booleano	Datos Generales y Fechas
7	APACHE	Entero	Datos Generales y Fechas
8	VMI	Booleano	Ventilación Mecánica
9	Días VMI	Decimal	Ventilación Mecánica
10	Barotrauma	Booleano	Ventilación Mecánica
11	Grados Inclinación	Entero	Ventilación Mecánica
12	Episodios NAV	Booleano	Ventilación Mecánica
13	Registro Intubaciones	Texto (Lista)	Ventilación Mecánica
14	Extubación Programada	Booleano	Ventilación Mecánica
15	Extubación por Maniobras	Booleano	Ventilación Mecánica
16	Sepsis	Booleano	Sepsis y Hemodinámica
17	Shock Séptico	Booleano	Sepsis y Hemodinámica
18	PAM ingreso	Entero	Sepsis y Hemodinámica
19	PAM 6h	Entero	Sepsis y Hemodinámica
20	PAM 24h	Entero	Sepsis y Hemodinámica
21	Diuresis Ingreso	Decimal	Sepsis y Hemodinámica
22	Diuresis 6h	Decimal	Sepsis y Hemodinámica
Continúa en la siguiente página...			

Tabla I.1 – Continuación de la página anterior

Nº	Nombre de la Variable	Tipo de Dato	Categoría Clínica
23	Diuresis 24h	Decimal	Sepsis y Hemodinámica
24	Lactato Ingreso	Decimal	Sepsis y Hemodinámica
25	Lactato 6h	Decimal	Sepsis y Hemodinámica
26	Lactato 24h	Decimal	Sepsis y Hemodinámica
27	Tratamiento adecuado	Booleano	Infectología y Antibióticos
28	Cobertura empírica completa	Booleano	Infectología y Antibióticos
29	Resistencia antibióticos	Booleano	Infectología y Antibióticos
30	Num Episodios Bacteriemia	Entero	Infectología y Antibióticos
31	Num total de días CVC	Decimal	Infectología y Antibióticos
32	RASS	Entero	Sedación y Neurología
33	BIS	Entero	Sedación y Neurología
34	RASS objetivo	Entero	Sedación y Neurología
35	BIS objetivo	Entero	Sedación y Neurología
36	Ventana de sedación	Decimal	Sedación y Neurología
37	Fecha Inicio NE	Fecha	Nutrición y Gastrointestinal
38	Omeprazol	Booleano	Nutrición y Gastrointestinal
39	Tratamiento Corticoides	Booleano	Nutrición y Gastrointestinal
40	Antecedentes Hemorragia GI	Booleano	Nutrición y Gastrointestinal
41	Coagulopatía	Booleano	Nutrición y Gastrointestinal
42	Insuficiencia Renal	Booleano	Nutrición y Gastrointestinal
43	Insuficiencia Hepática	Booleano	Nutrición y Gastrointestinal
44	UPP	Booleano	Seguridad y Otros Cuidados
45	Profilaxis TVP	Booleano	Seguridad y Otros Cuidados
46	TVP	Booleano	Seguridad y Otros Cuidados
47	Glucemia	Decimal	Seguridad y Otros Cuidados
48	Tratamiento Insulina	Booleano	Seguridad y Otros Cuidados
49	Traslado Intrahospitalario	Booleano	Seguridad y Otros Cuidados
50	Listado de verificación	Booleano	Seguridad y Otros Cuidados
51	Eventos Adversos Traslado	Booleano	Seguridad y Otros Cuidados
52	Actos Transfusionales	Booleano	Seguridad y Otros Cuidados
53	Sangrado Activo	Booleano	Seguridad y Otros Cuidados
54	Cantidad Transfusion Dia	Entero	Seguridad y Otros Cuidados

Bibliografía

- [311/2022, 2022] 311/2022, R. D. (2022). Qué es el esquema nacional de seguridad (ens). URL: <https://ens.ccn.cni.es/es/que-es-el-ens>.
- [Amazon,] Amazon. Amazon. URL: <https://www.amazon.es/>.
- [Ana Yomaira Reina, 2009] Ana Yomaira Reina, S. V. R. (2009). Intervención del profesional de enfermería en la aplicación de escalas de sedoanalgesia en el paciente de la unidad de cuidado intensivo. URL: <https://apidspace.javeriana.edu.co/server/api/core/bitstreams/48689f6b-c2a3-4cdf-8f91-0bc4e9572852/content>.
- [async/await,] async/await. Coroutines and tasks. URL: <https://docs.python.org/es/3.13/library/asyncio-task.html>.
- [Biomedica, 2007] Biomedica, I. (14/2007). Ley 14/2007, de 3 de julio, de investigación biomédica. URL: <https://www.boe.es/buscar/pdf/2007/BOE-A-2007-12945-consolidado.pdf>.
- [Bootstrap,] Bootstrap. Build fast, responsive sites with bootstrap. URL: <https://getbootstrap.com/>.
- [Bosch Costafreda et al., 2014] Bosch Costafreda, C., Riera Santiesteban, R., and Badell Pomar, C. (2014). Morbilidad y mortalidad en pacientes con ventilación mecánica invasiva en una unidad de cuidados intensivos. *MEDISAN*, 18:377 – 383. URL: http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S1029-30192014000300012&nrm=iso.
- [Brooke, 1995] Brooke, J. (1995). Sus: A quick and dirty usability scale. URL: https://www.researchgate.net/publication/228593520_SUS_A_quick_and_dirty_usability_scale.

- [CE, 24CE] CE, D. (2009/24/CE). Directiva 2009/24/ce del parlamento europeo y del consejo, de 23 de abril de 2009, sobre la protección jurídica de programas de ordenador (versión codificada). URL: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2009-80808>.
- [Christian Robertson, a] Christian Robertson, ParaType, F. B. Roboto. URL: https://fonts.google.com/specimen/Roboto?preview.lang=es_Latn.
- [Christian Robertson, b] Christian Robertson, ParaType, F. B. Roboto hoja de estilo. URL: <https://fonts.google.com/selection/embed>.
- [de shell,] de shell, S. Script de shell. URL: https://es.wikipedia.org/wiki/Script_de_shell.
- [Decreto, 2010] Decreto, R. (1093/2010). Real decreto 1093/2010, de 3 de septiembre, por el que se aprueba el conjunto mínimo de datos de los informes clínicos en el sistema nacional de salud. URL: <https://www.boe.es/eli/es/rd/2010/09/03/1093>.
- [Decreto, 1996] Decreto, R. (1/1996). Real decreto legislativo 1/1996, de 12 de abril, por el que se aprueba el texto refundido de la ley de propiedad intelectual, regularizando, aclarando y armonizando las disposiciones legales vigentes sobre la materia. URL: <https://www.boe.es/buscar/act.php?id=BOE-A-1996-8930>.
- [Decreto, 2023] Decreto, R. (192/2023). Real decreto 192/2023, de 21 de marzo, por el que se regulan los productos sanitarios. URL: <https://www.boe.es/buscar/doc.php?id=BOE-A-2023-7416>.
- [Delgado, 2017] Delgado, M. C. M. (2017). Indicadores de calidad en el enfermo crítico. URL: https://semicyuc.org/wp-content/uploads/2018/10/indicadoresdecalidad2017_semicyuc_spa-1.pdf.
- [García Fernández et al., 2008] García Fernández, F. P., Pancorbo Hidalgo, P. L., Soldevilla Ágreda, J. J., and Blasco García, C. (2008). Escalas de valoración del riesgo de desarrollar úlceras por presión. *Gerokomos*, 19:136 – 144. URL: http://scielo.isciii.es/scielo.php?script=sci_arttext&pid=S1134-928X2008000300005&nrm=iso.
- [HL7,] HL7. Hl7 spain. URL: <https://www.hl7spain.org/>.
- [ING, 2022] ING (2022). ¿qué es la amortización? así puedes calcularla. URL: <https://www.ing.es/ennaranja/finanzas-personales/conceptos-utiles/amortizacion-que-es-como-se-calcula/>.

- [Jacobson, 1992] Jacobson, I. (1992). *Object-Oriented Software Engineering A Use Case Driven Approach*. Addison-Wesley. URL: <https://es.scribd.com/document/551870335/Object-Oriented-Software-Engineering-a-Use-Case-Driven-Approach-by-Ivar-Jacobson-Z-lib-org>.
- [Licence, V 2] Licence, A. (V 2). Apache license, version 2.0. URL: <https://www.apache.org/licenses/LICENSE-2.0>.
- [Mediavilla,] Mediavilla, E. Modelos y herramientas uml. URL: https://www.ctr.unican.es/asignaturas/mc_oo/doc/casos_de_uso.pdf.
- [Nicolás, 2026] Nicolás (2026). Github desarrollado para el proyecto. URL-Local: <https://github.com/ngg1017/TFG-2026-Nicolas-Garcia-Gomez>, URL-Produccion: <https://github.com/ngg1017/TFG-2026-Nicolas-Garcia-Gomez-Produccion>, URL-Portatil: <https://github.com/ngg1017/TFG-2026-Nicolas-Garcia-Gomez-Portatil>.
- [Orgánica, 2018] Orgánica, L. (3/2018). Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. URL: <https://www.boe.es/eli/es/lo/2018/12/05/3/con>.
- [Paciente, 2002] Paciente (41/2002). Ley 41/2002, de 14 de noviembre, básica reguladora de la autonomía del paciente y de derechos y obligaciones en materia de información y documentación clínica. URL: <https://www.boe.es/eli/es/l/2002/11/14/41/con>.
- [Procter, 2024] Procter, L. D. (2024). Reanimación con líquidos intravenosos. URL: <https://www.msdmanuals.com/es/professional/cuidados-cr%C3%ADticos/shock-y-reanimaci%C3%B3n-con-l%C3%ADquidos/reanimaci%C3%B3n-con-l%C3%ADquidos-intravenosos>.
- [shop,] shop, R. P. Raspberry pi shop. URL: <https://www.raspberrypi.com/products/>.
- [Sommerville, 2005] Sommerville, I. (2005). *Ingeniería del software*. Pearson Educación, Madrid, 7 edition. URL: <https://ulagos.wordpress.com/wp-content/uploads/2010/07/ian-sommerville-ingenieria-de-software-7-ed.pdf>.
- [UAX, 2025] UAX (2025). ¿cuál es el salario de un graduado en ingeniería biomédica? URL: <https://www.uax.com/blog/salud/cuanto-cobra-ingeniero-biomedico>.

- [(UE), 679] (UE), R. (2016 /679). Reglamento (ue) 2016/679 del parlamento europeo y del consejo de 27 de abril de 2016. URL: <https://www.boe.es/doue/2016/119/L00001-00088.pdf>.
- [(UE), 7745] (UE), R. (2017/745). Reglamento (ue) 2017/745 del parlamento europeo y del consejo, de 5 de abril de 2017, sobre los productos sanitarios, por el que se modifican la directiva 2001/83/ce, el reglamento (ce) nº 178/2002 y el reglamento (ce) nº 1223/2009 y por el que se derogan las directivas 90/385/cee y 93/42/cee del consejo. URL: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2017-80916>.
- [Vega, 2010] Vega, M. (2010). Casos de uso uml. URL: <https://lsi2.ugr.es/~mvega/docis/casos%20de%20uso.pdf>.
- [Yablonski, 2022] Yablonski, J. (2022). *Las leyes del UX*. Parramón Paidotribo. URL: <https://www.perlego.com/es/book/3767489/las-leyes-del-ux-utilizando-la-psicologa-para-mejorar-la-experiencia-de-usuario-ux-pdf>.
- [YoungWonks,] YoungWonks. Raspberry pi 3 pinout. URL: <https://www.youngwonks.com/blog/Raspberry-Pi-3-Pinout>.